Ministry of Education and Training

Nguyen Tat Thanh University

NTT Institute of International Education

GRADUATION PROJECT

TITLE:

DESIGN OF A GAMIFIED PYTHON SELF-LEARNING SUPPORT SYSTEM ON GOOGLE APPS SCRIPT

PROJECT SUPERVISOR: MSc. Lương Trần Ngọc Khiết

NO.	NAME	STUDENT ID
1	Trương Tường Vi	2200002438
2	Phạm Văn Giàu	2200004133

Ho Chi Minh City, September 2026

Ministry of Education and Training

Nguyen Tat Thanh University

NTT Institute of International Education

GRADUATION PROJECT

TITLE:

DESIGN OF A GAMIFIED PYTHON SELF-LEARNING SUPPORT SYSTEM ON GOOGLE APPS SCRIPT

PROJECT SUPERVISOR: MSc. Lương Trần Ngọc Khiết

NO.	NAME	STUDENT ID
1	Trương Tường Vi	2200002438
2	Phạm Văn Giàu	2200004133

Ho Chi Minh City, September 2026

STUDENT DECLARATION

Student Name and Student ID:

Trương Tường Vi – 2200002438

Phạm Văn Giàu – 2200004133

Supervisor Name: MSc. Lương Trần Ngọc Khiết

We affirm that this project, "Design of a Gamified Python Self-Learning Support System on Google Apps Script" is the result of our own independent research efforts and has been entirely produced by us.

Ho Chi Minh City, September 20, 2026

Supervisor	**Student**
(Name and Signature)	*(Name and Signature)*
MSc. Lương Trần Ngọc Khiết	**Trương Tường Vi**
	Phạm Văn Giàu

MODULE ASSIGNMENT

Person in charge: Phạm Văn Giàu

Responsibilities:

1. **Research and Development of Theoretical Background**

- Study the theoretical background related to online learning systems, personalized learning, artificial intelligence in education, and gamification in learning.
- Review and summarize the theoretical knowledge used as a basis for designing and developing the Python self-learning support system.

2. **Development and Integration of AI Functions**

- Integrate Gemini AI into the system to support the processing and generation of learning content from available learning materials.
- Develop functions to generate Mindmaps.
- Develop functions to generate multiple-choice questions and Mini Quizzes.
- Develop a function to generate term-definition pairs for the Matching Game.

3. **Development and Completion of the Administration System**

- Develop an administration area for Admin to manage the main data and functions of the system.
- Develop functions to manage users and account status.
- Develop functions to manage courses, topics, and lesson content.
- Develop functions to manage Quiz questions, Matching data, and Code Arrangement exercises.
- Develop functions to manage learning reminder emails and Pet data.
- Develop functions to monitor and view statistics about users' learning activities.

4. **System Testing and Evaluation**

- Test the AI functions and administration functions after they are integrated into the system.
- Summarize the functions completed during the development process.
- Identify the current limitations of the system.
- Suggest future improvements and development directions for the system.

Person in charge: Trương Tường Vi

Responsibilities:

1. **Development of Basic User Interfaces and Functions**

- Develop the Landing Page and authentication interfaces, including registration, login, and forgot password.
- Develop the Dashboard for users after login.
- Develop interfaces for the course list, topic list, and lesson page.
- Develop interfaces for Quiz, Matching Game, and Code Arrangement Game.
- Develop the Code Playground for Python programming practice.
- Develop the personal profile interface and Pet page.

2. **Development of Self-Learning Support Functions**

- Allow users to access and view lesson content by course and topic.
- Allow users to take Quizzes and practice through learning games.
- Develop functions to track and update the learning progress of each user.
- Develop functions to display personal learning results.
- Develop functions to set up and receive learning reminders.

3. **Development of Gamification Functions**

- Develop a reward point system to record users' learning activities.
- Develop a leaderboard based on learning results and learning activities.
- Develop daily learning tasks to encourage users to maintain regular learning.
- Develop a learning streak system to record continuous learning days.
- Develop the Pet system and related interaction functions based on learning activities.

4. **Improvement of User Interface and User Experience**

- Create an intuitive and straightforward layout for the user interface.
- Make the design more contemporary and unified throughout the various pages.
- Adapt the UI so that it functions properly on various screen sizes.
- Test the navigation flow and usability of functions for learners.

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to MSc. Lương Trần Ngọc Khiết, our supervisor, for his valuable guidance, continuous support, and helpful feedback throughout the completion of this project. His advice and encouragement helped us overcome difficulties, improve our work step by step, and complete the project more effectively.

We would also like to sincerely thank the lecturers at the NTT Institute of International Education (NIIE), Nguyen Tat Thanh University, for providing us with the knowledge and skills necessary for this project. Their teaching and support during our studies gave us a strong foundation to apply what we had learned to the analysis, design, development, and testing of the system.

Although we have made every effort to complete this project, we understand that it may still have some limitations due to our limited practical experience. We sincerely hope to receive valuable comments and suggestions from the committee members so that we can further improve the project and gain useful experience for our future work.

We are truly grateful for all the support and guidance we have received throughout this project.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABBREVIATIONS LIST

Abbreviation	Explanation
AI	Artificial Intelligence
API	Application Programming Interface
CLASP	Command Line Apps Script Projects
CSS3	Cascading Style Sheets 3
D3	Data-Driven Documents
DOM	Document Object Model
ES6+	ECMAScript 2015 and Later Versions
HTML5	HyperText Markup Language 5
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
MASTER_DB	Master Database
MCQ	Multiple-Choice Question
OAuth	Open Authorization
REST	Representational State Transfer
SHA-256	Secure Hash Algorithm 256-bit
SPA	Single Page Application
SVG	Scalable Vector Graphics
UI	User Interface
URL	Uniform Resource Locator
USER_DB	User Database
XP	Experience Points
XQP	Virtual Currency Used in TerraCode

ABSTRACT

In the context of the growing digital transformation in education, developing a self-learning system that combines learning content, practice, assessment, and progress tracking in one environment is useful for programming learners. The project "Design of a Gamified Python Self-Learning Support System on Google Apps Script" aims to develop a Web application for beginners to learn Python through an organized learning process. At the same time, the system supports learning motivation through gamification and artificial intelligence.

Guests, Users, and Administrators are the three primary groups for which the system is intended. Guests can register, check their email, log in, and reset their password. Users can select courses, topics, and lessons in addition to studying through theory information, mind maps, and flashcards. Additionally, they can practice with code playgrounds, matching games, quizzes, and code arrangements. Learning progress, practice outcomes, wrong answers, activity history, learning objectives, and reminders are all stored in the system. Users, courses, topics, lesson content, practice data, pet data, and statistical data can all be managed by administrators.

TerraCode is developed using the Client-Server paradigm. Google Apps Script V8 Runtime is used on the server side, whereas HTML5, CSS3, and JavaScript ES6+ are used on the client side. Data is stored in two primary groups using Google Sheets and Google Drive: MASTER_DB and USER_DB. In order to facilitate learning content, authentication, reminders, and AI features, the system also incorporates Google Docs, Google OAuth 2.0, Gmail API, and Gemini API.

Gamification is implemented through XP, XQP, Daily Quest, learning progress, and Pet. These elements are updated based on the User's learning and practice activities. Gemini API supports the generation of Mindmaps, Quiz questions, Matching Game content, and new questions based on the content that Users answered incorrectly. AI-generated content is checked for its structure and controlled by Admin before being used.

The project development process includes requirement analysis, system design, development, testing, and evaluation. The results show that the system can connect learning, practice, gamification, progress management, and AI functions in one

environment. Therefore, TerraCode shows that Google Apps Script can be effectively used to develop a Python self-learning support system at a scale suitable for the research objectives.

CHAPTER 1
INTRODUCTION

1.1. Rationale for Topic Selection

Programming is a highly practical subject. Learners need to understand knowledge, read source code, write programs, and practice regularly to develop programming skills. For beginners, Python is often chosen because its syntax is relatively clear and suitable for learning basic programming concepts. However, self-learning Python still has many difficulties.

One common difficulty is that learners can easily lose direction when studying by themselves. There are many learning materials on the Internet, but they have different levels, presentation styles, and learning orders. Beginners may find it difficult to decide where to start, what to learn first, and how much practice is needed before moving to the next topic.

Learning programming can also become repetitive if learners mainly read theory and complete similar exercises. Without regular feedback and short-term goals, learners may find it difficult to maintain a learning habit for a long period. The gap between understanding a concept and applying it to an exercise can also make self-learning more difficult.

Gamification is the use of some common game elements in learning activities. These elements are used to create clear goals, provide progress feedback, and encourage learners to continue learning. Gamification does not replace learning content. Its role is to support the organization of the learning process and provide additional motivation during self-study.

In addition, cloud-based application development platforms make it easier to build online learning systems with reasonable deployment costs for academic projects. Google Apps Script is one suitable platform for this approach because it supports online application development and can connect with services in the Google ecosystem.

Based on these issues, this project studies the design of a Python self-learning support system with gamification on Google Apps Script. The main focus is to build an

approach that combines learning content, practice activities, and learning motivation mechanisms in one system.

1.2. Research Objectives

The main objective of this project is to study, design, and develop a system that supports the self-learning process of Python programming. Gamification is included as a supporting mechanism to help learners maintain learning activities and track their learning progress.

To achieve this objective, the project defines the following specific objectives:

1. Study the needs and common problems in the self-learning process of Python programming.
2. Identify the main requirements of a learning support system, including learner needs, content management requirements, and system operation requirements.
3. Study how to organize learning content in a clear and logical order that is suitable for self-learning.
4. Study the use of gamification in programming education to support progress tracking, create learning goals, and maintain learner participation.
5. Design the system model, processing flows, data structure, and main components before development.
6. Develop the system on Google Apps Script based on the proposed design.
7. Perform system testing to check the correctness of the main functions, data processing, and common error cases.
8. Evaluate the results based on the original objectives, identify limitations, and suggest future improvements.

1.3. Research Context

Online self-learning has become increasingly common in the field of information technology. Learners can easily access textbooks, tutorials, exercises, and programming support tools. The variety of learning resources allows learners to manage their own learning time and learning speed.

However, self-learning requires a high level of self-management. Learners need to select suitable content, make learning plans, track their progress, and evaluate their own results. This requirement is especially important in programming because

programming knowledge is often connected. If learners do not understand an earlier concept, they may have difficulty learning the next topic.

Current online learning systems often combine several methods to support learners, such as topic-based lessons, practical exercises, result evaluation, and progress tracking. Gamification is also applied in many learning environments to provide short-term goals and feedback during the learning process.

At the same time, cloud platforms make the development of learning applications more convenient. Instead of building and maintaining a complete server infrastructure, developers can use platforms that provide application environments and available services.

In this context, this project studies a Python self-learning support system with gamification on Google Apps Script. The project focuses on the design and development of a software solution for learning. It does not focus deeply on educational theories or psychological measurement of the effects of gamification.

1.4. Research Scope and Subjects

1.4.1. Research Subjects

The project has three main research subjects.

The first subject is the self-learning process of Python programming. The project focuses on how learners access learning content, practice, receive results, and track their learning progress.

The second subject is gamification in learning. The project studies how mechanisms for setting goals, recording results, and providing progress feedback can be included in a learning system.

The third subject is the software system model for self-learning support developed on Google Apps Script. This includes system requirements, system design, data, development, and testing at a level suitable for a technical graduation project.

1.4.2. Research Scope

The project focuses on the following areas:

- Studying the requirements of a system that supports self-learning of Python programming.
- Organizing learning content into courses, topics, or suitable knowledge units.

- Organizing practice activities to help learners strengthen their programming knowledge.
- Applying gamification to record learning activities, progress, and results.
- Designing data management mechanisms for learning content, users, and learning progress.
- Developing the system as an online application on Google Apps Script.
- Testing the main functions, processing flows, data, access permissions, and some error situations.
- Evaluating the system based on the requirements and objectives defined within the project scope.

This scope focuses on developing and evaluating a software product at the level of a graduation project. Specific functions and technical solutions are presented in later chapters and are not described in detail in this introduction.

1.4.3. Limitations and Scope Exclusions

Because of limitations in time, resources, and project scope, several areas are not the main focus of this study.

- The project does not build a complete Python training program for all skill levels and all learner groups.
- The project does not replace teachers, lecturers, or formal training programs.
- The project does not study all methods of teaching programming.
- Gamification is mainly studied as part of the software system design. The project does not conduct an in-depth psychological study of learner behavior.
- The project does not aim to prove that gamification improves learning results through a large-scale experimental study.
- The evaluation mainly focuses on how well the system meets its requirements and on the testing results within the project scope.
- The system is studied using platforms and resources suitable for the project. Large-scale deployment with a high number of users is not a main part of the evaluation.

These limitations help separate the technical results of the graduation project from conclusions that would require larger and longer experimental studies.

1.5. Research Methodology

The project uses a system development approach. The process is divided into five stages: requirements analysis, system design, system development, system testing, and finally evaluation and refinement. This structure allows the results of each stage to become the basis for the next stage.

1.5.1. Requirements Analysis

The first stage focuses on identifying the problem and the needs of the user groups related to the system.

The main tasks include:

- Defining the objectives and scope of the system.
- Analyzing learners' needs in the self-learning process of Python programming.
- Identifying the user groups and the responsibilities of each group.
- Identifying the main groups of system functions.
- Identifying requirements for data, access permissions, and system operation.
- Separating essential requirements from additional functions.

The result of this stage is a set of requirements that provides the basis for system design.

1.5.2. System Design

Based on the identified requirements, the project develops the overall system model before programming begins.

The design stage focuses on:

- Dividing the system into functional groups.
- Defining the relationships between users and system functions.
- Designing the main processing flows.
- Designing the overall system architecture.
- Designing the data structure and relationships between data groups.
- Defining principles for authentication, authorization, and data protection.

The purpose of this stage is to create a clear design so that system development does not depend on implementing separate functions without an overall structure.

1.5.3. System Development

The development stage converts the designs into a working software system on Google Apps Script.

The system is developed by functional groups. Each group is implemented based on the requirements and processes defined earlier. After one part is completed, functional testing is performed before it is integrated with the remaining components. The development process also includes organizing the source code, connecting data, processing interactions between system components, and deploying the application for testing.

Details about technologies, source code structure, and deployment methods are not presented in Chapter 1. They are explained in the later technical chapters.

1.5.4. System Testing

After the development stage, the system is tested to determine whether the implemented results meet the defined requirements.

The testing process focuses on:

- Testing the main functions.
- Testing valid and invalid user flows.
- Testing input data processing.
- Checking the results after each operation.
- Testing access permissions between different user groups.
- Testing error and exception handling.
- Checking data consistency after update operations.

Testing results are recorded and compared with the expected results. Cases that do not meet the expected results are corrected and tested again.

1.5.5. Evaluation and Refinement

The final stage compares the developed system with the objectives and requirements defined at the beginning of the project.

The evaluation focuses on several areas:

- The level of completion of the main functional groups.
- The ability of the system to support the proposed self-learning process.
- The suitability of gamification mechanisms for learning activities.
- The ability of the system to operate on the selected platform.
- Testing results and remaining errors.
- Limitations that have not been solved within the project scope.

Based on the evaluation results, the project makes final improvements and identifies suitable directions for future development.

1.6. Structure of the Graduation Project

Chapter 1. Introduction

This chapter presents the rationale for topic selection, research objectives, research context, research subjects and scope, research methodology, and the structure of the graduation project.

Chapter 2. Theoretical Background and Technological Foundation

This chapter presents the concepts and foundations related to self-learning Python programming, learning support systems, gamification, related technologies, and related studies or solutions. This chapter provides the theoretical basis for the approach used in the project.

Chapter 3. Requirement Analysis and System Design

This chapter presents the system requirements, user groups, functions, Use Cases, system architecture, database design, main processing flows, and security design. The chapter focuses on how the system is organized before implementation.

Chapter 4. System Implementation, Testing and Evaluation

This chapter presents the development and deployment environment, the implementation results of the main functional groups, learning support mechanisms, gamification, artificial intelligence functions included in the project scope, system testing, and system evaluation results.

Chapter 5. Conclusion and Future Development

This chapter evaluates how well the research objectives have been achieved, presents the main results, identifies the remaining limitations, and suggests future development directions for the system.

CHAPTER 2
THEORETICAL BACKGROUND AND TECHNOLOGICAL FOUNDATION

2.1. Overview of Learning Python Programming

2.1.1. Introduction to Python

Python is a high-level, interpreted programming language with dynamic typing. It was developed by Guido van Rossum in the early 1990s and has continued to be developed by the community until today.

Python supports different programming styles, such as procedural programming, object-oriented programming, and functional programming. Programmers can choose a suitable style based on the purpose of the program.

One clear feature of Python is the use of indentation to define blocks of code. In many other programming languages, code blocks are usually defined by brackets. Python requires programmers to show the program structure through indentation. This makes the source code easier to read, but learners also need to pay attention to code formatting.

Python also provides an interactive mode. Learners can enter a command, run it, and immediately see the result. This feature is suitable for learning basic concepts such as variables, data types, operators, strings, and expressions.

A Python learning process usually starts with basic topics such as variables, data types, conditional statements, loops, and functions. At a higher level, learners continue with data structures, exception handling, files, modules, and object-oriented programming.

2.1.2. Advantages of Python

Python has several characteristics that are suitable for learning programming.

First, Python syntax is relatively short and easy to read. Learners do not need to work with too many symbols in simple programs. This helps them focus more on how the program works and how to solve the problem.

Second, the program structure is clearly shown through indentation. Learners can easily recognize which group of statements belongs to a condition, loop, or function. This supports the reading and analysis of source code.

Third, Python supports interactive mode. Learners can quickly test a command or expression without creating a complete program. Immediate results make the testing process more convenient.

Fourth, Python has a large standard library and a wide library ecosystem. After learning the basic knowledge, learners can continue to areas such as data processing, Web development, automation, data science, and artificial intelligence.

Fifth, Python can run on many popular operating systems. Learners can use the language in different environments without making many changes to the program.

These advantages help reduce some of the difficulties when beginners start learning programming. However, easy-to-read syntax does not mean that programming thinking is also easy. Learners still need regular practice to understand problem analysis and algorithm development.

2.1.3. Challenges in Learning Python

One major difficulty in learning Python is converting a problem into specific processing steps. Learners may understand the question but may not know how to divide the problem into smaller steps that a computer can perform.

The second difficulty is understanding the program execution flow. When a program contains many conditions, loops, or functions, beginners may lose track of variable values and may not understand why the program produces a certain result.

Debugging is also a difficult part of learning programming. Programming errors are not only syntax errors. A program may run normally but still give the wrong result. Learners need to know how to read error messages, check each step, and find the cause of the error.

Python's dynamic typing provides flexibility, but learners also need to clearly understand the type of data being processed. A variable can contain different types of values during program execution. If learners do not control the data well, errors can occur.

Indentation is also part of Python syntax. A statement placed at the wrong indentation level can change the program flow or cause an error. Beginners need to develop the habit of writing code with the correct structure from the beginning.

Besides technical knowledge, learners may also have difficulty maintaining motivation. Programming requires many cycles of trying, making mistakes, and correcting them. Without small goals, feedback, and suitable practice activities, learners may easily give up after facing repeated errors.

Therefore, learning Python should combine theory with practice. Learners should study knowledge step by step, complete exercises after each topic, and receive feedback to improve their understanding.

2.2. Gamification in Education

2.2.1. Concept of Gamification

Gamification is the use of game design elements in non-game contexts. Common elements include points, levels, badges, missions, challenges, rewards, progress bars, and leaderboards.

Gamification is different from Game-Based Learning. In Game-Based Learning, learners gain knowledge through a complete game. The game usually has its own rules, goals, context, and gameplay process.

In Gamification, the original activity is still kept. The designer adds some game mechanisms to provide feedback or encourage certain behaviors.

For example, an activity may require learners to complete five exercises. The learning content remains the same. Gamification can add points after each exercise, show the number of completed exercises, give a reward after all five exercises are completed, or record a continuous learning streak.

The main idea of Gamification is the design of the learning experience. Adding many points or rewards does not always create better learning results if these elements are not connected with the learning goals.

2.2.2. Role of Gamification in Learning Motivation

Gamification can divide a large goal into smaller goals. Learners can understand their current position, what they have completed, and how much work is still left.

Mechanisms such as points and progress bars provide immediate feedback after an activity. Learners do not need to wait until the final test to know their results. They can see their progress through each small task.

Missions and challenges can also guide learning activities. Instead of allowing learners to decide everything by themselves, a specific goal can help them understand what they should complete within a short period.

Some studies show that Gamification can have a positive effect on learning performance, motivation, and learning behavior. However, the effect is not the same for all learners and situations. Its effectiveness depends greatly on the selected elements, reward design, and the relationship between Gamification and learning goals.

Gamification also has some limitations. If rewards are too important, learners may focus more on points or rankings than on knowledge. Strong competition can also create pressure for learners who often have lower results.

Therefore, Gamification is more suitable when feedback and rewards are directly connected with meaningful learning behaviors, such as completing lessons, practicing regularly, or improving results.

2.2.3. Gamification Elements

❖ **Points**

Points are numerical values given after learners complete an activity. Points help show results in an easy-to-understand way. Learners should understand which activities give points and why they receive those points.

❖ **Levels**

Levels show progress through different stages. When learners reach a certain number of points or meet specific conditions, they move to the next level. This mechanism helps divide a long learning process into smaller stages.

❖ **Badges and Achievements**

Badges are used to recognize a specific milestone or behavior. For example, learners may receive a badge after completing a group of tasks or maintaining learning activities for a period of time.

❖ **Progress Bars**

A progress bar shows the percentage of work that has been completed. Learners can easily see their progress and the amount of work remaining. This is a simple but useful feedback mechanism.

❖ **Missions and Challenges**

A mission gives learners a clear goal to complete. A suitable mission should have clear conditions, reasonable difficulty, and an easy-to-understand result.

❖ **Rewards**

Rewards are given after learners achieve a goal. Rewards can be points, virtual items, access permissions, or other forms of recognition. Rewards should be connected with learning activities instead of becoming the only purpose of learning.

❖ **Leaderboards**

Leaderboards allow learners to compare their results with other participants. This element can create competitive motivation, but it may also create pressure. Therefore, leaderboards should be used carefully and should match the learner group.

❖ **Streaks**

A streak records the number of days or times that learners continue an activity without interruption. This mechanism supports habit building and continued participation.

❖ **Immediate Feedback**

Immediate feedback shows the result directly after an action. In education, feedback should not only tell learners whether an answer is correct or incorrect. When suitable, the system should also provide the reason or a correction so that learners can understand their mistakes.

Not all Gamification elements need to be used at the same time. The selection should depend on the learning goals, learner group, and type of activity.

2.3. Artificial Intelligence (AI)

2.3.1. Overview of AI

Artificial Intelligence, or AI, is a broad field in computer science. The purpose of AI is to build systems that can perform tasks that require information analysis and produce results based on input data.

AI includes many different research areas.

Machine Learning focuses on building models that learn from data. Instead of programming every rule in advance, the system finds patterns in data to make predictions or classifications.

Deep Learning is a branch of Machine Learning. This method often uses multi-layer neural networks and is applied to image, audio, and language processing.

Natural Language Processing focuses on helping computers process human language. Common tasks include text analysis, machine translation, question answering, summarization, and content generation.

Generative AI refers to AI systems that can create new content based on input requests. The output can be text, images, audio, video, or programming code.

Generative models work based on patterns and relationships learned from data. Therefore, an answer may sound reasonable but can still contain incorrect information.

2.3.2. Gemini AI

Gemini is a family of multimodal AI models developed by Google. Multimodal means that the model can work with different types of data, such as text, images, audio, video, and programming code.

The ability to process different types of data makes Gemini suitable for many tasks. A request can include text together with an image or source code. The model analyzes the input information and generates a result based on the request.

Gemini supports natural language processing. Users can describe their requests in normal sentences instead of writing specific programming commands. The model then analyzes the context and creates a response.

Another ability of Gemini is content generation. The model can generate text, suggest structures, create questions, explain content, or support programming code processing.

Gemini versions continue to be developed and changed over time. Therefore, in the theoretical background, Gemini should be understood as a family of multimodal AI models rather than focusing too much on one specific version.

2.3.3. Applications of AI in Learning Content Generation

Generative AI has many applications in the preparation of learning materials.

❖ **Generating Summaries**

AI can process a long document and create a shorter version. A summary helps learners identify the main ideas and makes review easier. The generated content

should still be checked with the original document to avoid missing or incorrect information.

❖ **Generating Questions**

AI can create questions from existing content. The questions can be multiple-choice questions, short questions, or practice exercises. The content creator should check the questions, answers, and difficulty level before using them in learning materials.

❖ **Generating Flashcards**

From long content, AI can identify terms, concepts, and definitions to create flashcards. Flashcards are suitable for reviewing short concepts and key terms.

❖ **Generating Knowledge Maps**

AI can help identify main ideas and the relationships between them. The result can then be organized into an outline or data for a mind map.

❖ **Rewriting Content**

AI can rewrite a concept using shorter or easier language. This can support learners when the original content is difficult to understand.

❖ **Generating Examples**

AI can provide additional examples for a concept. In programming, examples can include source code, usage situations, or short exercises.

❖ **Supporting Programming Exercise Generation**

AI can suggest exercises, syntax questions, or programming tasks. It can also help generate input data, code examples, or instructions.

These applications can reduce the time needed to prepare initial drafts. However, AI-generated content still needs human review before it is included in learning materials.

2.3.4. Limitations and Considerations When Using AI

Accuracy is one of the first limitations that needs attention. AI can generate content that sounds reasonable but contains incorrect facts, formulas, or explanations. In learning materials, even a small error can cause learners to misunderstand the knowledge.

AI can also generate information that does not exist. This is commonly called hallucination. The model may provide a document name, reference, or fact in a convincing way even when the information is incorrect.

Bias is another issue. Training data may contain different opinions or imbalances. Therefore, AI-generated results should not automatically be considered neutral.

Privacy should also be considered when data is sent to an AI service. Personal information, learning data, and restricted documents should not be sent to an external service without understanding how the data is processed.

AI can also create the risk of overdependence. If learners use AI to get complete answers for every exercise, they have fewer opportunities to analyze problems and correct errors by themselves. This issue is especially important in programming because problem-solving skills are developed through writing, testing, and correcting programs.

The quality of AI output depends greatly on the input request. An unclear prompt can lead to unsuitable results. The same request may also produce different results at different times.

Copyright and content sources should also be considered. Users should not assume that all AI-generated content can be used without checking its source, usage rights, or originality.

In education, AI is more suitable as a supporting tool. AI can create drafts, summaries, question suggestions, or rewritten content. Teachers or content managers still need to check the accuracy, suitability, and learning value of the generated results.

2.4. Google Apps Script

2.4.1. Overview of Google Apps Script

Google Apps Script is a JavaScript-based application development platform provided by Google. The source code runs on Google servers and can access many services in the Google ecosystem through related libraries and APIs.

Apps Script supports many types of applications and tasks. Some common uses include:

- Automating tasks in Google Workspace.
- Processing data in Google Sheets.
- Creating and editing Google Docs documents.
- Managing files and folders in Google Drive.
- Sending and processing emails.

– Creating add-ons for Google Workspace applications.

– Deploying Web applications that users can access through a browser.

Web applications on Apps Script usually receive HTTP requests through functions such as doGet() or doPost(). The returned result can be HTML content or text data. This approach allows Apps Script to work as the Server side of Web applications with a suitable scale.

Apps Script currently supports the V8 Runtime. This is a JavaScript execution environment based on V8, which is also used by the Chrome browser. V8 allows developers to use many modern JavaScript features such as let, const, class, arrow functions, template literals, and newer ECMAScript features.

2.4.2. Advantages of Google Apps Script

Google Apps Script has several advantages for developing small and medium-sized applications.

First, the platform runs on Google's cloud infrastructure. Developers do not need to install and maintain a separate Web Server for applications that are suitable for the Apps Script model.

Second, Apps Script connects directly with Google Workspace services. Operations with Sheets, Docs, Drive, Gmail, and other services are supported through built-in services or Google APIs.

Third, the platform is based on JavaScript. JavaScript is a popular language in Web development. Using the same language platform can reduce the number of technologies that beginners need to learn when building small Web applications.

Fourth, Apps Script supports direct Web App deployment. After deployment, the application has an address that users can open in a browser. Version updates can also be managed inside the Apps Script environment.

Fifth, Google provides authentication and authorization mechanisms for many services. When code needs to access Google data, Apps Script manages permissions based on access scopes, usually called OAuth scopes.

Sixth, the platform is suitable for prototypes, internal tools, data management applications, and academic projects. The initial setup cost is lower than building a complete server infrastructure from the beginning.

2.4.3. Limitations of Google Apps Script

Google Apps Script is not suitable for every type of application. The platform has quotas and limits to control the use of resources.

One important limitation is the execution time for each run. According to Google documentation at the time of reference, a normal Apps Script execution has a limit of 6 minutes. If this limit is exceeded, the process is stopped.

Apps Script also limits the number of service calls, concurrent executions, data usage, and some daily activities. These quotas are different between personal Google accounts and Google Workspace accounts. Google may also change these limits over time.

Because Apps Script runs on infrastructure managed by Google, developers cannot control the server at the operating system level. Installing system software, configuring a Web Server, or deeply changing the runtime environment as on a private server is not part of the Apps Script model.

Library and package management is also different from a traditional Node.js environment. Some JavaScript libraries may need to be changed or bundled before they can run in Apps Script.

Application performance also depends on the number of calls to external services such as Sheets or Drive. Reading and writing data many times can increase processing time. Therefore, Apps Script is more suitable for applications with a moderate workload than for systems that need to handle a very large number of requests with low delay.

2.5. Google Services Used

2.5.1. Google Sheets

Google Sheets is an online spreadsheet application in Google Workspace. Data is organized in a Spreadsheet. Each Spreadsheet contains one or more Sheets, and the data in each Sheet is organized into rows, columns, and cells.

Google Sheets supports many types of data such as text, numbers, dates, formulas, and links. Users can edit data directly in a browser and share documents with other people using different permission levels.

The Google Sheets API allows applications to create spreadsheets, read data, write data, update values, and format cells. In the Apps Script environment, Spreadsheet Service provides direct access to spreadsheets without requiring developers to build all HTTP requests manually.

One advantage of Google Sheets is that the data structure is easy to view and edit. Administrators can see the data through a familiar spreadsheet interface without using separate database management software.

However, Google Sheets is not a relational database management system. It does not provide all features such as data relationships, SQL queries, multi-step transactions, or special indexing mechanisms like database systems. When the amount of data and the number of requests increase greatly, managing data in spreadsheets can become more difficult.

2.5.2. Google Docs

Google Docs is Google's online document editing service. A document can contain paragraphs, headings, lists, tables, images, links, and many other formatting elements. Google Docs is suitable for long-form text content. Users can edit content in a browser, share documents, and work together with other people.

The Google Docs API allows applications to create, read, and edit documents through programs. Each document has its own Document ID. This ID helps the application identify the correct document even if the document name is changed.

One benefit of Google Docs is that it separates text content from application source code. Content managers can edit documents through a familiar editor instead of directly changing HTML or programming code.

A limitation is that the document structure is more complex than plain text. When converting a document into another format, the application needs to process headings, paragraphs, lists, tables, images, and other formatting styles.

2.5.3. Google Drive

Google Drive is Google's cloud file storage service. Users can store, organize, search, and share files through folders and permission settings.

Google Drive manages many types of files such as Google Docs, Google Sheets, images, PDF files, and files uploaded from computers. Each file has identification

information and metadata, such as file name, file type, update time, and access permissions.

The Google Drive API allows applications to perform operations such as:

- Creating files and folders.
- Uploading and downloading files.
- Searching for files.
- Reading file information.
- Moving and organizing files.
- Setting sharing permissions.
- Tracking some activities related to files.

Google Drive is suitable when an application needs to store data as files instead of putting all data into a table or database. Access permissions still need to be designed carefully because a file shared with the wrong permission may expose data to users outside the expected group.

2.5.4. Gmail API

Gmail API is a REST API provided by Google that allows applications to access and manage data in a Gmail mailbox after receiving the required permissions.

The API supports objects such as messages, threads, drafts, and labels. An application with the correct permissions can read emails, search emails, manage labels, create drafts, and send emails.

For sending emails, Gmail API supports sending a message directly or sending content that has already been saved as a draft. The email content is converted into the required format before being sent through the API.

Gmail API uses authentication and authorization mechanisms. An application does not automatically have permission to access a user's mailbox. Permissions are controlled through OAuth scopes. The permission scope should be limited to what the application really needs.

Gmail API is suitable for automatic email sending and email processing by programs. Email sending still follows the limits and policies of Gmail or Google Workspace.

2.6. Front-End Technologies and Supporting Libraries

2.6.1. HTML5

HTML stands for HyperText Markup Language. It is a markup language used to describe the structure and content of a Web page.

HTML organizes content through elements such as headings, paragraphs, lists, tables, images, links, forms, and buttons. Each element represents the meaning or role of a part of the content.

The term HTML5 is commonly used to describe modern HTML. Today, WHATWG maintains HTML as a Living Standard. This means that the standard continues to be updated instead of being released as separate major versions as in the past.

Modern HTML provides more meaningful elements such as header, nav, main, section, article, and footer. Using suitable elements makes the document structure easier to understand for browsers, search engines, and accessibility technologies.

HTML also provides elements for forms, audio, video, Canvas, and many APIs related to Web applications.

2.6.2. CSS3

CSS stands for Cascading Style Sheets. It is a language used to control the presentation of HTML documents. CSS controls elements such as colors, sizes, spacing, positions, layouts, effects, and how the interface changes with different screen sizes.

The term CSS3 is commonly used to describe modern CSS. Under the current standardization approach, CSS is no longer developed as one single CSS3 or CSS4 version. Features are developed as separate modules such as Color, Grid Layout, Flexbox, Media Queries, and Animations.

A CSS rule usually contains a selector and a group of properties. The selector identifies which element should receive the style. The properties and values define how the element is displayed.

CSS supports layout techniques such as Flexbox and Grid. Media Queries allow the interface to change based on screen size or device characteristics. This is the foundation of Responsive Web Design, which helps the interface work on different screen sizes.

CSS also supports transitions and animations for visual effects. These effects should be used at a suitable level so that the interface remains easy to follow and does not distract users from the main content.

2.6.3. JavaScript ES6+

JavaScript is a popular programming language for the Web. HTML defines the structure, CSS controls the presentation, and JavaScript manages behavior and interaction.

The official standard name of the language is ECMAScript. ES6 is the common name for ECMAScript 2015. The term ES6+ is often used informally to describe ES2015 and the features added in later ECMAScript versions.

ES6 introduced many features that help make source code clearer, such as:

- let and const for declaring variables.
- Arrow functions.
- Template literals.
- Destructuring.
- Classes.
- Modules.
- Promises.

Later versions added more features such as async and await, optional chaining, and many other improvements.

In a browser, JavaScript works with Web APIs such as DOM, Fetch API, History API, and Web Storage. These APIs allow programs to read or change page content, process user actions, send requests to the Server, and update the interface without reloading the whole page.

2.6.4. Single Page Application

A Single Page Application, or SPA, is a Web application model where most user interactions happen inside one main page. When users move between functions, JavaScript updates only the required part of the content instead of asking the browser to load a complete new HTML document for every action.

A SPA usually works through the following process:

User performs an action

↓

JavaScript identifies the request

↓

The required data is loaded or retrieved

↓

Part of the interface is updated

↓

The current page state is kept

One advantage of SPA is that it reduces the number of full-page reloads. Interface transitions can feel smoother, and Client-side state can be kept more easily between different screens.

However, SPA also gives more responsibility to Client-side JavaScript. Navigation, state management, error handling, and interface updates need to be organized clearly. If the application becomes large without a good structure, Client-side code can become difficult to maintain.

Browser history also needs to be handled so that the Back and Forward buttons work correctly. The History API provides functions that help manage SPA navigation history without reloading the entire page.

2.6.5. Chart.js

Chart.js is an open-source JavaScript library used to display charts on the Web. It provides many common chart types such as bar charts, line charts, pie charts, doughnut charts, radar charts, and scatter charts.

Chart.js renders charts on the HTML Canvas element. This approach can reduce the number of DOM elements when a chart contains many data points.

A chart is usually created from three main groups of information:

- Chart type.
- Data and labels.
- Display options.

Chart.js supports animations, tooltips, legends, scales, and plugins. The library is suitable when developers need common chart types without building all visualization logic from the beginning.

Compared with a lower-level library such as D3.js, Chart.js provides ready-made chart structures and is easier to use. However, highly customized visualizations may require more configuration.

2.6.6. D3.js and Markmap

D3.js, or Data-Driven Documents, is an open-source JavaScript library for data visualization on the Web. D3 provides tools for connecting data with visual elements and working with DOM, SVG, or Canvas.

D3 is different from ready-made chart libraries. It works at a lower level and provides components such as scales, axes, shapes, transitions, hierarchies, and layouts. Developers combine these components to create visualizations that match the data.

The main advantage of D3 is its high flexibility. The disadvantage is that developers need to write more logic compared with using a chart library with predefined structures.

Markmap is a tool that converts Markdown content into a mind map. Heading and list structures in Markdown are analyzed and converted into nodes with hierarchical relationships.

Markmap separates the process into two main steps: converting Markdown into structured data and rendering that data as a diagram. The Markmap display layer supports SVG and uses D3 during the rendering process.

Combining Markdown with mind maps makes source data easier to write and edit than manually defining the position of every node in a diagram.

2.6.7. html2canvas

html2canvas is a JavaScript library used to create an image that represents part or all of a Web interface directly in the browser.

The library reads the DOM structure, processes information from the elements, and rebuilds the result on a Canvas. The Canvas can then be processed further to create an image.

html2canvas does not take a direct screenshot at the operating system level. Instead, it rebuilds the interface based on the DOM and the properties that it can understand. Therefore, the final image may not always look exactly the same as the browser interface.

Some CSS properties may not be fully supported. Images from other domains are also affected by the Same-Origin Policy if they are not configured correctly. These factors need to be considered when using html2canvas to export Web content as images.

2.6.8. DiceBear API

DiceBear is an open-source library and API used to generate avatars. Avatars are generated based on a seed and a selected image style.

The seed is used as input data for avatar generation. When the same style and the same seed are used again, DiceBear generates a consistent result. This feature is useful for applications that need default avatars when users have not uploaded their own images.

DiceBear supports many avatar styles and options such as background, size, ratio, and other features depending on each style.

The service provides an HTTP API that can generate avatars from a URL. The library can also run inside an application to generate avatars directly. DiceBear source code is released under an open-source license, while individual avatar styles may use different licenses. Developers need to check the license of each style before using it in a product.

2.7. Development Tools

2.7.1. Visual Studio Code

Visual Studio Code, commonly called VS Code, is a source code editor developed by Microsoft. It works on Windows, macOS, and Linux.

VS Code provides many useful programming features, such as:

- Syntax highlighting.
- IntelliSense and code suggestions.
- Search and replace across a project.
- Refactoring.
- Debugging.

– Integrated Terminal.
– Source control management.
– Extension installation.

The extension system allows VS Code to support many languages and technologies without including every function in the default application.

VS Code also includes Git integration. Developers can view changes, create commits, and work with repositories directly inside the editor.

The main role of VS Code is to provide an environment for editing and managing source code. It does not replace the application's execution platform. The source code still runs on the related runtime or platform.

2.7.2. CLASP

CLASP stands for Command Line Apps Script Projects. It is a command-line tool for developing and managing Google Apps Script projects from a local environment.

Instead of editing all source code in the online Apps Script editor, CLASP allows developers to download the code to a computer and work with an external editor.

Common operations include:

– Signing in to a Google account.
– Creating or copying an Apps Script project.
– Downloading source code from Apps Script to a local computer.
– Uploading local source code to Apps Script.
– Viewing and managing versions.
– Managing some deployment activities.

An important advantage of CLASP is that it connects Apps Script with a local software development workflow. When source code is stored in a local folder, developers can easily combine it with an editor, Git, code checking tools, and other collaboration processes.

The official CLASP repository notes that it is not a Google product officially supported at the same level as Google's commercial services. Therefore, developers should monitor tool version changes when setting up the development environment.

2.7.3. Git

Git is a distributed version control system. It records the history of source code changes and supports multiple people working on the same project.

In Git, source code is stored in a repository. Each time developers want to save an important state of the project, they create a commit. A commit records the changes made at that time.

Some main Git concepts include:

- Repository: A storage area for the project and its version history.
- Commit: A recorded point of changes.
- Branch: An independent development branch.
- Merge: The process of combining changes from different branches.
- Remote: A repository stored on another computer or online service.

Git helps developers view change history, compare different versions, and return to an earlier state when errors occur. Branches allow developers to test new functions without directly affecting the main development branch.

Git is a version control tool and is not the same as GitHub. Git can work independently on a local computer. GitHub, GitLab, and similar platforms provide online Git repository hosting and collaboration features.

CHAPTER 3

REQUIREMENT ANALYSIS AND SYSTEM DESIGN

3.1. Overview of the TerraCode System

3.1.1. Purpose of the System

TerraCode is developed to create a clear and organized environment for self-learning programming. Learners can know what they are studying, which parts they have completed, and what they should continue next. Instead of only reading theory for a long time, the system divides the learning content into different topics and provides different learning forms such as lessons, mind maps, flashcards, quizzes, and learning games.

Another purpose of TerraCode is to keep learners active through gamification. After completing a lesson or practice activity, users receive XP, and their learning progress is updated. The system also records learning streaks, daily quests, and progress on the "learning mountain." These data help learners see their learning process instead of judging their results only through a single test.

TerraCode also helps reduce repeated work when preparing learning content. Administrators can link lesson materials from Google Docs and use Gemini AI to analyze the documents and generate Mindmaps, Quiz questions, and Matching Game content. AI-generated content can be reviewed and edited before it is published in the system.

3.1.2. Main Functions

The functions of TerraCode are divided into two main groups: functions for learners and functions for administrators.

For learners, the system provides account registration, login, email verification, and password recovery. After logging in, learners can access the Dashboard to view their XP, XQP, learning streak, daily quests, study calendar, and activity statistics. Learners can select courses, topics, and lessons based on published content and their learning progress.

In each lesson, TerraCode displays content from Google Docs together with mind maps and flashcards. Learners can continue practicing through Quiz, Matching, and Code Arrangement activities.. The Quiz function supports scoring, hints, answer

feedback, and saving incorrect answers for later review. Code Playground provides an online area where learners can write and run code.

The system records practice results, XP, and progress for each topic. Information about XP, level, streak, and daily quests is shown on the Dashboard so that learners can follow their progress. TerraCode also includes a virtual pet feature. Users can receive and manage features related to their pets based on their learning data and XQP.

For administrators, TerraCode provides functions for managing users, courses, topics, and lesson content. Administrators can create, edit, delete, and arrange courses or topics. They can also set unlock conditions and XP rewards. The system also supports the management of Quiz questions, Matching cards, and Code Arrangement exercises.

The administration area also provides statistics about learners, courses, and individual topics. Information such as completion numbers, accuracy rates, and average scores helps administrators identify the content that learners often find difficult. Based on these data, administrators can adjust learning content or practice activities when needed.

3.1.3. General System Workflow

The TerraCode workflow starts when a user accesses the system. A user who does not have an account can register with an email address and verify the account using a code sent by email. A user who already has an account can log in with an email and password or with a Google account. After successful authentication, the system creates a login session and redirects the user to the Dashboard.

On the Dashboard, TerraCode loads personal information, XP, XQP, learning streak, and current progress. The learner selects a course and then chooses a topic to study. The system combines learning content data with the learner's personal progress data to show the status of each topic. Topics that meet the required conditions are unlocked for the learner.

When the learner opens a lesson, the system gets the content from Google Docs and displays it on the learning page. The learner can read the lesson and view the

Mindmap or Flashcards. After the learner completes the lesson content, TerraCode updates the topic status and adds XP based on the lesson settings.

The learner can then complete practice activities such as Quiz, Matching, or Code Arrangement. The system loads the activity data for the selected topic, receives the learner's answers, checks the results, and saves the activity history. Scores and XP are then updated in the learner's profile. For Quiz activities, incorrect answers are saved for later review. The Quiz process includes loading questions, answering questions, checking results, saving history, and updating XP.

On the administration side, administrators create and update courses, topics, lessons, and practice activities. When AI-supported content is needed, the administrator selects a topic, links a Google Docs document, sends the content to Gemini AI, reviews the result, edits it, and approves it before publication.

Data created during the learning process is stored in two main groups. MASTER_DB stores common system data such as accounts, courses, topics, and question banks. USER_DB stores personal learning data for each learner, such as profiles, learning progress, activity history, and learning results. This structure helps TerraCode separate common system content from personal learning data and makes it easier to track the progress of each user.

3.1.4. System Users

TerraCode serves three main user groups: guests, learners, and administrators. Each group has its own access rights and functions based on its role in the system.

Guests are people who are new to TerraCode or do not have an account. They mainly view general information about the system, register an account, log in, verify their email, and recover their password when needed. After completing verification and login, the user is moved to the personal learning area.

Learners are the main users of the system. After logging in, learners can view their XP, XQP, learning streak, daily quests, and recent activities on the Dashboard. They can select courses and topics, read lessons, and view mind maps and flashcards. Practice activities such as Quiz, Matching, and Code Arrangement help learners check their understanding and record their results for each topic.

Learners also have a personal profile area where they can edit personal information, change their password, set learning goals, set reminder times, and follow information such as XP, XQP, and streak.

Administrators are responsible for managing the content and data of TerraCode. They manage user accounts, courses, topics, lessons, Quiz questions, Matching cards, and Code Arrangement exercises. Administrators can also monitor learning statistics and set XP rewards, unlock conditions, and content status.

For AI-supported content, administrators can link Google Docs materials to each topic and send the content to Gemini AI to generate Mindmaps, Quiz questions, and Matching Game content. The generated results are reviewed and edited before they are published for learners.

The division into three user groups helps TerraCode organize the system clearly. Guests focus on accessing the system and creating accounts, learners focus on learning and practice, while administrators are responsible for content, data, and system activity management.

3.2. System Requirements Analysis

3.2.1. Functional Requirements

Functional requirements describe the tasks that TerraCode must perform when users interact with the system. These functions are defined based on the roles of learners and administrators.

For user accounts and authentication:

- The system allows users to register an account using an email and password.
- The system sends an email verification code after registration.
- Users can log in using an email and password or a Google account.
- The system manages login sessions and allows only one active session for each account.
- Users can recover their password using a verification code sent by email.
- The system checks the USER and ADMIN roles before allowing access to administrator functions.

For learners:

- Learners can view the Dashboard, including XP, XQP, learning streak, daily quests, study calendar, and activity statistics.
- Learners can view the list of courses and topics, search for content, and track their completion status.
- The system checks the conditions of each topic before allowing learners to access the next content.
- Learners can view lesson content, mind maps, and flashcards.
- The system records lesson completion and updates XP.
- Learners can take a Quiz for each topic. The system checks answers, calculates scores, records time, gives answer feedback, and saves the results.
- Incorrect answers in Quiz activities are saved for later review.
- Learners can play Matching activities by matching terms with the correct definitions.
- Learners can complete Code Arrangement activities by arranging code blocks in the correct order.
- Learners can access Code Playground to write and run code online.
- The system updates XP, streak, and learning progress after learning activities.
- Learners can manage their personal profile, change their password, set learning goals, and set reminder times.
- Learners can manage their virtual pets and features related to XQP.

For administrators:

- Administrators can view and manage user account status.
- Administrators can view learner statistics, learning activities, and account status information.
- Administrators can track completion rates, average scores, and results by course or topic.
- Administrators can create, edit, delete, and arrange courses and topics.
- Administrators can set access conditions, display status, and XP rewards for each topic.
- Administrators can link Google Docs to lesson content.

- Administrators can manage the Quiz question bank, Matching cards, and Code Arrangement exercises.
- Administrators can use Gemini AI to generate Quiz questions, Mindmaps, and Matching Game content from lesson materials.
- AI-generated content must be previewed, edited, and approved before it is used as learning content.
- Administrators can manage email templates and schedules for learning reminder emails.

3.2.2. Non-Functional Requirements

Non-functional requirements focus on how TerraCode should operate when learners and administrators use the system.

- The interface should display clearly on computers and devices with different screen sizes. The learner sidebar can be collapsed on mobile devices. The system is built with HTML5 and CSS3 using responsive design.
- Common actions such as changing pages, loading courses, opening topics, and viewing lessons should respond within a reasonable time for online learning. TerraCode uses CacheService to temporarily store server data and reduce repeated loading of data that does not change often.
- The system must keep learning progress data consistent after learners complete lessons or practice activities. LockService is used when writing data to reduce conflicts when several write actions happen at nearly the same time.
- The system must control access based on user roles. Administrator actions must check the ADMIN role before processing. TerraCode also uses a single-session system, email verification, and processed passwords instead of storing original passwords directly.
- Common data and personal data must be clearly separated. TerraCode stores shared system data in MASTER_DB, while each learner has a separate USER_DB for personal information and learning progress.
- The system must record AI activities and errors to support checking when problems occur. AI_Generation_Logs stores the processing status, errors, token usage, and processing time for each AI request.

- Lesson content and data shown to learners must match the data published by administrators. When administrators update courses, topics, or practice content, the learner side must load the updated data correctly.
- The interface structure and navigation should be consistent across pages so that learners can easily know their current location and continue their activities. TerraCode uses a Single Page Application model, where page changes are handled on the client side.

3.2.3. System Constraints

TerraCode is developed as a Web App in the Google ecosystem. Therefore, the design and operation of the system have several constraints based on the selected platform.

- The system runs on Google Apps Script with the V8 Runtime. JavaScript ES6+ is the main programming language for both the user interface and server-side processing. The interface is built with HTML5 and CSS3.
- Google Sheets is used as the database. TerraCode does not use a separate database management system. MASTER_DB stores shared system data, while USER_DB stores personal data for each learner.
- Lesson content depends on Google Docs. The system reads content from Google Docs and converts it into HTML before displaying it to learners.
- File storage and personal databases depend on Google Drive. Therefore, access permissions and the status of Google services can directly affect some TerraCode functions.
- Google login functions depend on Google OAuth 2.0. Email verification, password reset, and learning reminder functions depend on the Gmail API.
- AI content generation functions depend on the Gemini API and a valid API key. If the AI connection has an error or the API does not respond, the learning functions already stored in the system remain separate from the AI content generation process. TerraCode also stores AI content in cache and records the status of each API request.
- The system uses a single-session mechanism. One account can have only one active login session at a time. If the user logs in from another session, the previous session is disabled.

– AI-generated content is not added directly to the learning content bank after it is created. Administrators must review, edit, or approve the content before it is made available to learners. The question management process includes content creation, preview, approval, and saving to the question bank.

3.3. Use Case Model

3.3.1. Overall Use Case

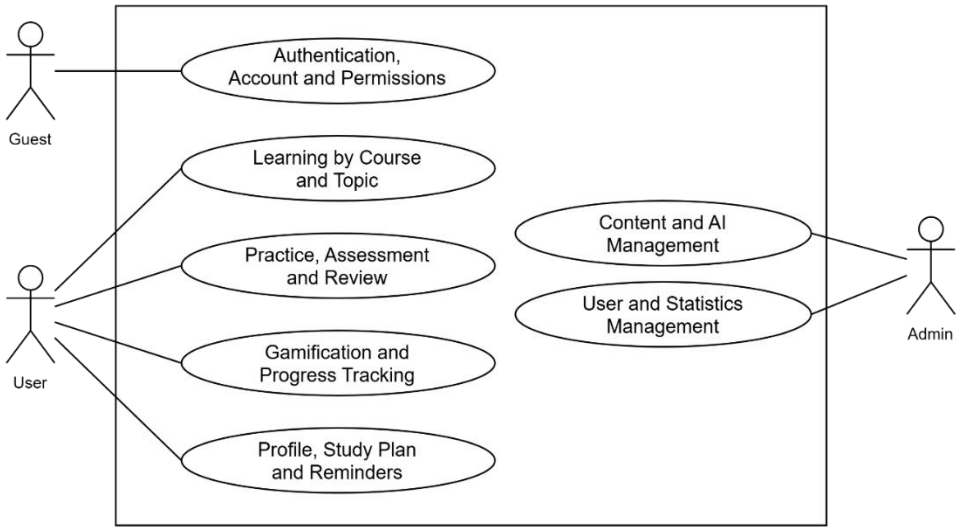

Figure 3.1 TerraCode Overall Use Case Diagram

– **Use Case Name:** Use the TerraCode System
– **Use Case ID:** UC01
– **Actors:** Guest, User, Admin
– **Brief Description:** This Use Case describes the main functions available to Guest, User, and Admin in the TerraCode system. Guest can access account authentication functions. User can access learning and practice functions, track learning progress, and manage personal information. Admin can access system management functions.
– **Preconditions:** The user has a device with an Internet connection and can access the TerraCode system.
– **Postconditions:** The user's request is handled, and any necessary data is updated.
– **Main Flow:**
  o The user accesses the TerraCode system.
  o The system checks the login status and identifies the user group.

- The system displays the available functions.
- The user selects a function.
- The system processes the request and updates the related data.
- The user continues using the system or ends the current session.

– **Alternative Flow:**
- Guest can register an account, verify an account, or log in to the system.
- User can access learning, practice, gamification, or profile functions.
- Admin can access content, user, or statistics management functions.

– **Exception Flow:**
- If the login session has expired, the system requires the user to log in again.
- If the user attempts to access a function without the required permission, the system denies access.
- If the system cannot load or update data, an error message is displayed.

3.3.2. Authentication and Account Use Case

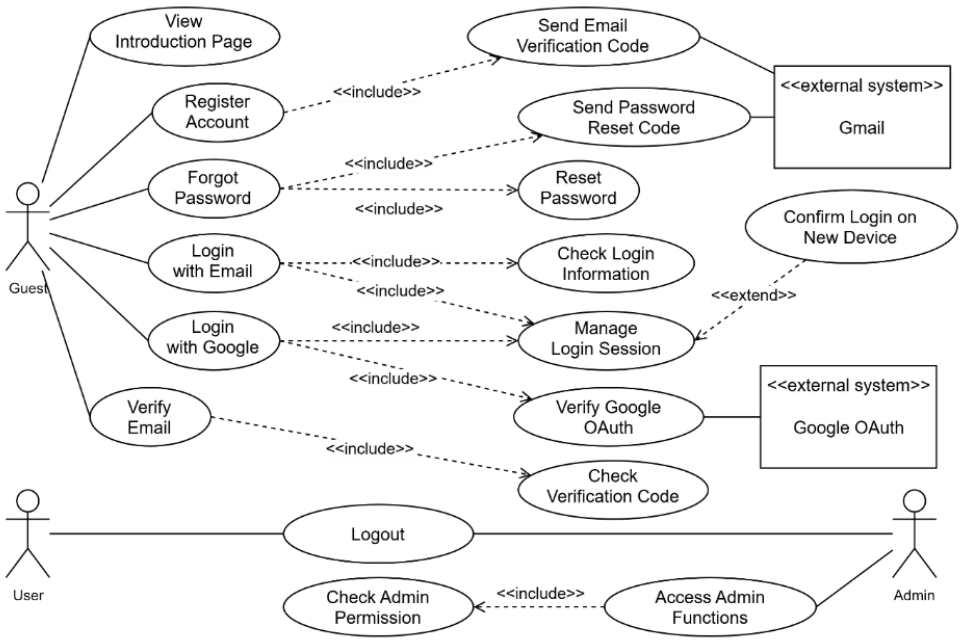

Figure 3.2 Authentication and Account Use Case Diagram

– **Use Case Name:** Authenticate and Manage Account

– **Use Case ID:** UC02

– **Actors:** Guest, User, Admin, Gmail API, Google OAuth 2.0

– **Brief Description:** This Use Case allows users to register an account, verify their email, log in using email/password or Google OAuth 2.0, change their password,

manage active login sessions, log out, and access functions according to their account role.

- **Preconditions:** The user has accessed the TerraCode system. For login, the account must already exist in the system.
- **Postconditions:** The account is created, verified, or updated depending on the performed action. After a successful login, the system creates a login session and determines the user's role.
- **Main Flow:**
  - Guest selects Register or Login.
  - The system receives and validates the submitted information.
  - The system verifies the account information.
  - If the information is valid, the system creates a login session.
  - The system determines whether the account role is User or Admin.
  - The user can access the functions assigned to the identified role.
- **Alternative Flow:**
  - Guest creates a new account, and the system sends a verification code to the registered email address.
  - Guest enters the verification code, and the system verifies the email address.
  - Guest signs in using a Google account through Google OAuth 2.0.
  - Guest uses the Forgot Password function, and the system sends a password reset code to the registered email address.
  - If the account already has an active session on another device, the system requests confirmation before creating a new session.
  - User or Admin ends the active session by logging out.
  - When Admin accesses an administration function, the system verifies Admin permission.
- **Exception Flow:**
  - If the email or password is incorrect, the login request is rejected.
  - If the verification code is incorrect or expired, the system asks the user to enter a valid code.
  - If the account is locked, the system denies the login request.

- If Google OAuth 2.0 authentication fails, the system returns the user to the Login page.
- If the system cannot send a verification or password reset email, an error message is displayed.

3.3.3. Course and Topic Learning Use Case

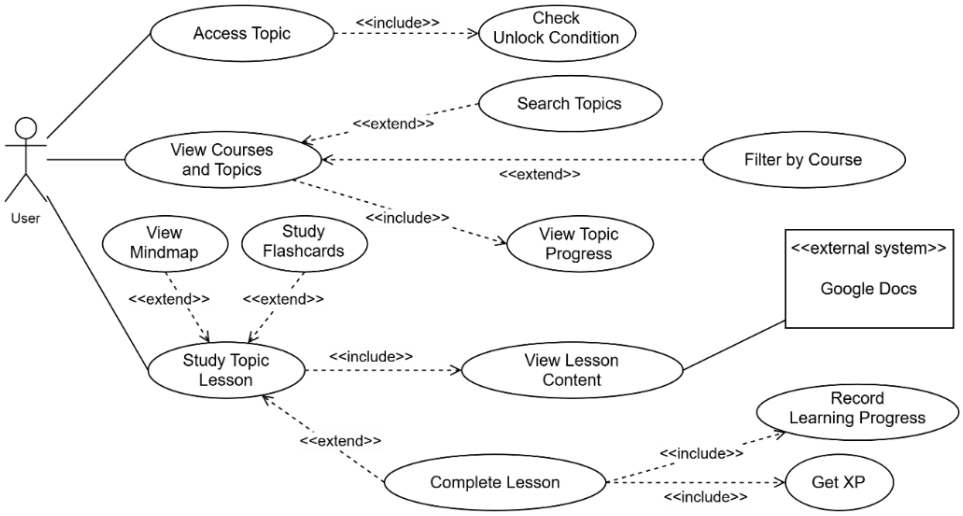

Figure 3.3 Course and Topic Learning Use Case Diagram

- **Use Case Name:** Learn by Course and Topic
- **Use Case ID:** UC03
- **Actors:** User, Google Docs
- **Brief Description:** This Use Case allows User to view courses and topics, search and filter learning content, check prerequisite conditions, study lessons, use Mindmap and Flashcard learning resources, and track learning progress and XP.
- **Preconditions:** User has logged in successfully. Course and topic data are available in the system.
- **Postconditions:** Learning progress is updated. When User completes a lesson, the system records the completion status and updates the related XP.
- **Main Flow:**
  - User opens the course and topic list.
  - The system displays the available courses, topics, and learning progress.
  - User selects a topic.
  - The system checks the prerequisite and unlock conditions.
  - The system displays the lesson content.

- User studies and completes the lesson.
- The system records the learning progress and updates XP.

– **Alternative Flow:**
- User searches for a topic by keyword.
- User filters the list by course.
- User views a Mindmap during the lesson.
- User uses Flashcards to review knowledge.

– **Exception Flow:**
- If the prerequisite or unlock condition is not satisfied, the system denies access to the topic.
- If the system cannot load lesson content from Google Docs, an error message is displayed.
- If the system cannot update learning progress or XP, the existing data is preserved and an error message is displayed.

3.3.4. Practice, Assessment and Review Use Case

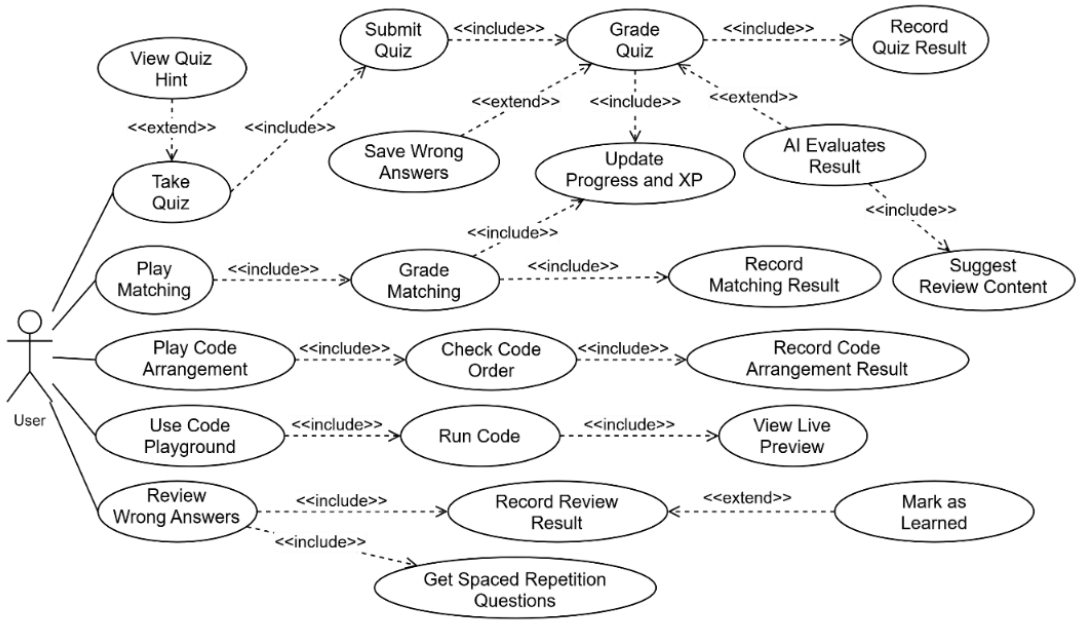

Figure 3.4 Practice, Assessment, and Review Use Case Diagram

– **Use Case Name:** Practice, Assessment, and Review

– **Use Case ID:** UC04

– **Actor:** User

- **Brief Description:** This Use Case allows User to complete Quiz, Matching, Code Arrangement, and Code Playground activities and review incorrect answers. The system checks results, records practice data, updates learning progress, and supports review through Spaced Repetition.
- **Preconditions:** User has logged in and has permission to access the selected practice content.
- **Postconditions:** Practice or review results are recorded. Progress, XP, and wrong-answer data are updated when needed.
- **Main Flow:**
  - User selects a practice or review activity.
  - The system loads the related practice content.
  - User completes the selected activity.
  - The system checks or scores the result.
  - The system records the practice result.
  - The system updates the related learning data.
- **Alternative Flow:**
  - User completes a Quiz, uses a hint when needed, and submits the answers for scoring.
  - The system can use AI to evaluate Quiz results and suggest related review content.
  - User completes a Matching activity, and the system records the result.
  - User completes a Code Arrangement activity, and the system checks the order of the code blocks.
  - User uses Code Playground to run code and view the output.
  - User reviews incorrect answers from the Spaced Repetition list.
  - A question is marked as learned when the required review condition is satisfied.
- **Exception Flow:**
  - If the system cannot load practice data, it shows an error message.
  - If the system cannot save Quiz or Matching results, it does not confirm completion.
  - If the code cannot run, the system shows an execution error.

- If no questions are available for review, the system indicates that no review content is currently available.

3.3.5. Gamification and Progress Tracking Use Case

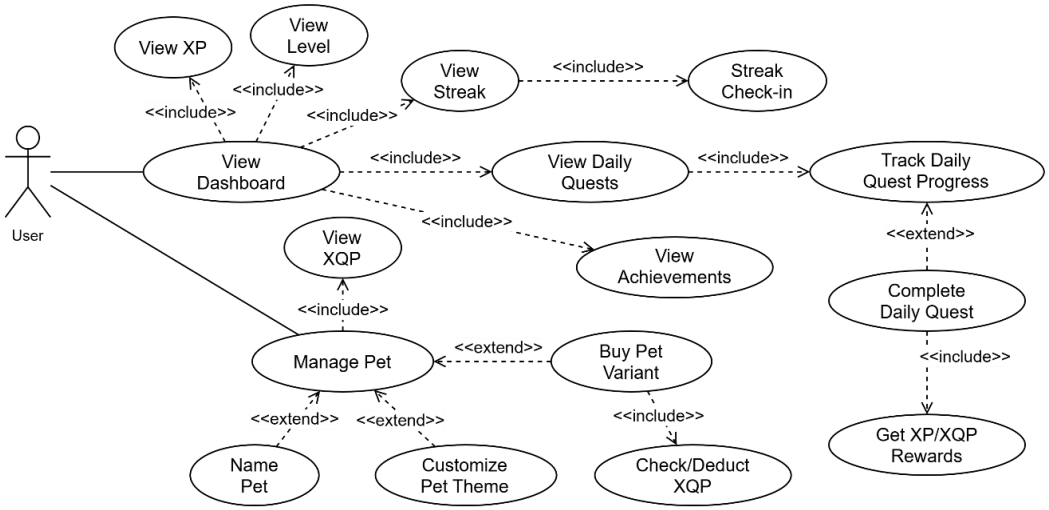

Figure 3.5 Gamification and Progress Tracking Use Case Diagram

- **Use Case Name:** Track Progress and Gamification
- **Use Case ID:** UC05
- **Actor:** User
- **Brief Description:** This Use Case allows User to track XP, XQP, Streak, Daily Quest, achievements, and other learning progress information on the Dashboard. User can also complete tasks, receive rewards, and interact with the Pet.
- **Preconditions:** User has logged in successfully, and the related learning progress data is available in the system.
- **Postconditions:** XP, XQP, Streak, Daily Quest, Pet data, and related progress information are updated according to the completed activities.
- **Main Flow:**
  - User opens the Dashboard.
  - The system displays learning progress and gamification information.
  - The system displays Daily Quests and achievements.
  - The system tracks the progress of related tasks.
  - When User completes an eligible task, the system updates the related XP or XQP reward.
  - User accesses and interacts with the Pet.

- **Alternative Flow:**
  - The system updates the Streak according to the User's learning activities.
  - User assigns a name to the Pet.
  - User changes the Pet theme.
  - User selects a Pet Variant for purchase.
  - Before the purchase is completed, the system checks the User's XQP balance and deducts the required amount if the balance is sufficient.
- **Exception Flow:**
  - If User does not have enough XQP, the purchase request is rejected.
  - If a reward has already been granted, the system does not grant the same reward again.
  - If Dashboard data cannot be loaded, the system displays an error message.

3.3.6. Profile, Study Plan and Reminder Use Case

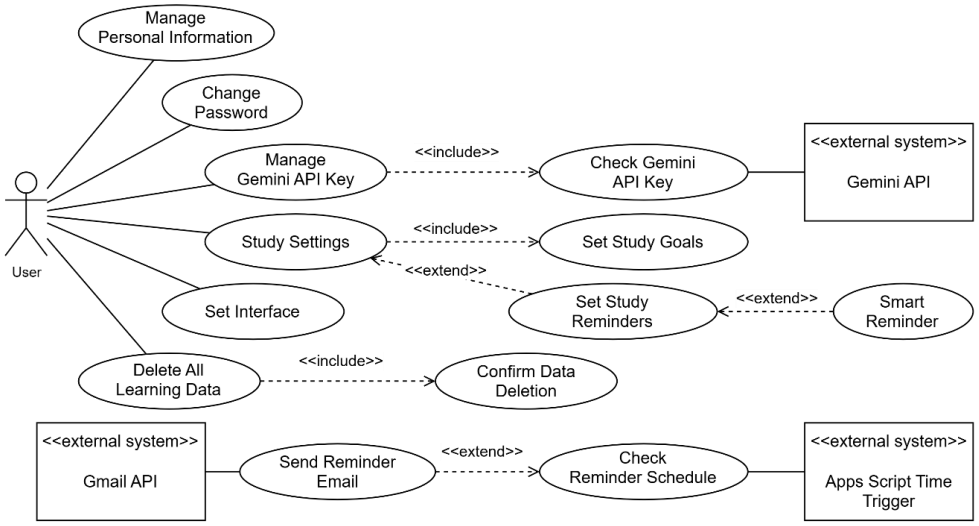

Figure 3.6 Profile, Study Plan, and Reminder Use Case Diagram

- **Use Case Name:** Manage Profile, Study Plan, and Reminders
- **Use Case ID:** UC06
- **Actors:** User, Gemini API, Gmail API, Apps Script Time Trigger
- **Brief Description:** This Use Case allows User to manage personal information, change the password, manage the Gemini API Key, configure interface settings, and manage learning settings. The system also supports learning goals, study reminders, Smart Reminder, and the deletion of personal learning data.
- **Preconditions:** User has logged in successfully.

- **Postconditions:** User information and settings are updated according to the completed actions. Reminder emails are sent according to the configured schedule. Personal learning data is deleted only after User confirms the deletion request.
- **Main Flow:**
  - User opens the Profile page.
  - The system displays the current personal information and settings.
  - User selects a function.
  - User enters or updates the required information.
  - The system validates and saves the submitted data.
  - The system applies the updated settings.
- **Alternative Flow:**
  - User manages personal information.
  - User changes the password.
  - User enters a Gemini API Key, and the system validates the key through the Gemini API.
  - User changes interface settings.
  - User sets learning goals.
  - User sets study reminders.
  - User enables Smart Reminder.
  - Apps Script Time Trigger checks the reminder schedule, and the system sends an email through the Gmail API when the configured conditions are met.
  - User requests the deletion of personal learning data and confirms the action.
- **Exception Flow:**
  - If the Gemini API Key is invalid, the system displays an error message.
  - If the new password does not meet the requirements, the system requests a valid password.
  - If a reminder email cannot be sent, the system records the error.
  - If User cancels the deletion confirmation, no data is deleted.
  - If the deletion process fails, the system displays an error message.

3.3.7. Content and AI Management Use Case

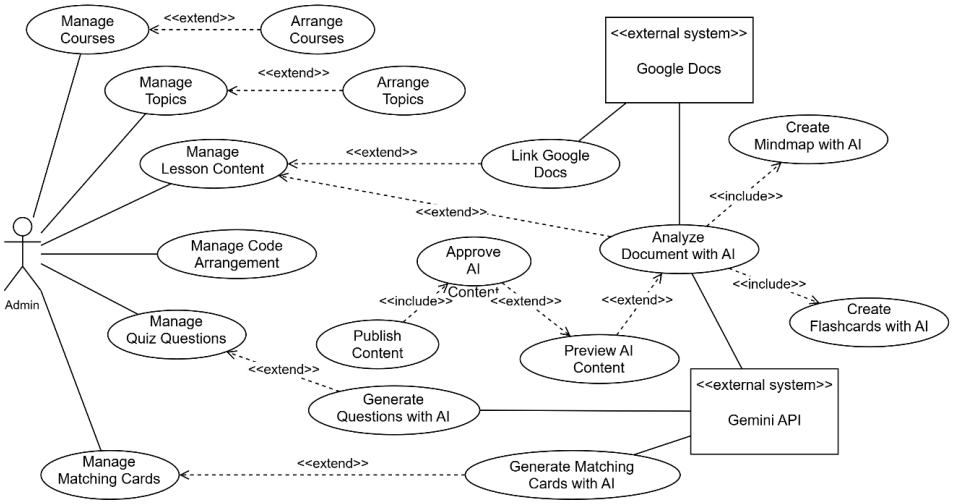

Figure 3.7 Content and AI Management Use Case Diagram

- **Use Case Name:** Manage Content and AI
- **Use Case ID:** UC07
- **Actors:** Admin, Google Docs, Gemini API
- **Brief Description:** This Use Case allows Admin to manage courses, topics, lesson content, Quiz questions, Matching Cards, and Code Arrangement exercises. The system also connects with Google Docs and the Gemini API to analyze lesson documents and generate learning content with AI.
- **Preconditions:** Admin has logged in successfully and has the required administration permission.
- **Postconditions:** Content or settings confirmed by Admin are updated. AI-generated content can be previewed, approved, and published.
- **Main Flow:**
  - Admin opens the content management module.
  - Admin selects a content group.
  - The system displays the related data.
  - Admin performs the required management action.
  - The system validates and saves the changes.
  - Approved and published content becomes available to User.
- **Alternative Flow:**
  - Admin changes the display order of courses.
  - Admin changes the display order of topics.

- Admin links lesson content to Google Docs.
- Admin requests the Gemini API to analyze a document.
- The system generates a Mindmap using AI.
- Admin previews and approves AI-generated content.
- Admin publishes the approved content.
- Admin generates Quiz questions using AI.
- Admin generates Matching Cards using AI.
- Admin manages Code Arrangement content.

– **Exception Flow:**

- If the account does not have Admin permission, the system denies access.
- If Google Docs cannot be accessed, the system stops the document processing operation and displays an error message.
- If the Gemini API does not respond or returns an error, the system reports that AI content generation has failed.
- If Admin does not approve AI-generated content, the content is not published.
- If the system cannot save the changes, the previous data is preserved and an error message is displayed.

3.3.8. User and Statistics Management Use Case

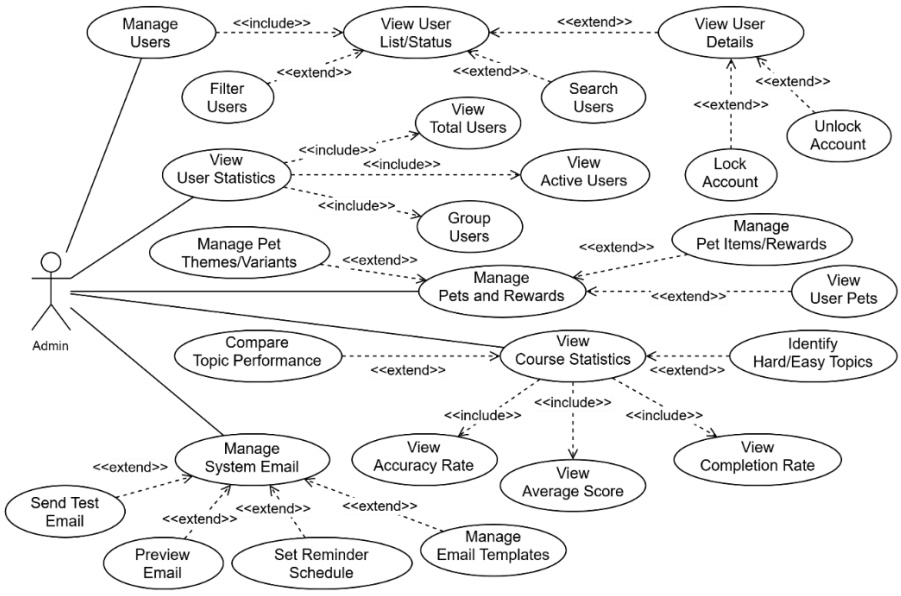

Figure 3.8 User and Statistics Management Use Case Diagram

– **Use Case Name:** Manage Users and Statistics

– **Use Case ID:** UC08

- **Actor:** Admin
- **Brief Description:** This Use Case allows Admin to manage user accounts, view user and course statistics, manage Pet data and rewards, and manage system email functions.
- **Preconditions:** Admin has logged in successfully and has Admin permission.
- **Postconditions:** Changes to user accounts, Pet data, rewards, or email settings are saved. Statistical data is processed and displayed to Admin.
- **Main Flow:**
  - Admin opens the User and Statistics Management module.
  - Admin selects a management function.
  - The system loads the related data.
  - Admin views or modifies the data.
  - The system processes the request.
  - The system saves the changes and displays the updated result.
- **Alternative Flow:**
  - Admin views, searches, filters, and checks user account details.
  - Admin locks or unlocks an account.
  - Admin views the total number of learners, active users, and user groups.
  - Admin views accuracy rates, average scores, and course completion rates.
  - Admin compares learning performance across different topics.
  - Admin identifies topics with high or low learning performance.
  - Admin manages Pet themes, variants, items, and rewards.
  - Admin views Pet information for individual users.
  - Admin manages email templates, previews emails, sends test emails, or configures reminder email schedules.
- **Exception Flow:**
  - If Admin permission is no longer valid, the system denies access.
  - If no matching user is found, the system indicates that no results are available.
  - If an account cannot be locked or unlocked, the existing account status is preserved.

- If the available statistical data is insufficient, the system displays the data that is currently available.
- If a test email cannot be sent, the system displays an error message.

3.4. System Architecture

3.4.1. Overall Architecture

The overall architecture of the system has four main layers. The Client layer is built with HTML5, CSS3, and JavaScript ES6+. It provides the interface for Guest, User, and Admin. The Server layer runs on Google Apps Script V8 Runtime. It receives requests from the Client and handles authentication, learning, practice, gamification, administration, and AI functions.

The Data Layer is built on Google Sheets. It includes MASTER_DB for shared system data and USER_DB for each user's personal data. Google Drive manages personal data files. In addition, the Server connects to Google Docs, Google OAuth 2.0, Gmail API, and Gemini API to perform related functions.

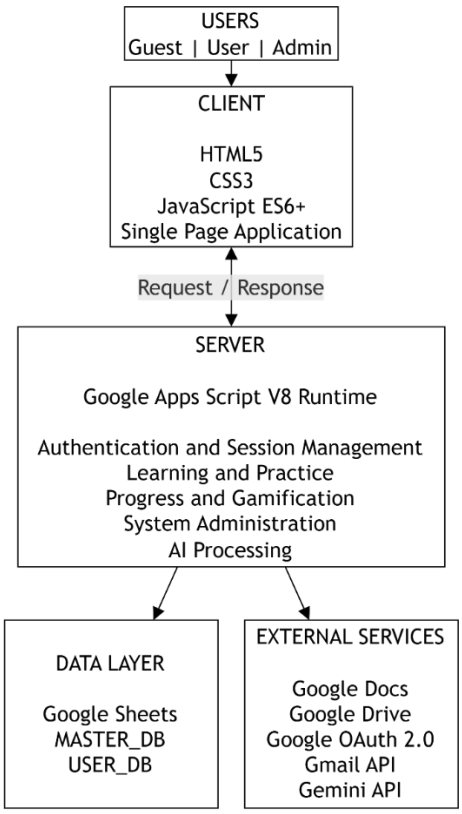

Figure 3.9 Overall System Architecture

3.4.2. Client-side Architecture

The Client side is built with HTML5, CSS3, and JavaScript ES6+. The interface follows the Single Page Application model. This allows users to move between different screens in the same application without reloading the whole page. The system divides the interface into three user roles: Guest, User, and Admin.

Guest can access registration, email verification, login, and password recovery functions. User can access the Dashboard, courses, topics, lessons, Quiz, Matching Game, Code Arrangement, Code Playground, Pet, and personal profile. Admin can access functions for managing users, courses, topics, learning content, and statistics.

The Client receives user actions, checks some input data, and sends requests to the Server. When the Server returns a result, the Client updates the related content on the interface. Libraries such as Chart.js, D3.js, and Markmap support charts and mindmaps. html2canvas supports exporting interface content as images.

3.4.3. Server-side Architecture

The Server side is developed with Google Apps Script on V8 Runtime. The Server receives requests from the Client, checks input data, verifies login sessions, checks access rights, and performs the related business logic. The main processing groups include accounts and login sessions, courses and topics, lesson content, Quiz, Matching Game, Code Arrangement, learning progress, gamification, user profiles, administration, and AI processing.

The Server also works as the middle layer between the Client, the Data Layer, and External Services. The Client does not directly access MASTER_DB or USER_DB. Instead, it sends requests to the Server. When data or an external service is needed, the Server reads or writes data, sends requests to the related service, and then returns the result to the Client.

Some Google Apps Script services are also used on the Server side. CacheService temporarily stores frequently accessed data. LockService controls cases where many requests write data at the same time. PropertiesService stores system settings, API keys, and other values used by the system.

3.4.4. Data Layer

The Data Layer is built on Google Sheets and Google Drive. Data is divided into two main groups: MASTER_DB and USER_DB. This structure separates shared system data from the personal learning data of each user.

MASTER_DB is the central Google Spreadsheet file. It stores account information, user statistics, courses, topics, Quiz questions, Matching data, Code Arrangement data, AI data, Pet data, and system logs. Sheets such as Users, Courses, Topics, MCQ_Questions, Matching_Term_Cards, Code_Arrangement, and AI_Content_Cache store different groups of data.

USER_DB is a separate Google Sheet file for each user. It stores personal profiles, topic progress, practice results, activity history, tasks, and review data. Each USER_DB is stored in Google Drive and is linked to the related account through information stored in MASTER_DB.

3.4.5. External Services

The system connects to several external services to perform functions outside the Client, Server, and Data Layer. Google Docs stores lesson content. The Server reads content from Google Docs, processes it into a suitable format, and sends it to the Client for display.

Google Drive manages data files and stores each user's USER_DB file. Google OAuth 2.0 supports login with a Google account. Gmail API sends email verification codes, password recovery codes, and learning reminder emails. Gemini API supports lesson analysis and the generation of Mindmaps, Quiz questions, Matching Game content, and new questions for reviewing incorrect answers. DiceBear API supports avatar creation for new accounts.

3.4.6. Client-Server Communication Flow

The communication flow starts when a user performs an action on the interface. The Client receives the data and sends a request to the Google Apps Script Server. The Server checks the input data, login session, and access rights when authentication is required. After that, the Server performs the related business logic.

If the request needs data, the Server reads or updates MASTER_DB and USER_DB. If the request is related to lesson content, Google login, email, or AI, the Server sends a request to the related service, such as Google Docs, Google OAuth 2.0, Gmail API,

or Gemini API. After processing is completed, the Server returns the data or result status to the Client. The Client receives the result and updates the interface for the user.

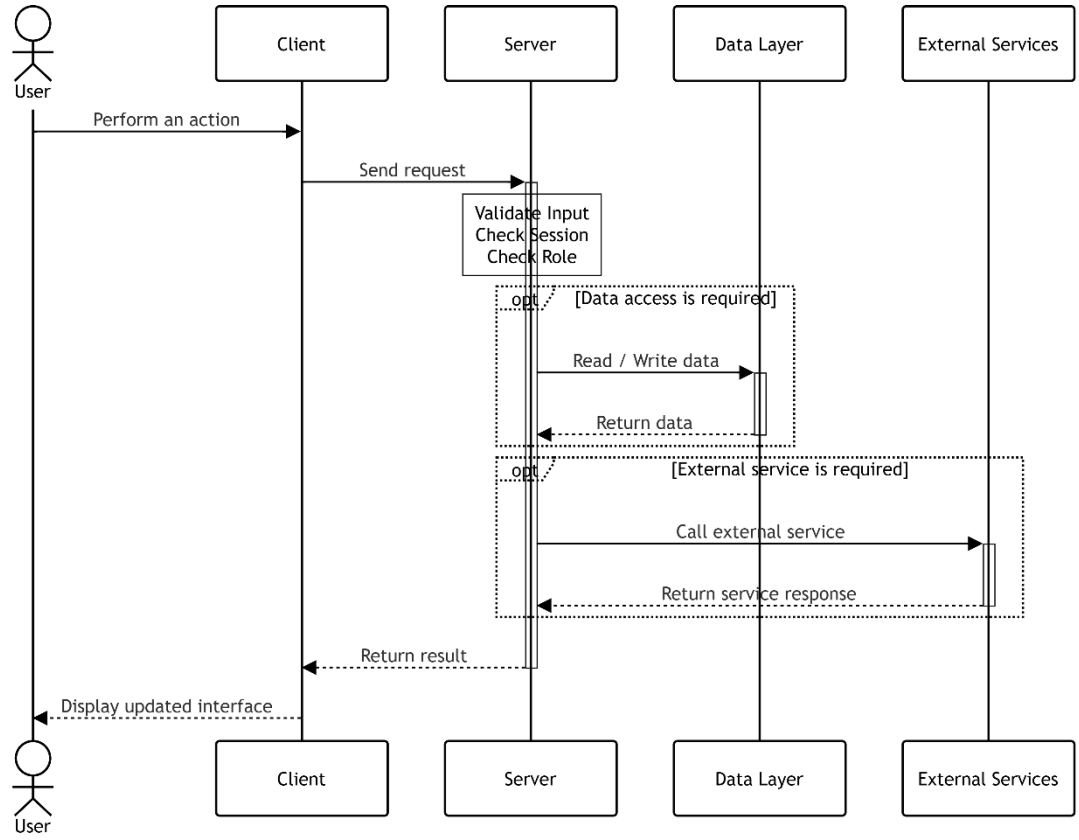

Figure 3.10 Client-Server Communication Sequence Diagram

3.5. Database Design

3.5.1. Database Overview

The system uses Google Sheets as its main data storage platform. Google Apps Script connects the application with Google Sheets. It is used to read, write, and process data. In addition, Google Drive is used to store each user's personal data files.

MASTER_DB and USER_DB are the two primary groups into which the system's data is separated. Data utilized by the entire system is stored in a shared Google Sheet called MASTER_DB. A Google Sheet called USER_DB is made especially for each user and kept in a designated Google Drive folder. This configuration makes it easier to manage and grow the system by separating data amongst accounts.

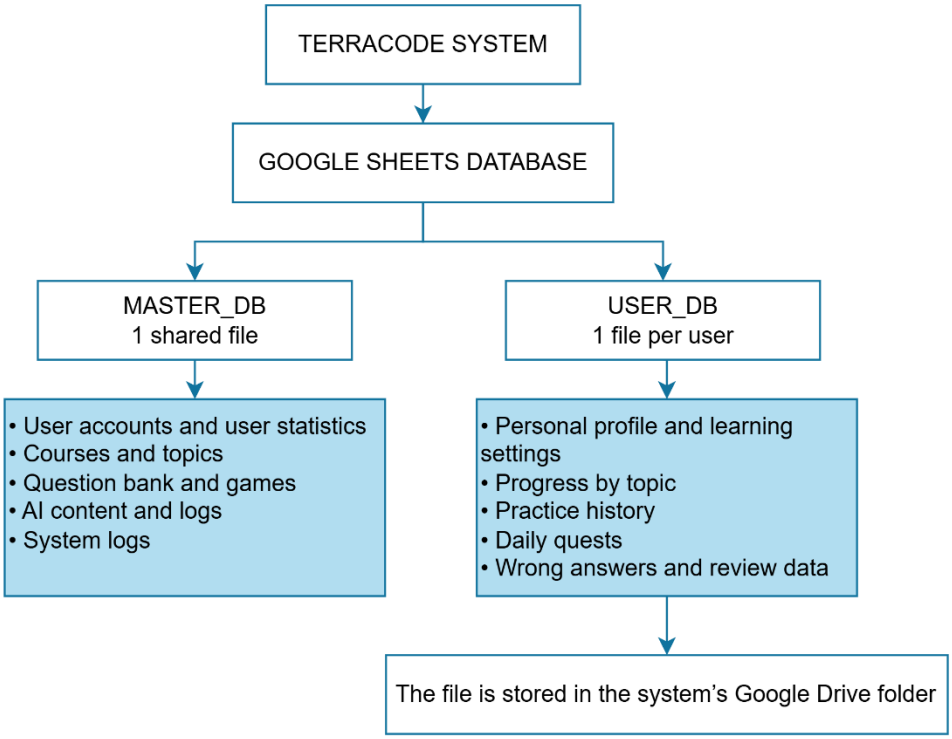

Figure 3.11 Overview of the TerraCode System Database Organization

3.5.2. MASTER_DB Design

MASTER_DB is the central Google Spreadsheet file that contains all shared data of the system. It includes 19 sheets divided into the following groups: user management, learning content, AI system, pet system, and system logs.

❖ **Sheet: Users**

The Users sheet stores account information for all users in the system, including identification data, login information, authentication status, active session information, learning reminder settings, and pet configuration.

Table 3-1 Structure of the Users Sheet

Column Name	Data Type	Description
userId	String	Unique user identifier
googleId	String	Google account ID
email	String	Login email address
displayName	String	User's display name
username	String	Account username, by default taken from the part before @ of the email

passwordHash	String	Hashed password or GOOGLE_OAUTH if logged in via Google
avatarUrl	String	URL of the user's avatar
role	String	User role: USER or ADMIN
isActive	Boolean	Account status: TRUE = active, FALSE = locked
createdAt	DateTime	Account creation time
lastLogin	DateTime	Most recent login time
lastActiveDate	DateTime	Most recent activity time
activeSessionId	String	ID of the currently active login session
activeSessionUpdatedAt	DateTime	Most recent update time of the active session
emailVerified	Boolean	Email verification status: TRUE /FALSE
verificationToken	String	Email verification token
verificationExpires	DateTime	Token expiration time
playerId	String	Short publicly displayed ID
progressSheetId	String	ID of the user's personal Google Sheet file
lastSeenAt	DateTime	Last time the system recorded the user as online
dailyGoal	Number	Daily learning goal in number of lessons
dailyTimeGoal	Number	Daily learning time goal
emailReminderEnabled	Boolean	Enable/disable learning reminders via email
reminderTimes	String (JSON)	List of reminder time points
reminderDays	String (JSON)	Days of the week to receive reminders

reminderLeadMinutes	Number	Number of minutes before the expected study time to send a reminder
repeatIfMissed	Boolean	Repeat reminder if missed
repeatIntervalMinutes	Number	Interval between repeated reminders
repeatMaxPerDay	Number	Maximum number of reminders per day
smartReminderEnabled	Boolean	Enable/disable smart reminders
reminderMode	String	Reminder mode: gentle, strict, smart
petConfig	String (JSON)	Current pet configuration of the user
totalXP	Number	Total experience points (XP)
totalXQP	Number	Total virtual currency (XQP)

❖ **Sheet: User_Stats**

The User_Stats sheet stores aggregated statistics on each user's learning process, including level, scores, streaks, and mountain-climbing progress.

Table 3-2 Structure of the User_Stats Sheet

Column Name	Data Type	Description
userId	String	User identifier
level	Number	User's current level
aiLevel	Number	Level assessed by AI based on actual ability
totalPoints	Number	Total accumulated points
totalXP	Number	Total experience points (XP)
totalXQP	Number	Total virtual currency (XQP) used to purchase items
currentStreak	Number	Current consecutive learning days
longestStreak	Number	Longest streak record
mountainPosition	Number	Position on the mountain-climbing map
mountainStage	Number	Current mountain-climbing stage
mountainProgress	Number	Progress percentage within the mountain-climbing stage

totalQuizAnswered	Number	Total number of quiz questions answered
totalPuzzleSolved	Number	Total number of puzzles solved
totalChallengeCompleted	Number	Total number of challenges completed

❖ **Sheet: User_Pets**

The User_Pets sheet stores information about the virtual pets owned by each user, including theme, name, and display configuration.

Table 3-3 Structure of the User_Pets Sheet

Column Name	Data Type	Description
userId	String	User identifier
theme	String	Currently used pet theme
petName	String	Pet name given by the user
petConfig	String (JSON)	Detailed pet configuration (JSON format)

❖ **Sheet: Courses**

The Courses sheet stores information about courses in the system, including title, description, display order, and unlock conditions.

Table 3-4 Structure of the Courses Sheet

Column Name	Data Type	Description
courseId	String	Unique course identifier
title	String	Course title
shortDescription	String	Short course description
description	String	Detailed course description
thumbnailUrl	String	URL of the course thumbnail
level	String	Course level
category	String	Course category
order	Number	Display order
status	String	Status: active, draft, archived
prerequisiteCourseIds	String (JSON)	List of prerequisite course IDs
unlockCondition	String (JSON)	Course unlock conditions
estimatedTime	String	Estimated learning time
createdBy	String	Course creator

createdAt	DateTime	Creation time
updatedAt	DateTime	Most recent update time

❖ **Sheet: Topics**

The Topics sheet stores information about lesson topics within each course, including document content, Google Docs links, cached HTML content (divided into multiple parts due to cell limitations), reward points, and display status.

Table 3-5 Structure of the Topics Sheet

Column Name	Data Type	Description
topicId	String	Unique topic identifier
title	String	Topic title
description	String	Topic content description
category	String	Category
order	Number	Display order within the course
iconUrl	String	URL of the topic icon
estimatedTime	String	Estimated learning time
prerequisiteTopics	String (JSON)	List of prerequisite topic IDs
isLocked	Boolean	Lock status: TRUE = not yet unlocked
unlockCondition	String (JSON)	Topic unlock conditions
createdBy	String	Topic creator
createdAt	DateTime	Creation time
updatedAt	DateTime	Most recent update time
contentDocId	String	ID of the Google Docs containing the lesson content
contentDocUrl	String	Direct URL to the content Google Docs
sourceHtml	String	Cached lesson HTML content
xpReward	Number	XP reward for completing the lesson
quizXpReward	Number	XP reward for completing the quiz
matchingXpReward	Number	XP reward for completing matching
isHidden	Boolean	Hide the topic from the user interface
course	String	Name of the course containing the topic

courseId	String	ID of the course containing the topic

❖ **Sheet: MCQ_Questions**

The MCQ_Questions sheet stores the Multiple Choice Questions (MCQs) created manually by administrators or generated by AI, used in tests and challenges.

Table 3-6 Structure of the MCQ_Questions Sheet

Column Name	Data Type	Description
questionId	String	Unique question identifier
topicId	String	Related topic ID
questionText	String	Question content
optionA	String	Answer A
optionB	String	Answer B
optionC	String	Answer C
optionD	String	Answer D
correctAnswer	String	Correct answer: A, B, C, or D
explanation	String	Answer explanation
difficulty	String	Difficulty: easy, medium, hard
hint	String	Hint for the user
points	Number	Reward points for a correct answer
relatedTopics	String (JSON)	List of related topics
imageUrl	String	Image URL
timeLimit	Number	Answer time limit
aiPromptTemplate	String	AI prompt template used to generate question variants
status	String	Status: draft, published, archived
source	String	Question source: manual, ai_generated
createdBy	String	Question creator
reviewedBy	String	Question reviewer
createdAt	DateTime	Creation time
updatedAt	DateTime	Most recent update time
publishedAt	DateTime	Publication time

topicTitle	String	Topic title

❖ **Sheet: Matching_Term_Cards**

The Matching_Term_Cards sheet stores term–definition card sets used in the matching game, helping users review vocabulary and concepts.

Table 3-7 Structure of the Matching_Term_Cards Sheet

Column Name	Data Type	Description
cardId	String	Unique card identifier
topicId	String	Related topic ID
topicTitle	String	Topic title
term	String	Term to be matched
definition	String	Full definition
shortDefinition	String	Short definition displayed on the card
example	String	Illustrative example
hint	String	Hint when the user encounters difficulty
difficulty	String	Difficulty: easy, medium, hard
tags	String (JSON)	Classification tags
status	String	Status: draft, published, archived
isActive	Boolean	Whether the card is currently being used
order	Number	Display order
source	String	Source: manual, ai_generated
createdBy	String	Creator
createdAt	DateTime	Creation time
updatedAt	DateTime	Most recent update time
approvedBy	String	Approver
approvedAt	DateTime	Approval time

❖ **Sheet: Code_Arrangement**

The Code_Arrangement sheet stores code arrangement exercises created by administrators, including original code, sliced blocks, and draft/published data.

Table 3-8 Structure of the Code_Arrangement Sheet

Column Name	Data Type	Description
arrangementId	String	Unique identifier

title	String	Arrangement exercise title
description	String	Requirement description
topicId	String	Related topic ID
programmingLanguage	String	Programming language
difficulty	String	Difficulty: easy, medium, hard
status	String	Status: draft, published
originalCode	String	Complete original code
slicedCodeBlocks	String (JSON)	List of sliced and shuffled code blocks
draftData	String (JSON)	Current draft data
publishedData	String (JSON)	Published version data
createdAt	DateTime	Creation time
updatedAt	DateTime	Most recent update time
orderIndex	Number	Display order in the list

❖ **Sheet: AI_Content_Cache**

The AI_Content_Cache sheet stores cached content generated by Gemini AI, helping reduce repeated API calls and processing costs.

Table 3-9 Structure of the AI_Content_Cache Sheet

Column Name	Data Type	Description
cacheId	String	Unique identifier
topicId	String	Related topic ID
contentDocId	String	Source Google Docs ID
contentType	String	Type of AI-generated content
generatedContent	String (JSON)	Generated content
promptUsed	String	Prompt template used
geminiModel	String	Gemini model used
tokensUsed	Number	Number of tokens consumed
generationTime	Number	Content generation time
qualityScore	Number	User quality rating
version	String	Content version
docLastModified	DateTime	Last modification time of the Google Docs

isActive	Boolean	Whether the content is currently being used
createdAt	DateTime	Creation time
expiresAt	DateTime	Cache expiration time

❖ **Sheet: AI_Generation_Logs**

The AI_Generation_Logs sheet records the history of all Gemini AI calls for content generation, supporting cost tracking, performance monitoring, and debugging.

Table 3-10 Structure of the AI_Generation_Logs Sheet

Column Name	Data Type	Description
logId	String	Unique identifier
userId	String	User ID triggering the request
topicId	String	Related topic ID
requestType	String	Type of AI generation request
inputDocId	String	Input Google Docs ID
inputDocLength	Number	Number of characters in the Google Docs
outputItemsCount	Number	Number of generated items
status	String	Status: pending, success, failed, partial
errorMessage	String	Error message
tokensConsumed	Number	Number of tokens consumed
costEstimate	Number	Estimated cost
processingTime	Number	Processing time
geminiModel	String	Gemini model used
promptVersion	String	Prompt template version
createdAt	DateTime	Execution time

❖ **Sheet: AI_User_Keys**

The AI_User_Keys sheet stores personal Gemini API keys provided by users, encrypted and managed with validation status.

Table 3-11 Structure of the AI_User_Keys Sheet

Column Name	Data Type	Description
keyId	String	Unique identifier
userId	String	ID of the user who owns the key
keyAlias	String	Descriptive name assigned by the user

keyHash	String	Hash of the API key
encryptedKey	String	Encrypted API key
isValid	Boolean	Whether the key is valid
status	String	Status: active, expired, revoked
lastValidatedAt	DateTime	Most recent validation time
lastUsedAt	DateTime	Most recent usage time
lastError	String	Most recent error when using the key
createdAt	DateTime	Time the key was added
updatedAt	DateTime	Update time

❖ **Sheet: AI_Key_Usage_Logs**

The AI_Key_Usage_Logs sheet records the history of Gemini API key usage for each user, supporting rate-limiting monitoring and error debugging.

Table 3-12 Structure of the AI_Key_Usage_Logs Sheet

Column Name	Data Type	Description
usageId	String	Unique identifier
userId	String	User ID
topicId	String	Related topic ID
contentType	String	Type of content requested for generation
model	String	Gemini model used
status	String	Status: success, error
httpCode	Number	HTTP status code returned
errorMessage	String	Error message
durationMs	Number	Processing time
createdAt	DateTime	Execution time

❖ **Sheet: Pet_Items**

The Pet_Items sheet stores the list of decorative items that can be purchased for virtual pets, including food, accessories, and backgrounds, together with their display positions.

Table 3-13 Structure of the Pet_Items Sheet

Column Name	Data Type	Description
itemId	String	Unique item identifier

itemType	String	Item type: food, accessory, backgrounds
name	String	Item name
file	String	Image file path
priceXqp	Number	Purchase price in XQP
unlockType	String	Unlock type: level, purchase, free
unlockValue	Number	Unlock condition value
petXpGain	Number	Amount of pet XP received when used
offsetX	Number	Horizontal offset position (px)
offsetY	Number	Vertical offset position (px)
scalePercent	Number	Scaling percentage (%)
positionMode	String	Positioning mode: absolute, relative
positionProfilesJson	String (JSON)	Position configuration for each pet type
orderIndex	Number	Display order in the shop
updatedAt	DateTime	Update time

❖ **Sheet: Pet_Variants**

The Pet_Variants sheet stores the list of virtual pet variants (skins), including images for each level and unlock conditions.

Table 3-14 Structure of the Pet_Variants Sheet

Column Name	Data Type	Description
variantId	String	Unique variant identifier
name	String	Variant name
tone	String	Primary color code
description	String	Variant description
level1File	String	Level 1 image file path
level2File	String	Level 2 image file path
eyeOpenFile	String	Open-eye image file path
eyeClosedFile	String	Closed-eye image file path
unlockType	String	Unlock type: level, purchase
unlockValue	Number	Unlock condition value
scalePercent	Number	Scaling percentage (%)

tiltDeg	Number	Display tilt angle
orderIndex	Number	Display order
updatedAt	DateTime	Update time
secondPetPriceXqp	Number	XQP price for purchasing a second pet of the same type

❖ **Sheet: Pet_Assets**

The Pet_Assets sheet stores shared graphic resources used by the virtual pet system, including images, animations, and display assets.

Table 3-15 Structure of the Pet_Assets Sheet

Column Name	Data Type	Description
assetId	String	Unique asset identifier
name	String	Asset name
type	String	Asset type: image, animation, icon
fileUrl	String	Asset file URL
priceXQP	Number	Purchase price in XQP
unlockType	String	Unlock type: level, purchase, free
unlockValue	Number	Unlock condition value
xpValue	Number	Amount of pet XP received
description	String	Asset description
createdAt	DateTime	Creation time
updatedAt	DateTime	Update time

❖ **Sheet: Admin_Food_Catalog**

The Admin_Food_Catalog sheet stores the food catalog for virtual pets managed by administrators, including selling price, pet XP received, and display status.

Table 3-16 Structure of the Admin_Food_Catalog Sheet

Column Name	Data Type	Description
foodId	String	Unique food identifier
foodName	String	Food name
xqpPrice	Number	Purchase price in XQP
petExpGain	Number	Amount of pet EXP received when feeding
active	Boolean	Whether the item is currently sold in the shop

sortOrder	Number	Display order in the shop
createdAt	DateTime	Creation time
updatedAt	DateTime	Update time

❖ **Sheet: Feature_Activity_Logs**

The Feature_Activity_Logs sheet records activities involving the main system features (lessons, quizzes, matching, and code games) to support administrator statistics.

Table 3-17 Structure of the Feature_Activity_Logs Sheet

Column Name	Data Type	Description
date	String	Activity date (YYYY-MM-DD)
timestamp	String	Detailed timestamp (YYYY-MM-DD HH:mm:ss)
userId	String	User ID or email
featureType	String	Feature type: lesson, quiz, matching, code_game
itemId	String	ID of the related topic or item

❖ **Sheet: StudyCalendarLogs**

The StudyCalendarLogs sheet stores daily learning calendar data for each user, supporting activity calendar display and daily goal tracking.

Table 3-18 Structure of the StudyCalendarLogs Sheet

Column Name	Data Type	Description
id	String	Unique identifier
userId	String	User ID
email	String	User email
date	String	Learning date (YYYY-MM-DD)
year	Number	Year
month	Number	Month
studyMinutes	Number	Total study minutes for the day
lessonCount	Number	Number of completed lessons
activityCount	Number	Number of activities performed
goalMinutes	Number	Daily study time goal
goalLessons	Number	Daily lesson goal
status	String	Status: achieved, partial, missed

lastActivityAt	DateTime	Last activity time of the day
createdAt	DateTime	Record creation time
updatedAt	DateTime	Record update time
source	String	Activity source: lesson, quiz, matching, flashcard, code_arrangement
theoryCount	Number	Number of theory lessons viewed
quizCount	Number	Number of quizzes completed
matchingCount	Number	Number of matching games played

❖ **Sheet: System_Logs**

The System_Logs sheet keeps track of system activities, which help with watching what's happening, fixing problems, and checking for security issues.

Table 3-19 Structure of the System_Logs Sheet

Column Name	Data Type	Description
logId	String	Unique identifier
timestamp	DateTime	Log timestamp
level	String	Level: INFO, WARNING, ERROR, DEBUG
category	String	Category: auth, database, api, system
userId	String	Related user ID
action	String	Action performed
details	String (JSON)	Additional details
ipAddress	String	IP address
sessionId	String	Login session ID
errorMessage	String	Error message

3.5.3. USER_DB Design

When someone first registers or logs in, the system automatically makes a special Google Spreadsheet for each user. The spreadsheet is named with the format `USER_DB_{userId}` and is stored in a specific Google Drive folder. This file has all of the user's personal information, including these 13 sheets:

❖ **Sheet: Profile**

The Profile sheet stores personal profile information and learning settings, including level, streak, daily goals, email reminder configuration, and virtual pet configuration.

Table 3-20 Structure of the Profile Sheet

Column Name	Data Type	Description
userId	String	User identifier
username	String	Account username
level	Number	Current level
totalXP	Number	Total accumulated XP
totalXQP	Number	Total virtual currency XQP
aiLevel	Number	Level assessed by AI
mountainStage	Number	Mountain-climbing stage
currentStreak	Number	Current consecutive learning days
bestStreak	Number	Highest streak record
totalTimeSpent	Number	Total learning time
joinedAt	DateTime	Time joined the system
lastActive	DateTime	Most recent activity time
dailyGoal	Number	Daily lesson goal
emailReminderEnabled	Boolean	Enable/disable email reminders
reminderTimes	String (JSON)	List of reminder times
reminderFrequency	String	Reminder frequency
reminderMode	String	Reminder mode
weeklyGoal	Number	Weekly learning goal
questsCompleted	Number	Total completed quests
streakRecoveryUsedAt	DateTime	Most recent time streak recovery was used
mostActiveHour	Number	Most active hour
reminderDays	String (JSON)	Days of the week to receive reminders
reminderLeadMinutes	Number	Number of minutes before the study time to send a reminder
repeatIfMissed	Boolean	Repeat reminder if missed
repeatIntervalMinutes	Number	Interval between repeated reminders
repeatMaxPerDay	Number	Maximum number of reminders per day

smartSuggestionUsed	Boolean	Whether smart suggestions have been used
smartReminderEnabled	Boolean	Enable/disable smart reminders
petConfig	String (JSON)	Virtual pet configuration

❖ **Sheet: Topic_Progress**

The Topic Progress sheet keeps track of how much the user has learned in each topic. It shows which activities like lessons, quizzes, matching games, flashcards, and mind maps have been completed. It also includes who can access the content and when it was accessed.

Table 3-21 Structure of the Topic_Progress Sheet

Column Name	Data Type	Description
progressId	String	Unique identifier
topicId	String	Topic ID
topicTitle	String	Topic title
quizDone	Boolean/Number	Main quiz completed
matchingDone	Boolean/Number	Matching game completed
challengeDone	Boolean/Number	Challenge/code game completed
attempts	Number	Total number of attempts
accuracy	Number	Accuracy rate (%)
bestScore	Number	Highest score achieved
xpEarned	Number	Total XP earned from the topic
aiScore	Number	AI assessment score
aiDecision	String	AI decision
status	String	Status: not_started, in_progress, completed
unlockedAt	DateTime	Time the topic was unlocked
completedAt	DateTime	Time the topic was completed
lessonCompleted	Boolean/Number	Theory lesson completed
mindmapViewed	Boolean/Number	Mindmap viewed
flashcardsCompleted	Boolean/Number	Flashcards completed

miniQuizCompleted	Boolean/Number	Mini quiz completed
completed	Boolean	Entire topic completed
progress	Number	Overall progress percentage
lastAccessed	DateTime	Most recent access time
accessGranted	Boolean	Special access granted
accessGrantedAt	DateTime	Time access was granted
accessGrantReason	String	Reason for granting access
accessGrantPrerequisiteIds	String (JSON)	List of prerequisite topic IDs that were bypassed
firstAccessAt	DateTime	First access time
lastUpdated	DateTime	Most recent update time

❖ **Sheet: Quiz_Results**

The Quiz_Results sheet keeps track of all the quizzes a user has done, along with their scores, how much time they took, and information about each question.

Table 3-22 Structure of the Quiz_Results Sheet

Column Name	Data Type	Description
id	String	Unique identifier
userId	String	User ID
mode	String	Mode: practice, exam, review
topicId	String	Topic ID
topicTitle	String	Topic title
score	Number	Score achieved
totalQuestions	Number	Total number of questions
percentage	Number	Correct answer rate (%)
timeTaken	Number	Time taken
gameMode	String	Game mode: normal, timed, survival
status	String	Status: completed, abandoned
currentQuestionIndex	Number	Current question index
completedAt	DateTime	Completion time
questionDetails	String (JSON)	Details of each question and answer

❖ **Sheet: Wrong_Answers**

The Wrong_Answers sheet stores a list of incorrectly answered questions for users to review, with a mechanism to mark questions as mastered when they have been learned.

Table 3-23 Structure of the Wrong_Answers Sheet

Column Name	Data Type	Description
id	String	Unique identifier
topicId	String	Topic ID
questionId	String	Question ID
question	String	Question content
selectedAnswer	String	Answer selected by the user
correctAnswer	String	Correct answer
retryCount	Number	Number of retry attempts
lastRetry	DateTime	Most recent retry time
mastered	Boolean	Whether the question has been mastered and no longer needs review
createdAt	DateTime	Time the error was recorded

❖ **Sheet: Wrong_Answer_Memory**

The Wrong_Answer_Memory sheet keeps track of the user's incorrect answers, which helps with spaced repetition and adaptive learning. Each wrong answer goes through five steps of checking over time, starting from 1 hour, then 1 day, 3 days, 7 days, and finally 14 days.

Table 3-24 Structure of the Wrong_Answer_Memory Sheet

Column Name	Data Type	Description
memoryId	String	Unique identifier
questionId	String	Question ID
topicId	String	Topic ID
conceptId	String	ID of the misunderstood concept
questionText	String	Question content
userAnswer	String	Answer selected by the user
correctAnswer	String	Correct answer
wrongCount	Number	Number of times answered incorrectly

lastAttempt	DateTime	Most recent attempt time
nextReviewDate	DateTime	Next review date
reviewStage	Number	Review stage: 0=1h, 1=1d, 2=3d, 3=7d, 4=14d
priority	Number	Priority level
isCleared	Boolean	Answered correctly consecutively and removed from the list
clearedAt	DateTime	Time cleared
variantsAttempted	String (JSON)	List of attempted variant IDs
notes	String	User's personal notes

❖ **Sheet: Matching_Results**

The Matching_Results sheet keeps track of the outcomes from every round of the matching game. It includes how many pairs were matched correctly or incorrectly, how much time was spent, and information about each matching done.

Table 3-25 Structure of the Matching_Results Sheet

Column Name	Data Type	Description
id	String	Unique identifier
userId	String	User ID
mode	String	Game mode: matching
topicId	String	Topic ID
topicTitle	String	Topic title
difficulty	String	Difficulty: easy, medium, hard
totalPairs	Number	Total number of pairs to match
correctPairs	Number	Number of correctly matched pairs
wrongAttempts	Number	Number of incorrect matching attempts
correctCount	Number	Total number of correct attempts
wrongCount	Number	Total number of incorrect attempts
moves	Number	Total number of moves
score	Number	Score achieved
elapsedTime	Number	Playing time

accuracy	Number	Accuracy rate (%)
completed	Boolean	Whether the game was completed
playedAt	DateTime	Time the game was played
pairDetails	String (JSON)	Details of each matched pair

❖ **Sheet: XP_Log**

The XP_Log sheet records the history of experience points (XP) and virtual currency (XQP) received by the user, including XP from daily quests, topic completion, and other activities.

Table 3-26 Structure of the XP_Log Sheet

Column Name	Data Type	Description
date	String	Date XP was received (YYYY-MM-DD)
timestamp	DateTime	Exact time
questId	String	Related quest ID
questTitle	String	Quest title
xpAmount	Number	Amount of XP received
source	String	Source: daily_quest, topic_completion, quiz, matching
xqpAmount	Number	Amount of XQP received
topicId	String	Related topic ID
eventId	String	Unique event ID
meta	String (JSON)	Additional data

❖ **Sheet: AI_Learning_Sessions**

The AI_Learning_Sessions sheet stores details of each learning session involving AI-generated content, tracking activities performed and results achieved during each session.

Table 3-27 Structure of the AI_Learning_Sessions Sheet

Column Name	Data Type	Description
sessionId	String	Learning session ID
topicId	String	Topic being studied
startedAt	DateTime	Session start time
completedAt	DateTime	Session completion time

lessonViewed	Boolean	Theory lesson viewed
mindmapViewed	Boolean	Mindmap viewed
infographicViewed	Boolean	Infographic viewed
flashcardsTotal	Number	Total flashcards in the session
flashcardsReviewed	Number	Number of flashcards reviewed
quizAttempted	Boolean	Quiz attempted
quizScore	Number	Quiz score achieved
quizAccuracy	Number	Correct answer rate (%)
totalTimeSpent	Number	Total learning time
scrollDepth	Number	Percentage of content read (%)
interactionCount	Number	Number of interactions
wrongAnswersCount	Number	Number of incorrect answers in the session
newConceptsLearned	Number	Number of new concepts learned
feedbackScore	Number	User rating
feedbackText	String	Written feedback

❖ **Sheet: Flashcard_Progress**

The Flashcard_Progress sheet stores the user's flashcard learning progress for each card, supporting the spaced repetition system.

Table 3-28 Structure of the Flashcard_Progress Sheet

Column Name	Data Type	Description
cardId	String	Flashcard ID
topicId	String	Topic ID
cardFront	String	Front of the card
cardBack	String	Back of the card
sourceConceptId	String	Source concept ID
difficultyRating	String	Self-assessed difficulty: easy, medium, hard
timesReviewed	Number	Number of times the card has been viewed
timesCorrect	Number	Number of times correctly recalled
lastReviewed	DateTime	Most recent review time
nextReview	DateTime	Next review time

reviewStage	Number	Stage: 0=new, 1=learning, 2=reviewing, 3=mastered
isMemorized	Boolean	Whether the card has been fully memorized
memorizedAt	DateTime	Time marked as memorized

❖ **Sheet: Mastered_Questions**

The Mastered_Questions sheet stores a list of questions that the user has answered correctly enough times to be marked as "mastered."

Table 3-29 Structure of the Mastered_Questions Sheet

Column Name	Data Type	Description
id	String	Unique identifier
topicId	String	Topic ID
questionId	String	Question ID
question	String	Question content
masteredAt	DateTime	Time marked as mastered
timesCorrect	Number	Number of consecutive correct answers

❖ **Sheet: Checkin_History**

The Checkin_History sheet stores the user's daily check-in history and serves as the basis for calculating streaks (consecutive learning days).

Table 3-30 Structure of the Checkin_History Sheet

Column Name	Data Type	Description
date	String	Check-in date (YYYY-MM-DD)
checkinTime	DateTime	Exact check-in time

❖ **Sheet: Login_History**

The Login_History sheet records user login information, such as the devices used and session duration.

Table 3-31 Structure of the Login_History Sheet

Column Name	Data Type	Description
id	String	Unique identifier
loginTime	DateTime	Login time
device	String	Device used
ipAddress	String	IP address

sessionDuration	Number	Session duration

❖ **Sheet: Activity_Log**

The Activity_Log sheet records all user activities, such as studying, taking tests, or playing matching games.

Table 3-32 Structure of the Activity_Log Sheet

Column Name	Data Type	Description
id	String	Unique identifier
type	String	Activity type: Learning, MCQ, Matching
topicId	String	Topic ID
topicTitle	String	Topic title
score	Number	Score achieved
totalQuestions	Number	Total number of questions
percentage	Number	Accuracy rate (%)
timestamp	DateTime	Time the activity was performed

3.6. Design of Main Processes by User Role

3.6.1. Processes for Guest

❖ **Account Registration and Verification Process**

The process begins when a Guest goes to the registration page and types in their email, creates a password, confirms the password again, and enters a display name. The system looks at the information and checks if the email is already used by another account. If the details are incorrect or the email is already taken, the guest gets a message and has to enter the information again. If the information is correct, a new account is made, the password is scrambled, the email is set as not verified, and a 6-digit code is generated for verification. This code is sent to the guest's email address. The guest types in the code, and the system verifies if it is right and still active. If the code is correct, the email is checked, the account is turned on, and the USER_DB is set up. The USER_DB link is also stored in the account. After that, the guest gets a message saying the action was successful and is then taken to the login page.

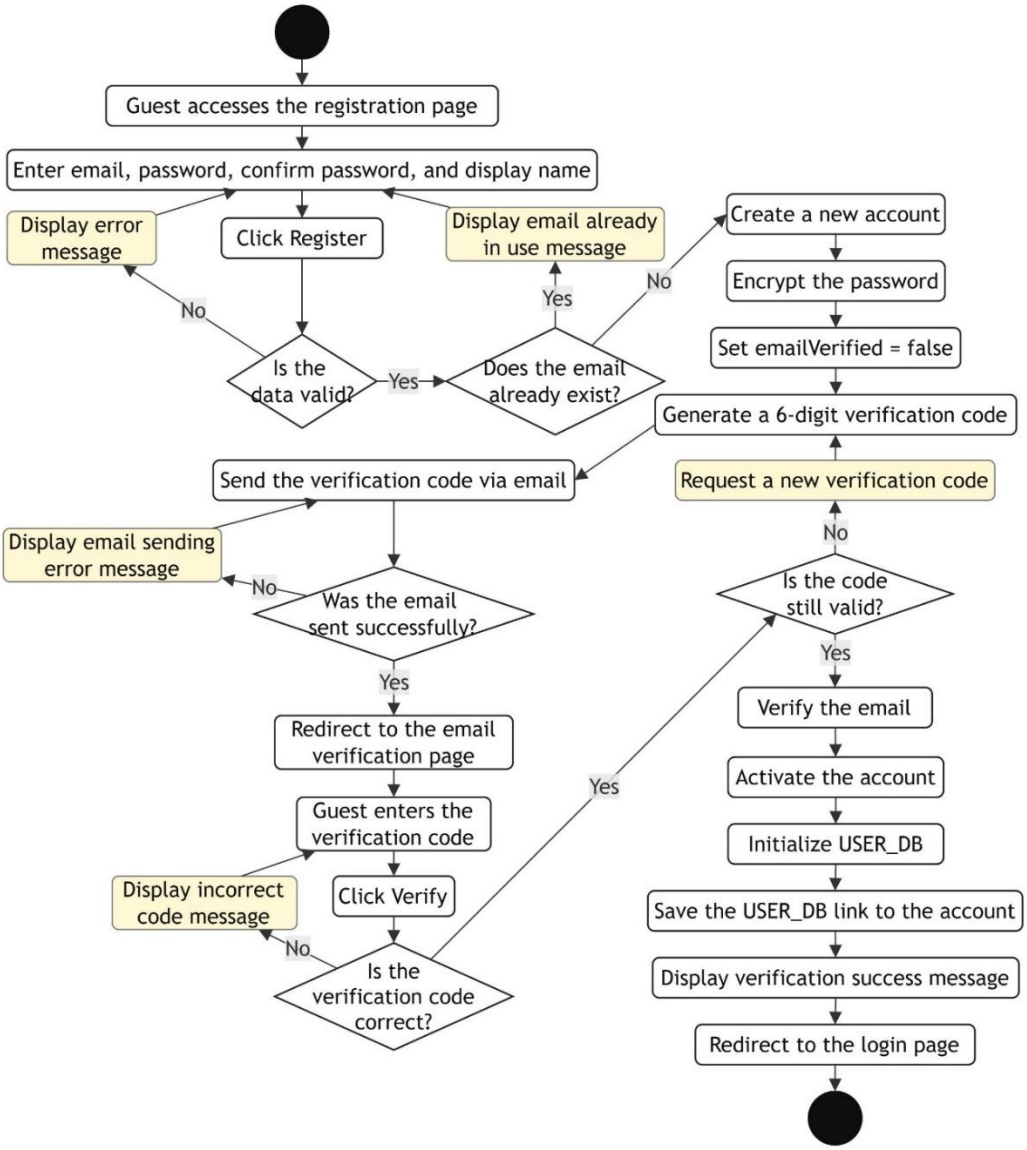

Figure 3.12 Diagram of Account Registration and Email Verification

❖ Login Process

A Guest can log in using two methods: Email/Password or Google OAuth. With Email/Password, the Guest enters the login information. The system then checks the information, account status, and email verification status. A locked account cannot continue the login process. If the email has not been verified, the Guest is moved to the verification page. If the account already has another login session, the system asks the Guest whether to replace it. When the Guest agrees, the old session is disabled and a new one is created. With Google OAuth, the Guest is sent to Google for verification. After successful verification, the system creates a new account or

updates the existing account and also creates a login session. At the end, the user role is checked, the session information is saved, and the user is moved to the Dashboard.

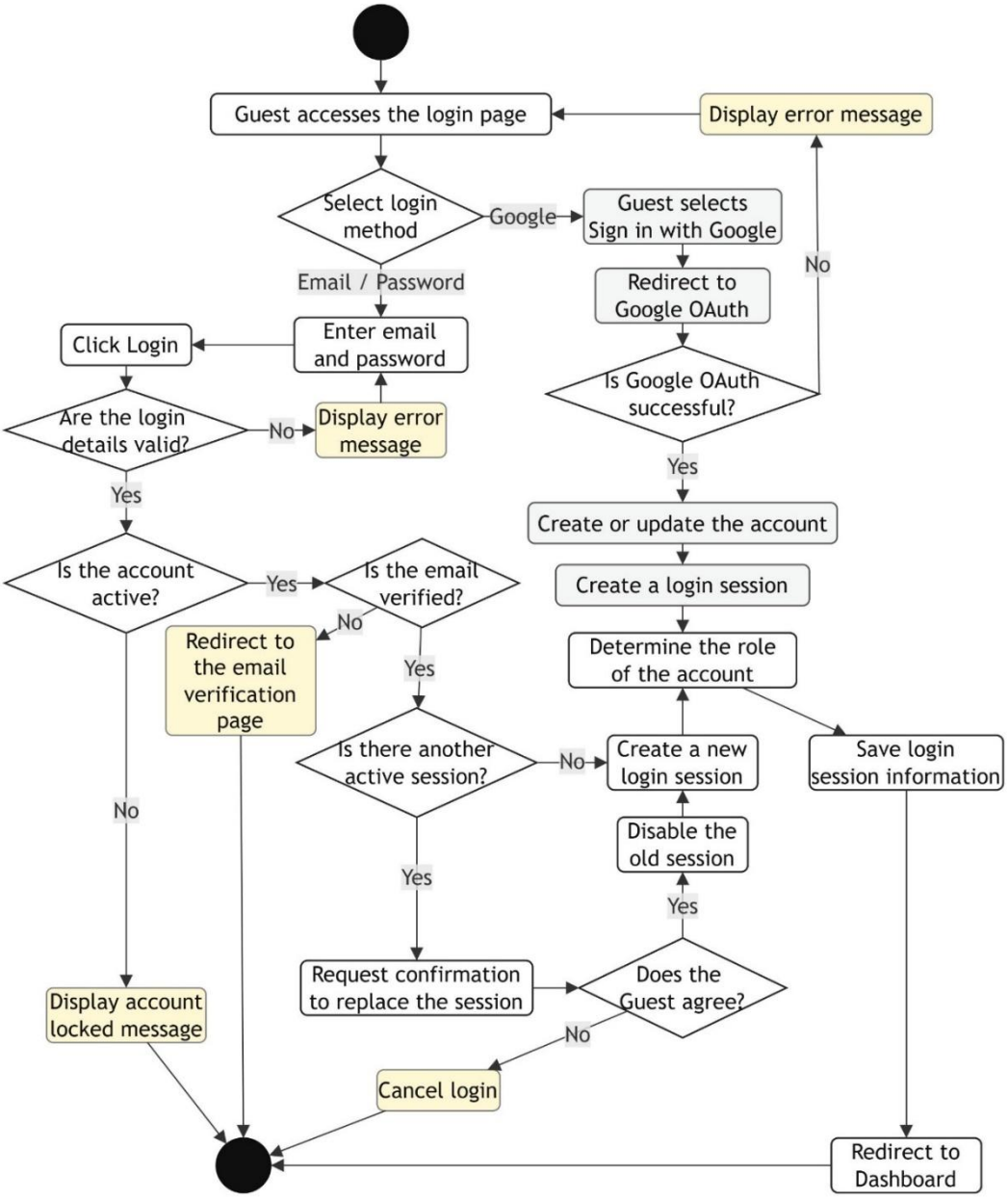

Figure 3.13 Diagram of the Login Process

❖ Forgot and Reset Password Process

The Guest selects Forgot Password, enters an email, and asks for a verification code. The system checks whether the email is valid and already exists in the system. If the email is valid, a 6-digit recovery code is created and sent by email. If there is an error when sending the code, the Guest receives a message and tries again. Next, the Guest enters the recovery code, a new password, and the password confirmation. The

system checks the code, its valid time, the new password requirements, and whether the two password fields match. When all information is correct, the new password is encrypted and updated in the account. The used recovery code is also disabled. The Guest then receives a success message and returns to the login page.

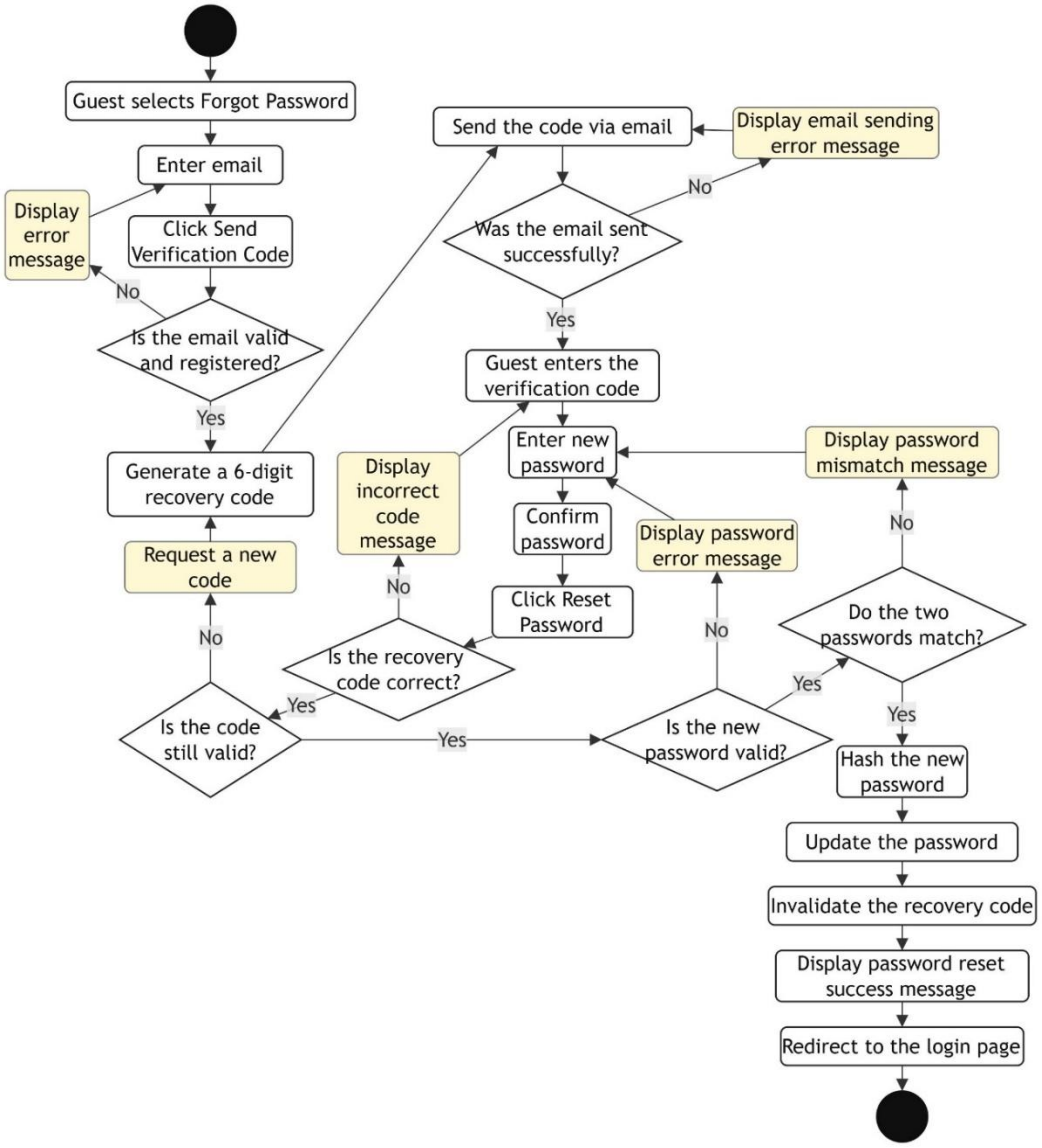

Figure 3.14 Diagram of the Forgot and Reset Password Process

3.6.2. Processes for User

❖ **Topic Learning Process**

The User opens the course list and chooses a course to study. The system shows the topics in that course together with the progress of each topic. When the User chooses a topic, the access conditions are checked before the lesson is opened. If the User does not meet the conditions, a message shows that the topic is locked and the User

returns to the topic list. If the conditions are met, the lesson content is loaded and displayed. The User studies the main content and can also view learning materials such as Mindmap and Flashcard. After the lesson is completed, the result is saved and Topic_Progress is updated. The Progress and Gamification Update Process is then started. If the lesson is not completed, the current progress is kept.

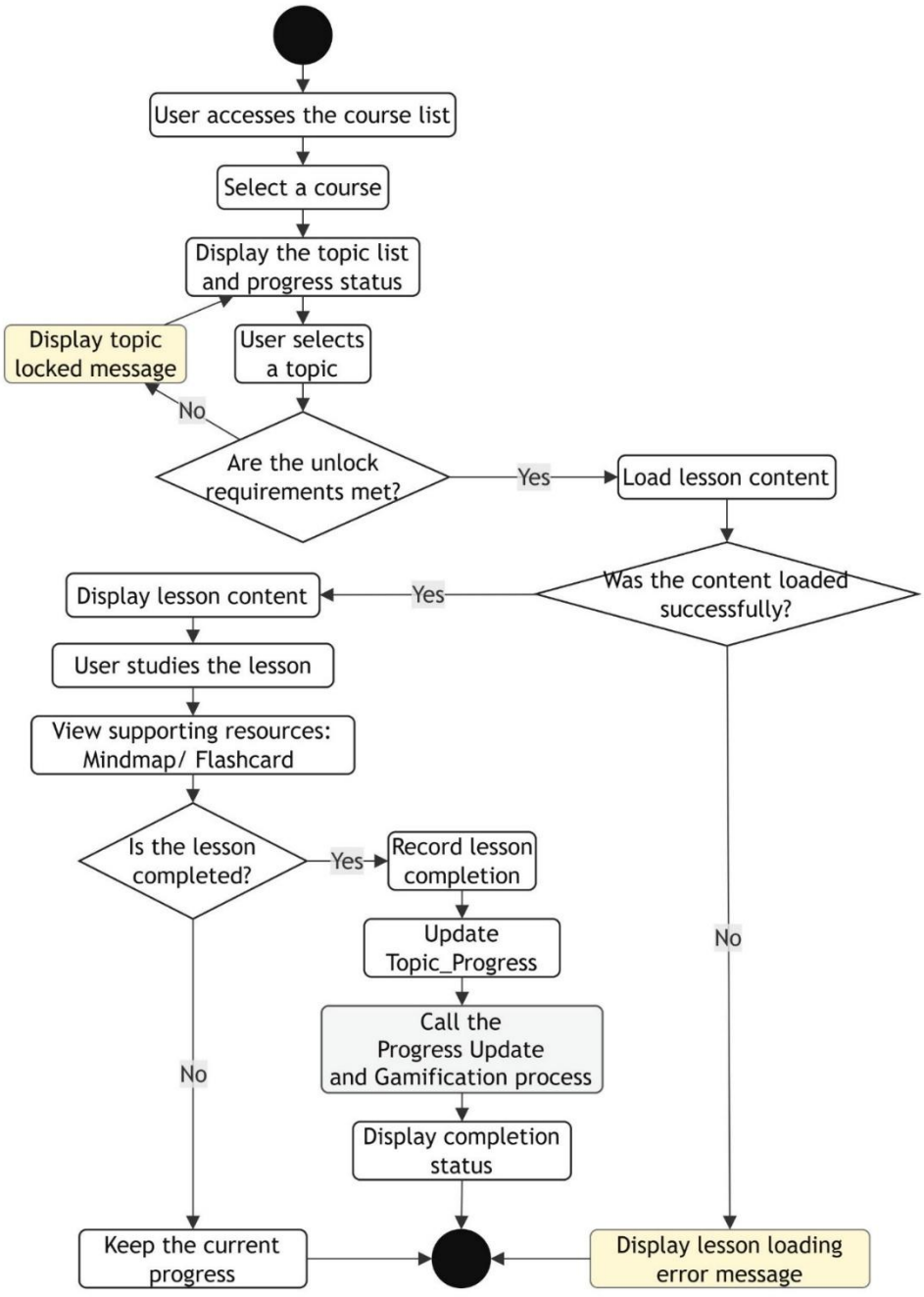

Figure 3.15 Diagram of the Topic Learning Process

❖ Quiz Process

The Quiz starts when the User chooses Quiz in a topic. First, the system loads the questions and checks the loading result. If there is an error, a message is shown. If the questions are loaded successfully, they appear on the screen for the User to answer. During the Quiz, the User can view a hint when needed and continue answering the questions. After the User submits the Quiz, the answers are checked and the score is calculated. Wrong answers are saved in Wrong_Answer_Memory for later review. The Quiz result and learning progress are also recorded. If AI evaluation is selected, AI analyzes the result and gives learning suggestions. Finally, the Progress and Gamification Update Process is started, and the score and result are shown to the User.

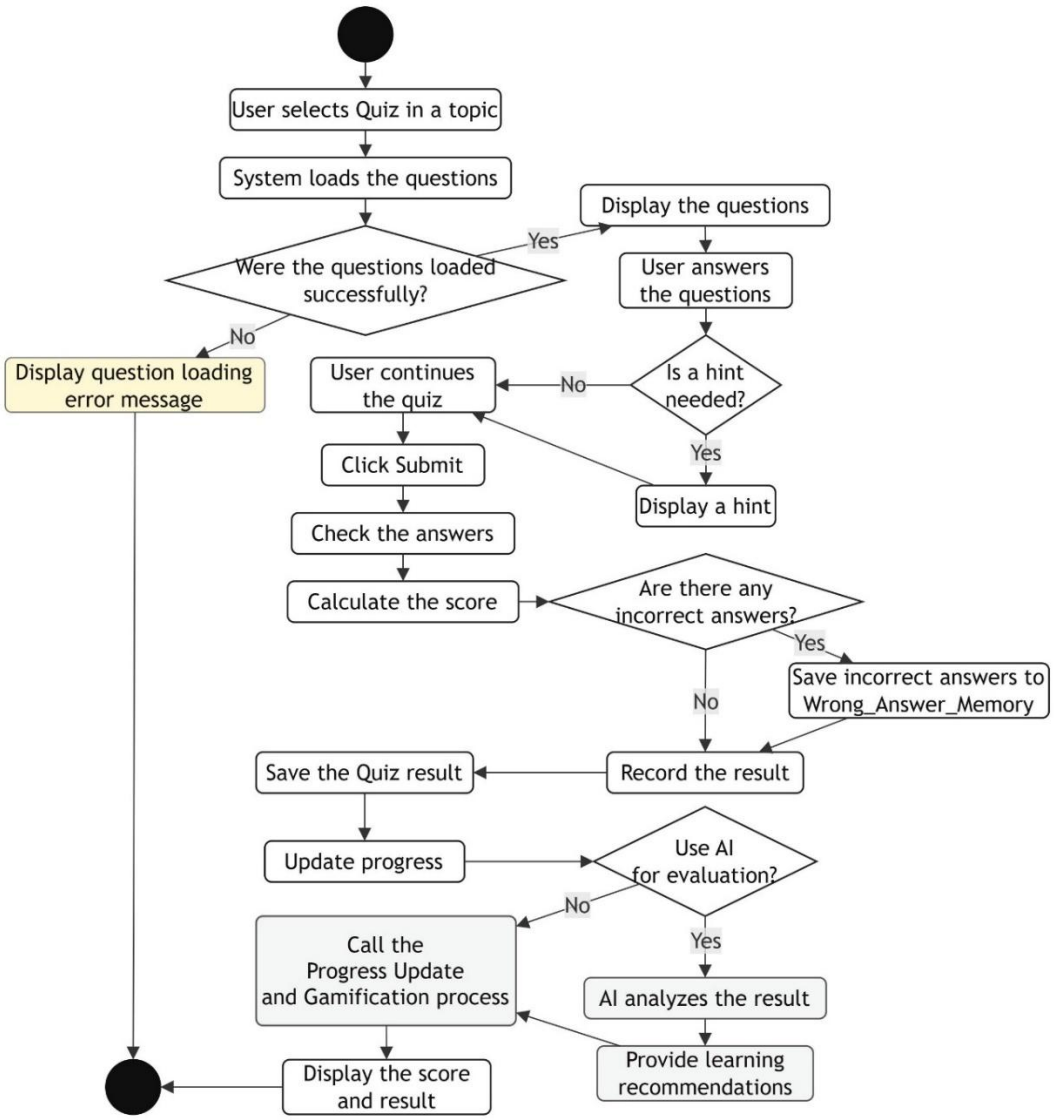

Figure 3.16 Diagram of the Quiz Process

❖ Matching Game Process

When the User selects Matching Game, the system loads the data and shows cards with terms and definitions. The User chooses the cards and matches them in pairs. After each match, the result is checked. If the pair is wrong, the User receives a message and can choose again. If the pair is correct, the result is recorded, and the system checks whether there are any pairs left. The game ends when all pairs are matched correctly. The score is then calculated, the Matching result is saved, and the learning progress is updated. After that, the Progress and Gamification Update Process is started. The final result is shown to the User.

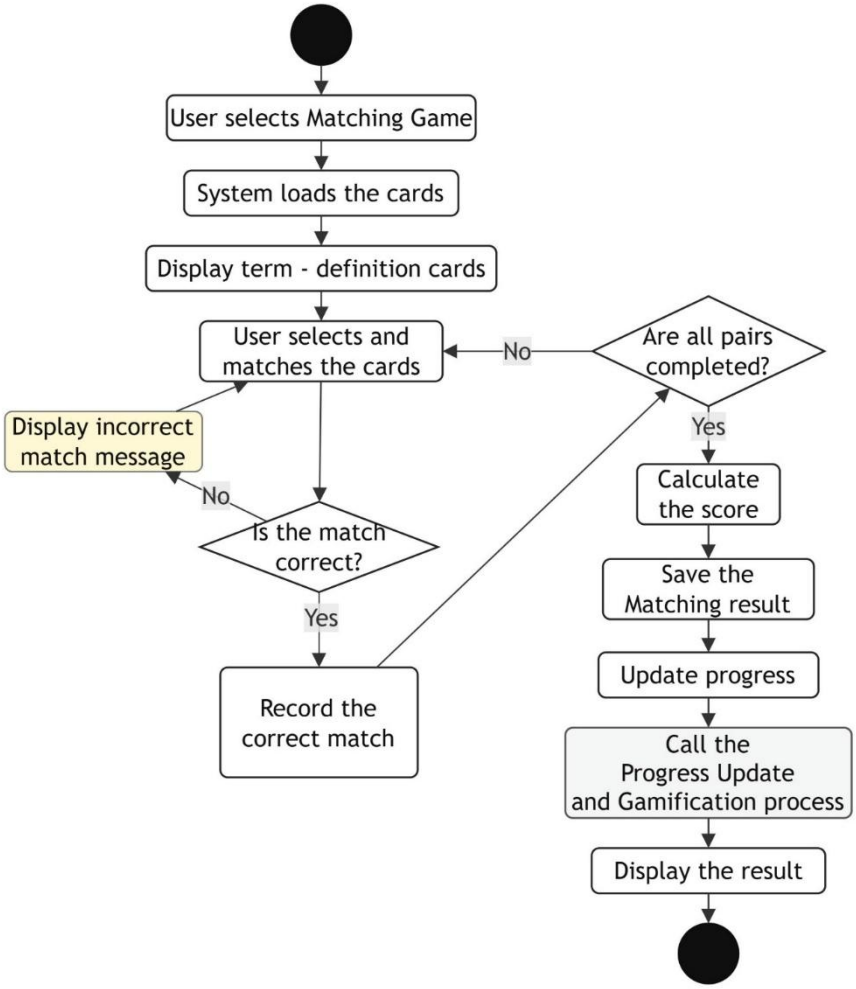

Figure 3.17 Diagram of the Matching Game Process

❖ Code Arrangement Process

The User opens Code Arrangement and chooses an exercise from the list. The system loads the code blocks of the exercise. If the data cannot be loaded, an error message is shown. If the data is loaded successfully, the code blocks are mixed and displayed on the screen. The User puts the blocks in the correct order and selects Check Answer.

The system compares the order with the correct answer. If the order is wrong, the User receives a message and continues arranging the blocks. When the order is correct, the exercise is marked as completed, the score is calculated, the result is saved, and the progress is updated. The Progress and Gamification Update Process is then started before the final result is displayed.

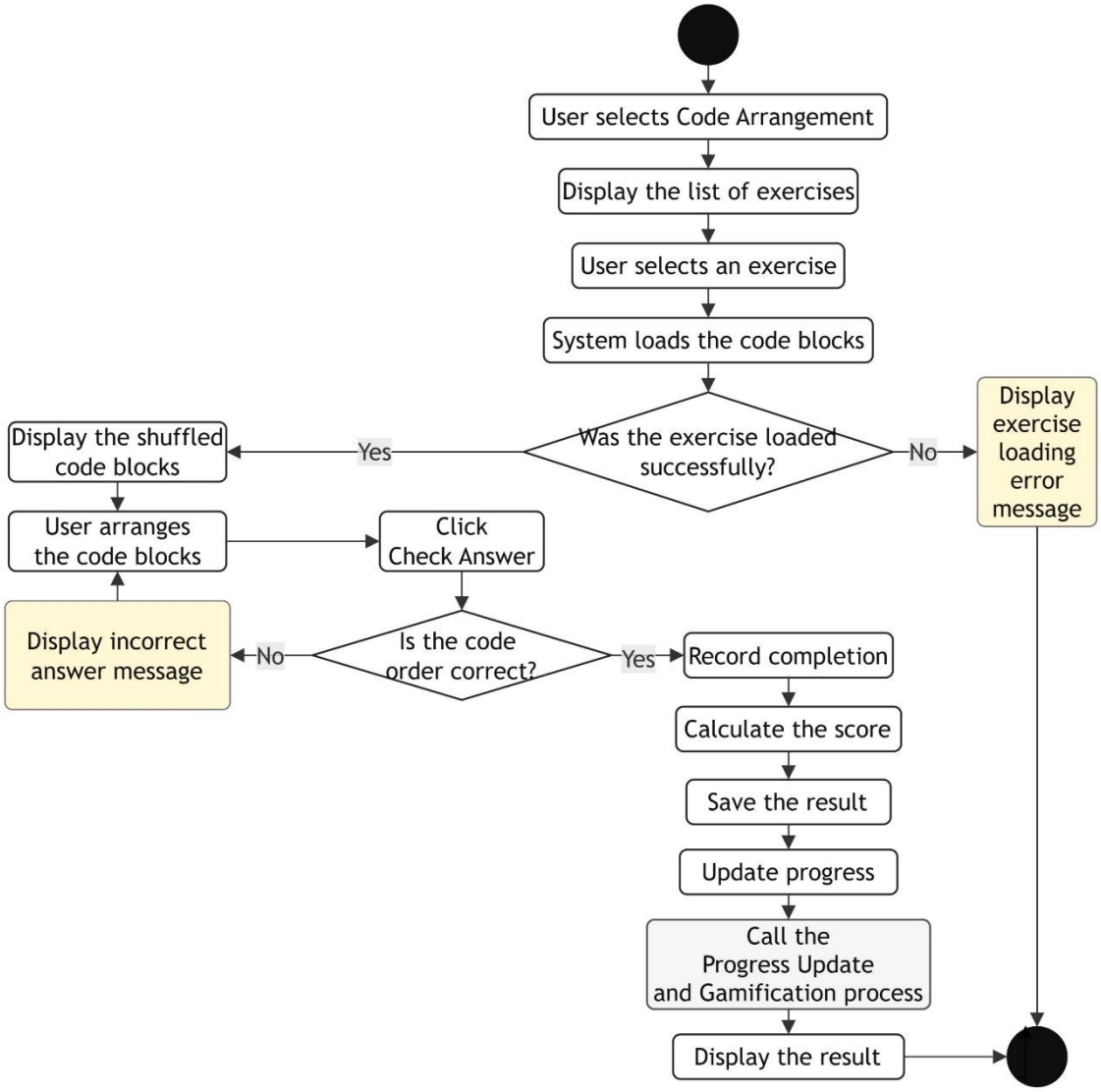

Figure 3.18 Diagram of the Code Arrangement Process

❖ Progress and Gamification Update Process

This process starts after the User completes a learning activity. The system receives the result and calculates the XP and XQP for that activity. The User's total XP, total XQP, and Streak are then updated. Next, the Daily Quest progress is recorded and checked. If the quest is completed, the task information and reward are saved. If it is not completed, the current progress is kept for the next time. The new data is updated

in User_Stats, while the XP history is saved in XP_Log. After the process is finished, the User can see XP, XQP, Streak, and other progress information.

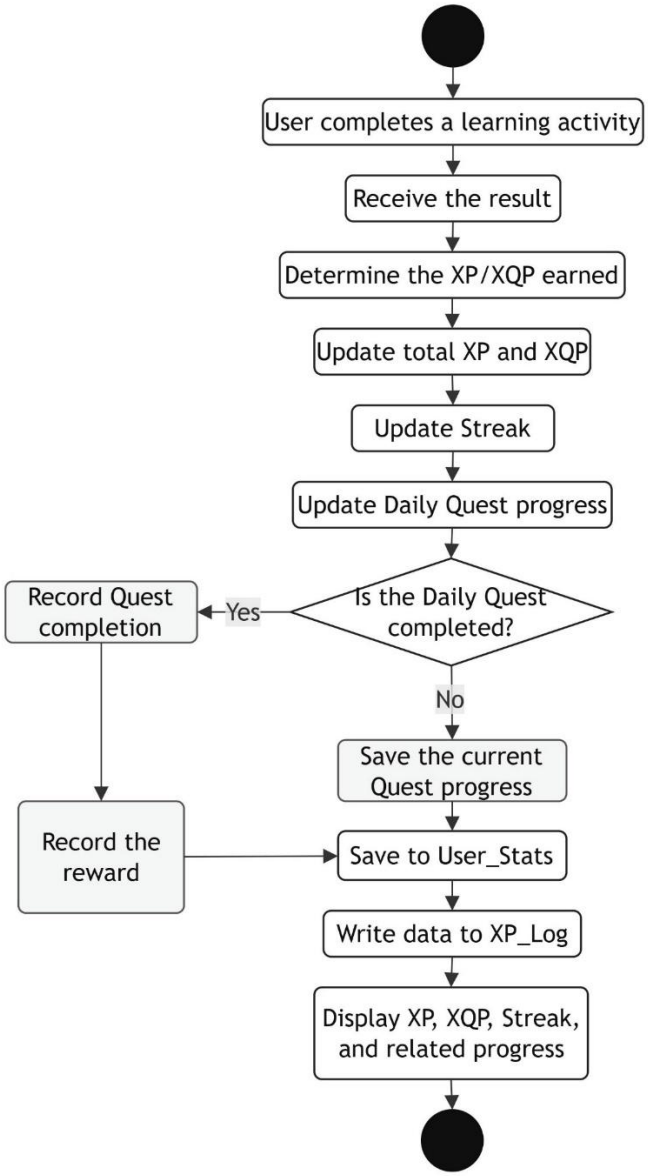

Figure 3.19 Diagram of the Progress Update and Gamification Process

❖ **Personal Learning Statistics Process**

Personal learning statistics come from the User's learning activities. After each activity, the related results are recorded and saved. When the User opens the Dashboard, the progress and Gamification data are collected and used to calculate the information that needs to be shown. The Dashboard displays XP, XQP, Streak, Daily Quest, recent activities, and the learning calendar. Charts are also created from the collected data. The User can use this information to follow learning results and progress over time.

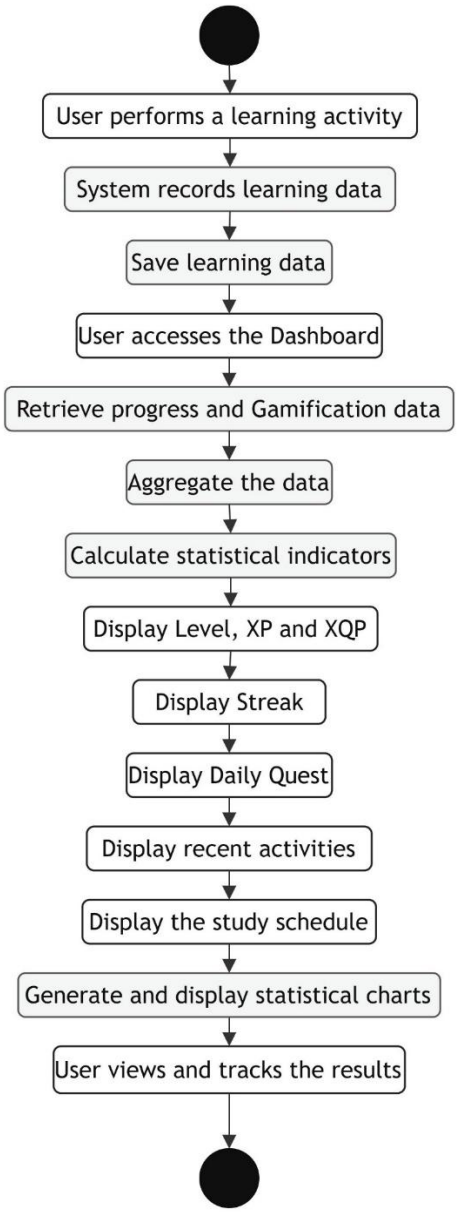

Figure 3.20 Diagram of the Personal Learning Statistics Process

3.6.3. Processes for Admin

❖ **Course and Topic Management Process**

The Admin opens the management function, and the system loads the current course and topic list. The Admin chooses an item to manage and can create, edit, delete, or arrange it. When working with a topic, the Admin can also set prerequisites, access conditions, XP rewards, display status, and a Google Docs link. The information is checked before it is saved. If the Admin chooses to delete an item, confirmation is required first. After saving the data, the Admin can publish the content or keep it unpublished. The system then updates the data and shows the result of the action.

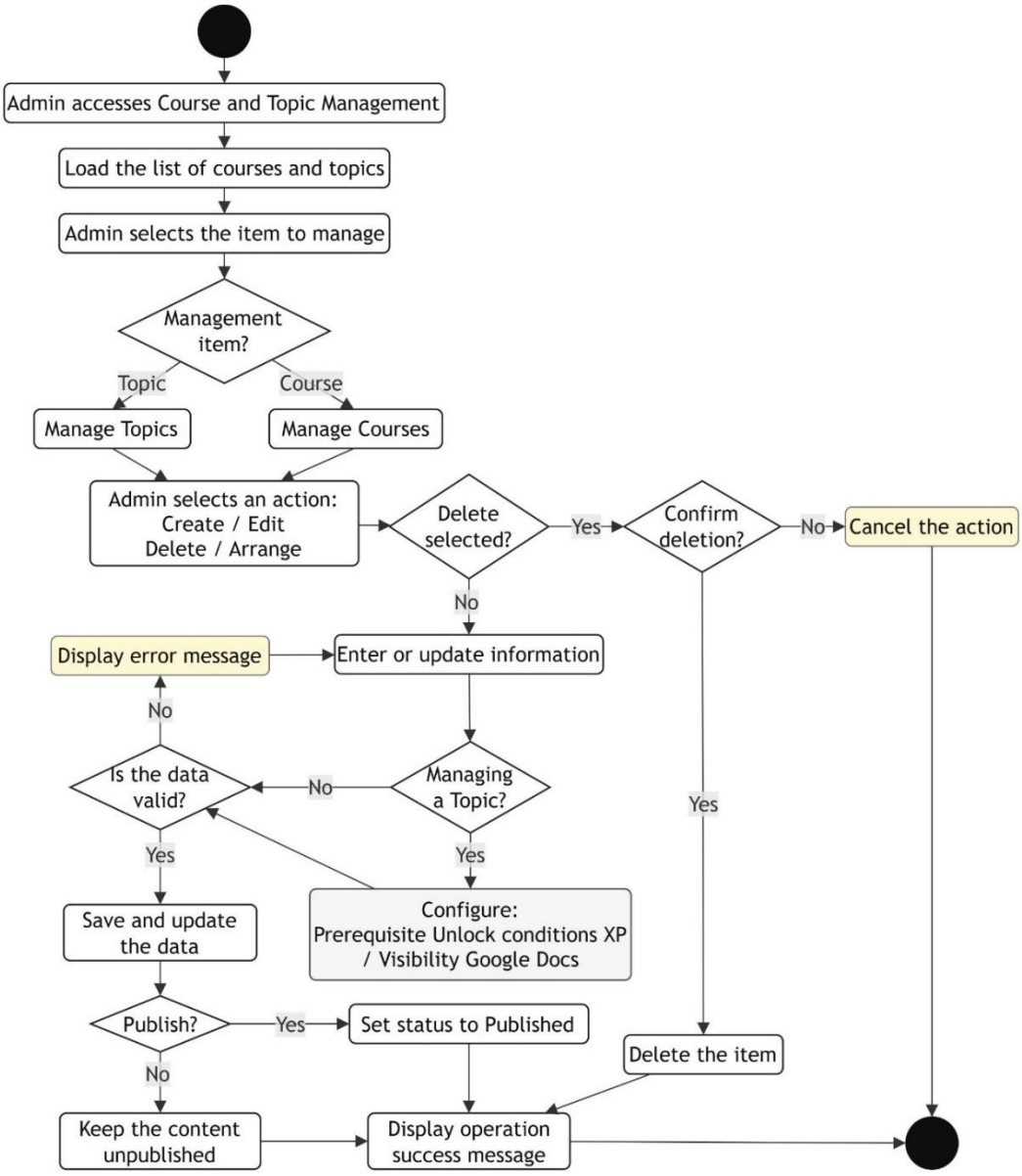

Figure 3.21 Diagram of the Course and Topic Management Process

❖ Practice Content Management Process

This process is used for Quiz, Matching, and Code Arrangement. The Admin chooses a course, a topic, and the type of content to manage. The related data is loaded, and the Admin can create new content, edit existing content, or delete it. Each type has different data. Quiz includes questions and answers, Matching includes terms and definitions, and Code Arrangement includes source code and code blocks. The system checks the information before saving it. After that, the Admin reviews the content. If it is not ready, the Admin continues editing. When the content is accepted, the Admin approves and publishes it so that Users can access it.

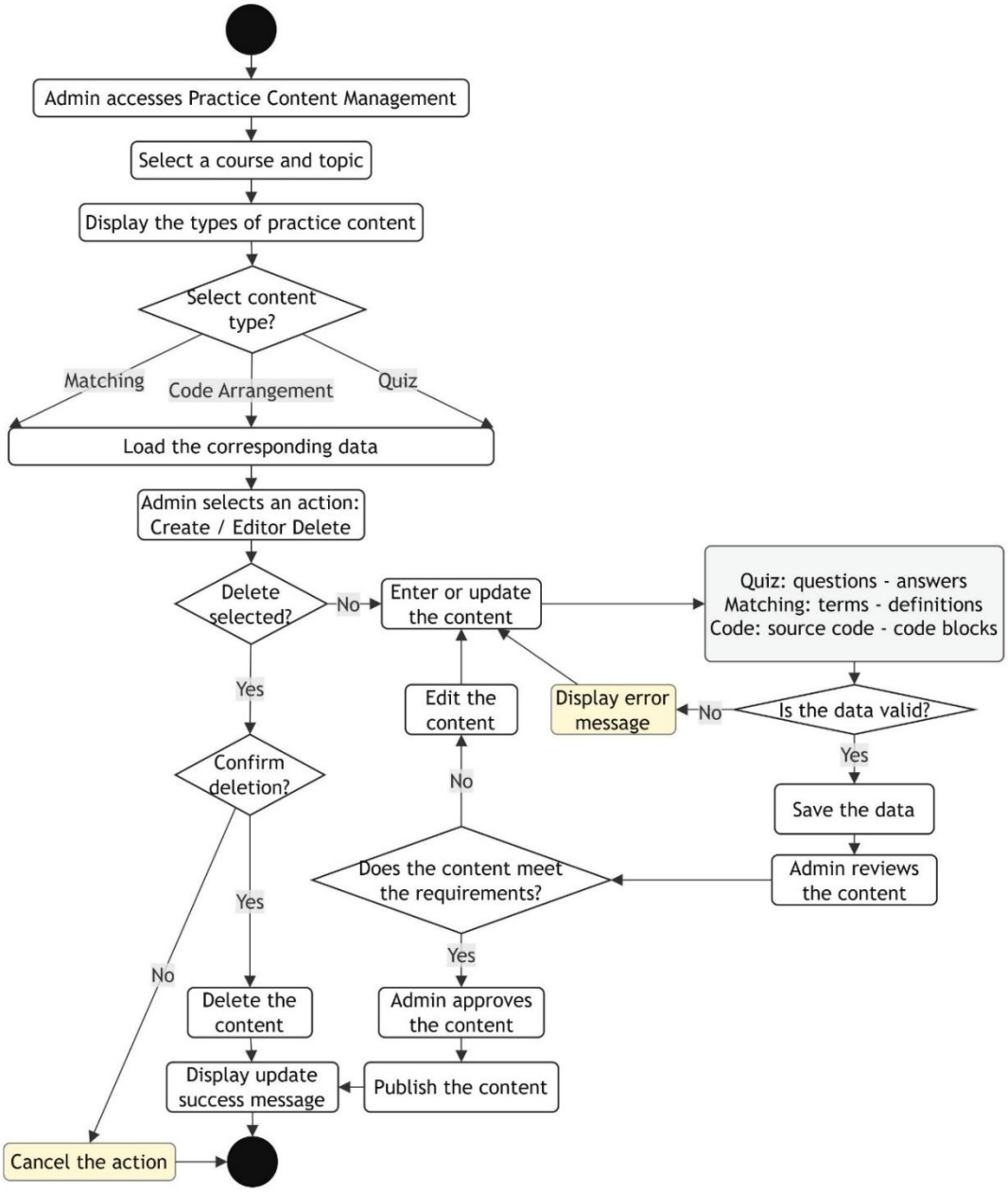

Figure 3.22 Diagram of the Practice Content Management Process

❖ AI Learning Content Generation Process

The Admin opens the content management function and chooses a topic for AI content generation. First, the system checks whether the topic has a Google Docs link. If there is no link, the Admin needs to select a document. If the link already exists, the content is taken from Google Docs and prepared before being sent to AI. The Admin then chooses the type of content to generate, such as Mindmap, Quiz Questions, or Matching Game content. The data is sent to the Gemini API for processing. If the process fails, the error is recorded and a message is shown to the

Admin. If it is successful, the content is displayed for review. The Admin can edit the content or ask Gemini to create it again if needed. When the content is accepted, it is saved in AI_Content_Cache, and the generation information is recorded in AI_Generation_Logs. Finally, the Admin can publish the generated content to the related learning or practice area, or save it for later editing.

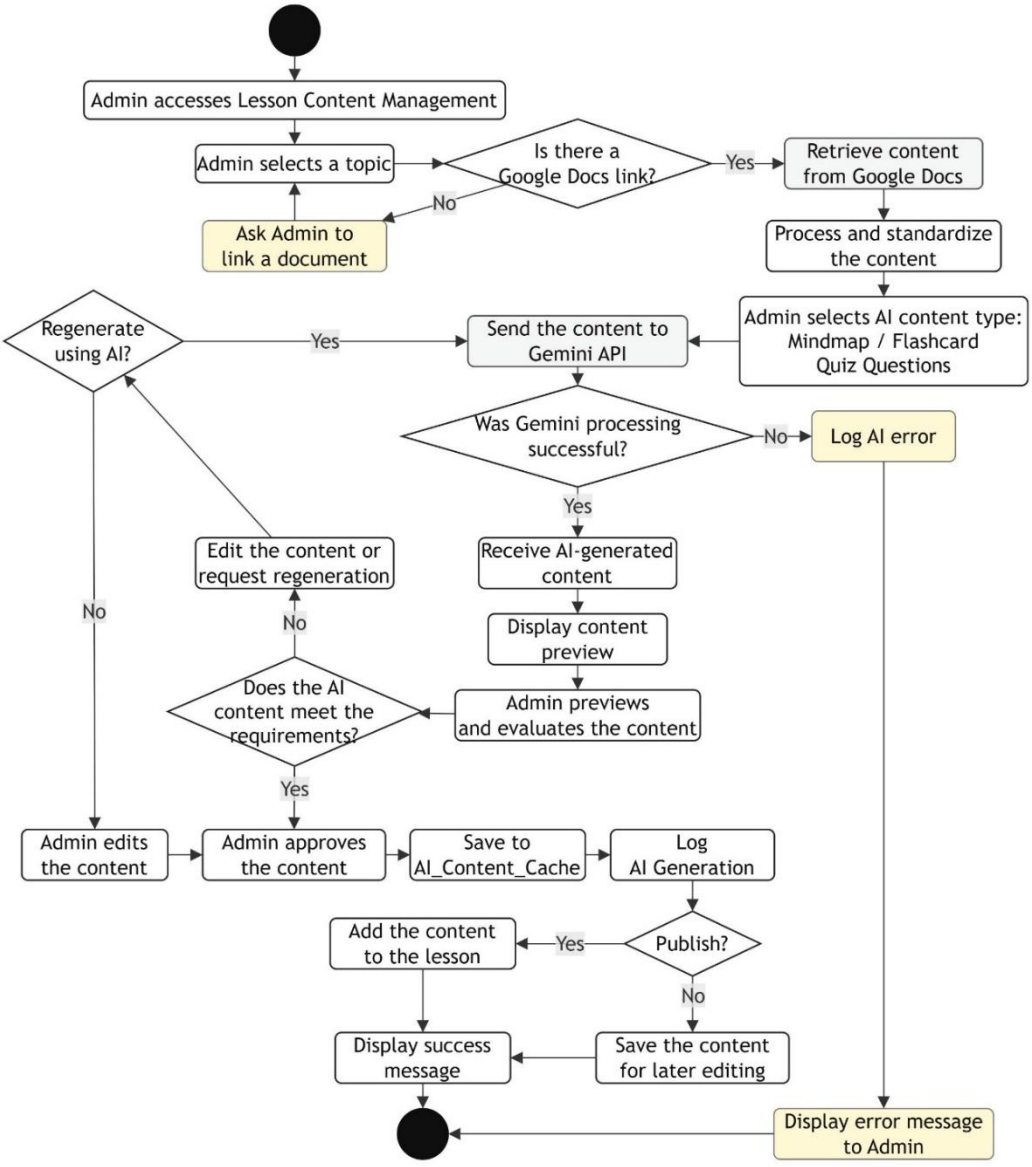

Figure 3.23 Diagram of the AI Learning Content Generation Process

❖ User Management Process

The Admin opens the account management page to view the user list and related information. The Admin can search or filter the list and then choose an account to see its details and current status. From this page, the Admin can view information, lock the account, or unlock it. For a lock or unlock action, the system asks for confirmation

before making the change. After the Admin confirms, the account status is updated, the data is saved, and the action history is recorded. The user list is then loaded again to show the latest result.

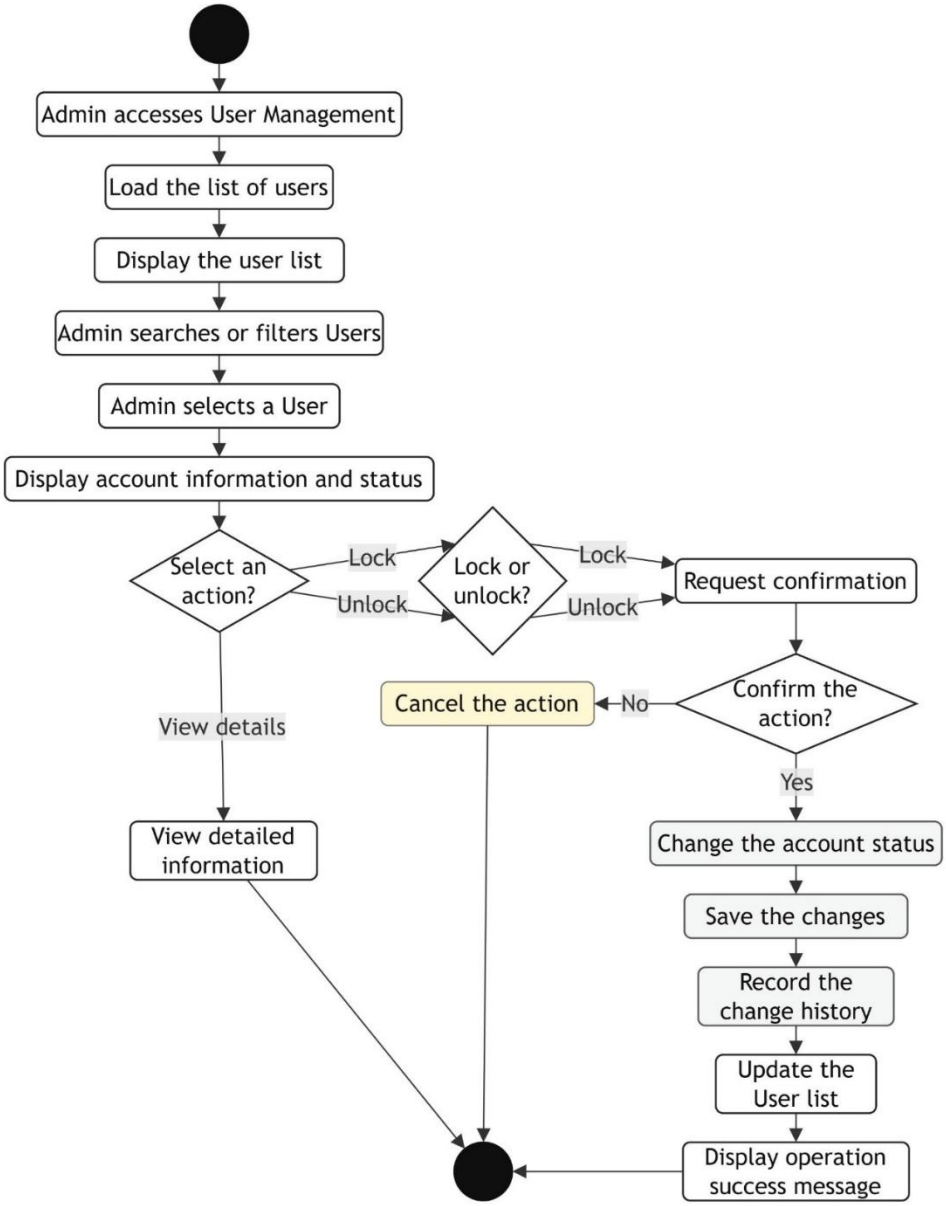

Figure 3.24 Diagram of the User Management Process

❖ **System Statistics and Monitoring Process**

The Admin opens the statistics function to check the activity of the system. User data and learning data are collected from related sources. If there is an error while loading the data, a message is shown to the Admin. When the data is loaded successfully, it is collected and used to calculate management statistics. User statistics include the total number of users, active users, and new or returning users. Course statistics

include completion rate, accuracy, average score, and performance by topic. The data is then prepared for the Dashboard and charts. The Admin chooses a statistics group, such as courses or topics, users, or account status. The related numbers and charts are displayed for monitoring and analysis.

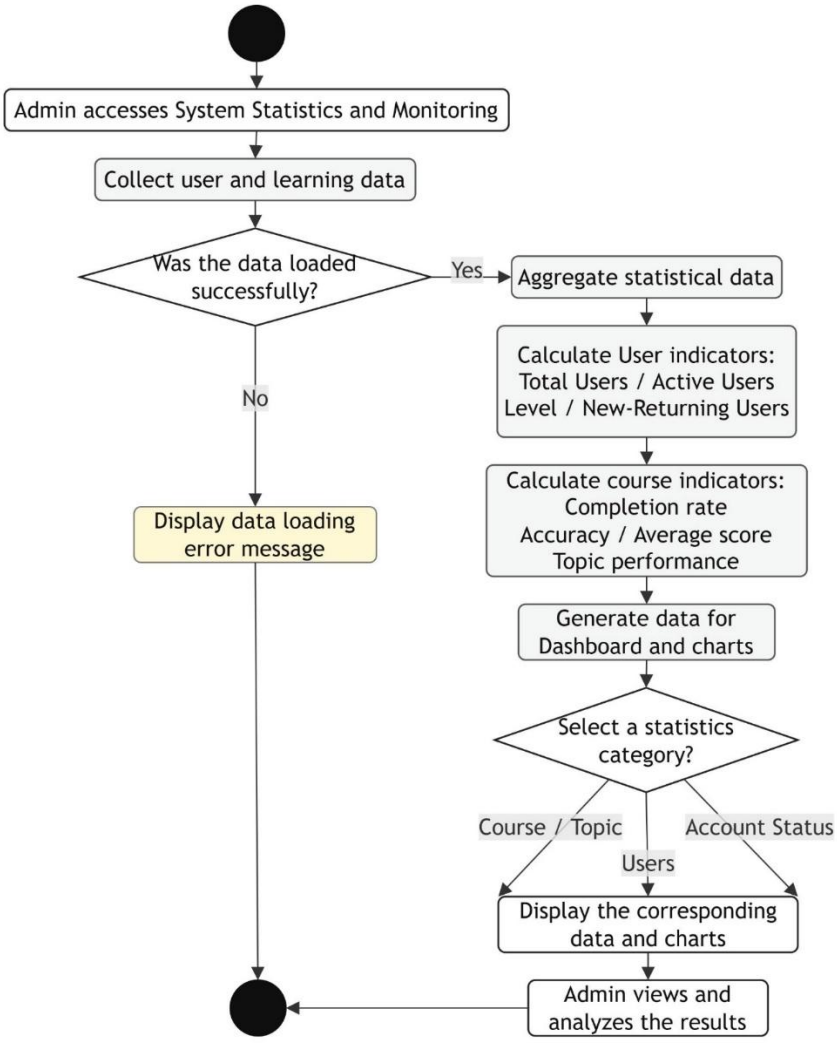

Figure 3.25 Diagram of the System Statistics and Monitoring Process

3.7. System Security Design

3.7.1. User Authentication

Access to the system can be gained through either email/password credentials or Google OAuth 2.0 authentication. For email/password login, the server undertakes a verification of the provided email address, its corresponding hashed password, the account's status, and email verification status prior to the establishment of a user session. Should the email remain unverified, a verification code, dispatched via email, will be required from the user. In the case of Google OAuth 2.0, the authentication

procedure is managed by Google, with the outcome subsequently transmitted to our system. Following this, either a new user account is provisioned or an existing one is updated. Crucially, the user's Google password is not accessed nor retained within our system's database.

3.7.2. Login Information Protection

User credentials are not retained in their plaintext form. Prior to storage, passwords undergo processing by the system utilizing SHA-256 encryption alongside a salt; the resultant hash is subsequently preserved within the passwordHash attribute. Upon user login, the submitted password is subjected to identical processing and then juxtaposed with the previously saved data. Email validation and the password reset procedure employ a six-digit code, which is associated with an expiry period. Specifically, for the password recovery mechanism, the generated code remains effective for a duration of one hour. Once a password reset has been completed without issue, a new passwordHash is recorded by the system, rendering the prior recovery code obsolete.

3.7.3. Session Management

Upon successful authentication, a unique session identifier is generated to signify the user's active session. The "Users" table is utilized to store the current session ID and its most recent update timestamp, thereby maintaining a record of the active session and its latest modification time. A single-session policy is enforced, ensuring that only one primary login session is permissible for each account concurrently. In instances where an account is already engaged on a separate device, user verification is mandated prior to initiating a login on the new device. Following such verification, the existing session is superseded by a newly generated session identifier. This procedure is instrumental in mitigating the simultaneous usage of a single account across multiple devices.

3.7.4. Authorization and Access Control

The system divides functions into Guest, User, and Admin groups, but the database stores only two role values: USER and ADMIN. Guest means a person who has not logged in, so it is not an account role in the database. USER accounts can access learning, practice, profile, and progress functions. ADMIN accounts can also access

management functions. Before loading data or performing an action in the Admin area, the server checks the userId and the account role. The isUserAdmin(userId) function is used to check Admin permission.

3.7.5. API Key and System Configuration Protection

Server-side settings such as database IDs, environment settings, and system API Keys are stored with PropertiesService in Google Apps Script instead of being placed directly in the client interface. Personal Gemini API Keys are managed in the AI_User_Keys sheet and linked to the owner's userId. The stored information includes keyHash, encryptedKey, key status, last validation time, last use time, and last error. In the current design, keyHash is created with SHA-256, while encryptedKey is stored with XOR and Base64. API calls are also recorded in AI_Key_Usage_Logs with information such as status, HTTP code, error message, and processing time.

3.7.6. Data Access Control and Protection

The data is divided into MASTER_DB and USER_DB instead of being stored in one Google Sheet. MASTER_DB stores shared data such as accounts, courses, topics, practice content, AI data, and statistics. Each user has a separate USER_DB for personal profile, learning progress, practice history, tasks, and review data. The progressSheetId field connects each account to its personal data file. Before reading or writing personal data, the server checks the userId and accesses the correct USER_DB. The client does not directly read from or write to the system's Google Sheets or Google Drive files.

3.7.7. Concurrent Access Control and System Logging

Google Sheets may have data conflicts when many requests write data at nearly the same time, especially when updating progress, XP, Quiz results, or Daily Quest data. LockService is used in data writing processes so that one request finishes before another request continues. This reduces race conditions and inconsistent data. The system also stores activity logs in the System_Logs sheet. Each record includes timestamp, level, category, userId, action, details, sessionId, and errorMessage. These logs are used to track errors, check User or Admin actions, and find the related session when a system problem occurs.

CHAPTER 4
SYSTEM IMPLEMENTATION, TESTING AND EVALUATION

4.1. Deployment Environment

4.1.1. Hardware

The development and testing of TerraCode were carried out on a personal computer with an Internet connection. The computer was used to write source code, manage project files, run Visual Studio Code, use CLASP commands, manage source code versions with Git, and access Google Apps Script.

Because the system works as a Web App, a Web browser was also used regularly during development. After each function was updated, the system was opened in the browser to test the interface, navigation flow, buttons, Server calls, and returned data. The development computer does not need a separate Web Server. The Server side runs on the Google Apps Script infrastructure. The personal computer is mainly used for source code development and Client-side interface testing. Consequently, fewer components need to be installed and set up on the development computer.

In addition to synchronizing source code with Google Apps Script, the Internet connection is used to access services including Google Sheets, Drive, Docs, OAuth, Gmail API, and Gemini API. A dependable Internet connection is necessary for continuous testing because the system's main components run online.

The configuration of the development computer is as follows:

- Processor: Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz (2.59 GHz)
- RAM: 16.0 GB (15.8 GB usable)
- Storage: 477 GB
- Operating System: Windows 11 64-bit
- Internet Connection: Internet connection

This hardware configuration is mainly used for writing code, running development tools, and testing the system in a Web browser. Server-side processing is performed on Google Apps Script, so the development computer does not need the hardware resources required to operate a separate application Server.

4.1.2. Software and Platform

TerraCode was mainly developed with JavaScript ES6+, HTML5, and CSS3. JavaScript is used to process functions on both the Client and Server. HTML5 is used to create the structure of the user interface. CSS3 is used to design the interface and support different screen sizes.

The Server runs on Google Apps Script V8 Runtime, and the system is deployed as a Web App.

The system also connects to several Google services, including Google Sheets, Google Drive, Google Docs, Google OAuth 2.0, and Gmail API.

- Google Sheets stores MASTER_DB and USER_DB.
- Google Drive manages personal data files of users.
- Google Docs stores lesson content.
- Google OAuth 2.0 supports login with a Google account.
- Gmail API sends verification emails, password reset emails, and study reminder emails.

The system also connects to the Gemini API to create and analyze learning content. Chart.js is used to display charts and learning statistics. D3.js and Markmap are used to display Mindmaps. html2canvas is used to export interface content as images. DiceBear API is used to create avatars for new users.

4.1.3. Google Apps Script Runtime

The Server side of TerraCode runs on the Google Apps Script V8 Runtime. The V8 Runtime allows the project to use modern JavaScript syntax and is suitable for the JavaScript ES6+ used in the system.

Google Apps Script has two main roles in the project. First, it provides the environment for running Server-side functions. Second, Google Apps Script allows the project to be deployed directly as a Web App. Therefore, TerraCode does not require Apache, Nginx, a Node.js Server, or another separate Web Server.

When users access the system, Google Apps Script returns the required HTML interface. Logging in, choosing a course, choosing a lesson, finishing a quiz, and updating a profile are all handled by client-side JavaScript. The Client sends a request to Google Apps Script functions when an activity needs data from the Server.

After receiving the request, the server reacts in line with the necessary function. When a user logs in using their email address and password, for example, the server finds the account and confirms the password, account status, email verification status, and login session. When a user begins a lesson, the server finds the topic information and loads the pertinent content.

When a User completes a Quiz, the Server checks the answers, saves the result, and updates learning data.

Google Apps Script also works as a connection between TerraCode and its data services. Server functions access Google Sheets to read or write data. When lesson content is needed, the Server works with Google Docs. When personal files need to be created or managed, the system accesses Google Drive. Email functions connect to Gmail, while AI functions send requests to the Gemini API.

Some processing tasks are performed on the Server instead of directly on the Client. Data related to accounts, access permissions, databases, and APIs is processed on the Server. This approach allows the Client to focus on the interface and reduces the amount of data-processing logic placed directly in the Web browser.

Google Apps Script also supports Time Trigger. The system uses this feature for time-based functions such as checking learning reminder schedules. At the required time, the trigger runs a Server function. The system then checks the User's settings and performs the required task.

With the V8 Runtime and built-in Google Apps Script services, the Server side of TerraCode works in the same environment as Google Sheets, Google Drive, and Google Docs. This is suitable for the way the system stores data and processes learning functions.

4.1.4. Development Tools

- Visual Studio Code was used as the main tool for writing and editing source code.
- CLASP was used to synchronize source code between the local computer and Google Apps Script. This allowed the project to be developed on the local computer instead of editing all source code directly in the Google Apps Script Editor.
- Git was used to manage source code versions and track code updates.

- PowerShell was used to support file processing and fix encoding problems during development.

4.1.5. Source Code Structure

The TerraCode source code is divided into different groups based on their functions. This structure separates the user interface, system functions, authentication, data processing, and support services.

- auth/: Contains functions related to user accounts and authentication. This group handles Email login, Google OAuth, password reset, email verification, and user profile functions. Some files include login.js, googleAuth.js, passwordReset.js, and profile.js.
- database/: Contains source code for database processing. The masterDb.js file manages MASTER_DB and shared data sheets. The userDb.js file manages USER_DB and personal learning data for each user.
- server/: Contains the main system functions. This group processes courses, topics, lesson content, Quiz, Matching, Code Arrangement, statistics, and Admin functions. Some files include courseManager.js, topics.js, content.js, adminQuestion.js, adminMatching.js, and adminStats.js.
- services/: Contains shared services of the system. These services include Gemini API connection, AI prompts, study reminders, learning activity tracking, and Admin permission checking. Some files include geminiService.js, geminiPrompts.js, learningTracker.js, timelineService.js, and adminFunctions.js.
- views/: Contains the Client interface. The subfolders are divided by pages such as Landing, Login, Register, Dashboard, Topics, Lesson, MCQ, Matching, CodeArrangement, CodePlayground, Pet, and Profile. The views/admin/ folder contains pages for Admin users.
- views/shared/: Contains common interface components such as header, footer, notifications, dialogs, loading overlay, and theme settings. The styles.html file manages common interface styles. The client_js.html file manages Client functions such as page navigation, login sessions, and calls to Server functions.

4.2. Implementation of Interface and Functions for Guest

4.2.1. Sitemap for Guest

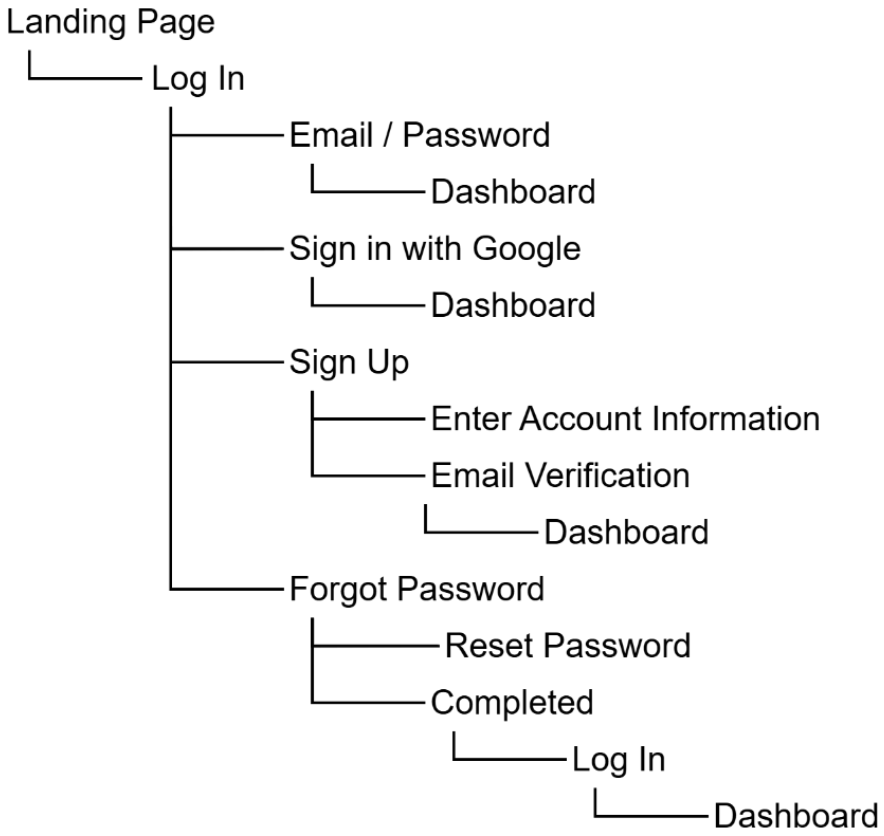

Figure 4.1 Sitemap for Guest

4.2.2. Guest Access and Authentication Interfaces

The Landing Page is the first interface shown when users access the system. This page gives a short introduction to TerraCode and provides options to log in or create an account. Guests who have not logged in can only access public functions and account-related functions. They cannot access learning content for Users.

The Login interface allows users to access the system by using Email and password or through Google OAuth 2.0. When users submit their information, the system checks the account, password, account status, and email verification status. If the information is valid, the system creates a login session and moves the user to the Dashboard. With Google OAuth 2.0, authentication is completed through the user's Google account before TerraCode creates or updates the related account information.

The Sign Up interface is for Guests who do not have an account. Users enter their Email, password, password confirmation, and display name. The system checks the entered data and checks whether the Email already exists before creating a new

account. After successful registration, the account is set as not yet verified, and the system sends a six-digit verification code to the registered Email.

The Email Verification interface allows users to enter the code received by Email. When the code is valid, the system verifies the Email address, activates the account, and creates a separate USER_DB for the User. After that, the user can use the account to log in to the system.

The Forgot Password function supports users who do not remember their login information. Users enter their registered Email to request a password reset. The system checks the account and sends a verification code to the Email if the information is valid. After confirming the code, the user is moved to the Reset Password interface to enter a new password.

Messages such as invalid data, an Email that already exists, an incorrect password, an incorrect verification code, or an expired code are displayed directly in the related interface. These states belong to the same authentication function group, so they do not need to be presented as separate interfaces in the main content.

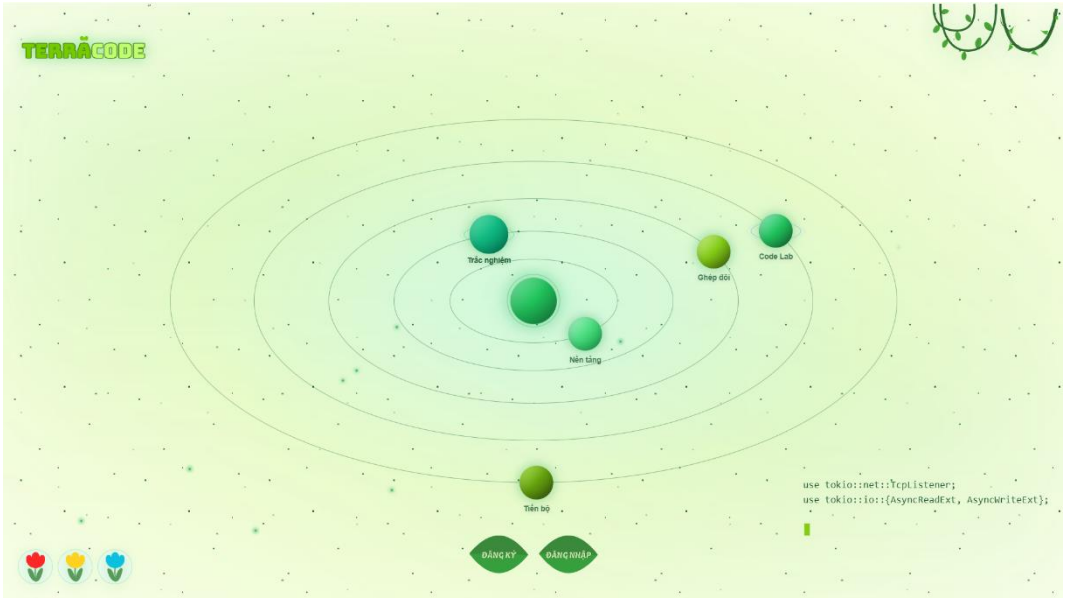

Figure 4.2 TerraCode Landing Page Interface

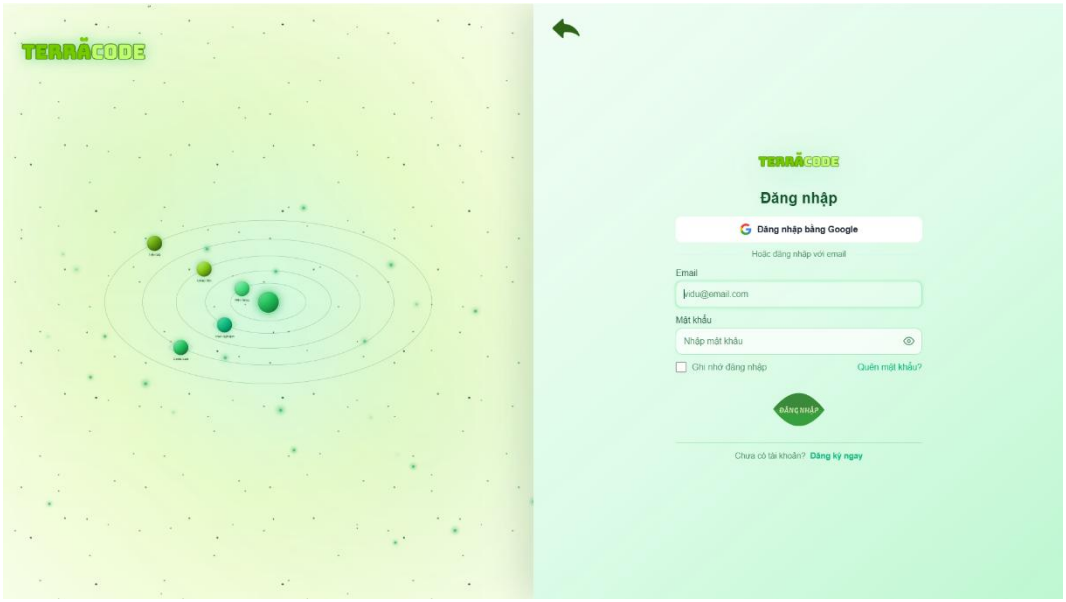

Figure 4.3 Login Interface

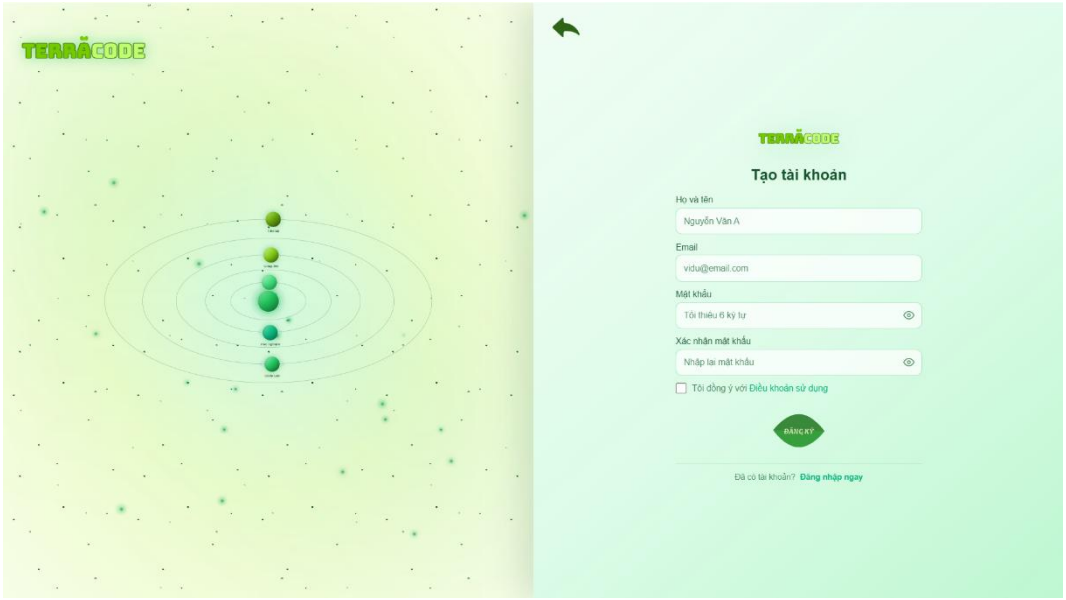

Figure 4.4 Sign Up Interface

4.3. Implementation of Interface and Functions for User

4.3.1. Sitemap for User

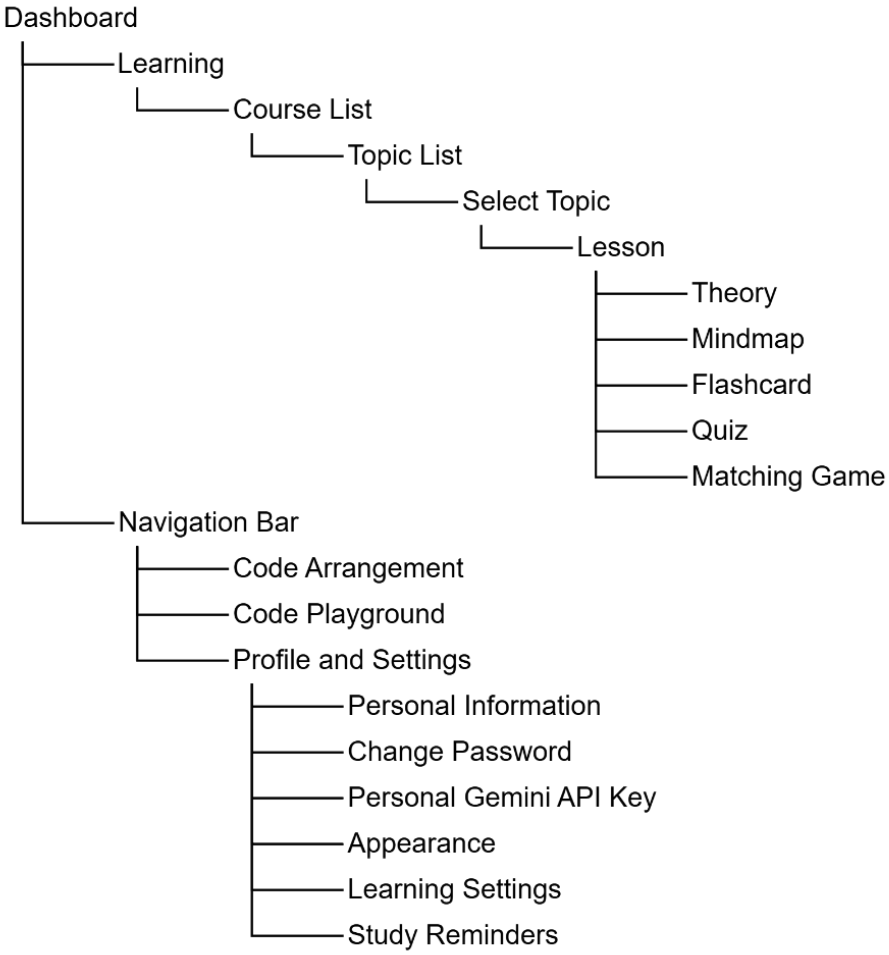

Figure 4.5 Sitemap for User

4.3.2. Learning Navigation and Overview Interfaces

This group of interfaces helps Users track their learning status and choose the content they want to study. The Dashboard shows general information such as the study schedule, activity history, XP, XQP, Daily Quest, and leaderboard. Daily Quest records daily tasks such as checking in, learning a new lesson, completing a Quiz, or completing a Matching Game. The system updates the task status when the User completes each task.

From the Dashboard, the User can access the Course List. Courses are displayed as cards with a cover image, course name, short description, and related information. The User can search, filter, or mark courses as favorites before choosing the content to study.

After the User selects a course, the system opens the Course Details page and shows the Topic List on the same screen. Each topic includes a name, description,

knowledge group, learning status, and related action button. A topic that has not been started allows the User to start learning. A topic in progress allows the User to continue learning. A completed topic allows the User to review it. If the prerequisite has not been completed, the topic is shown as locked.

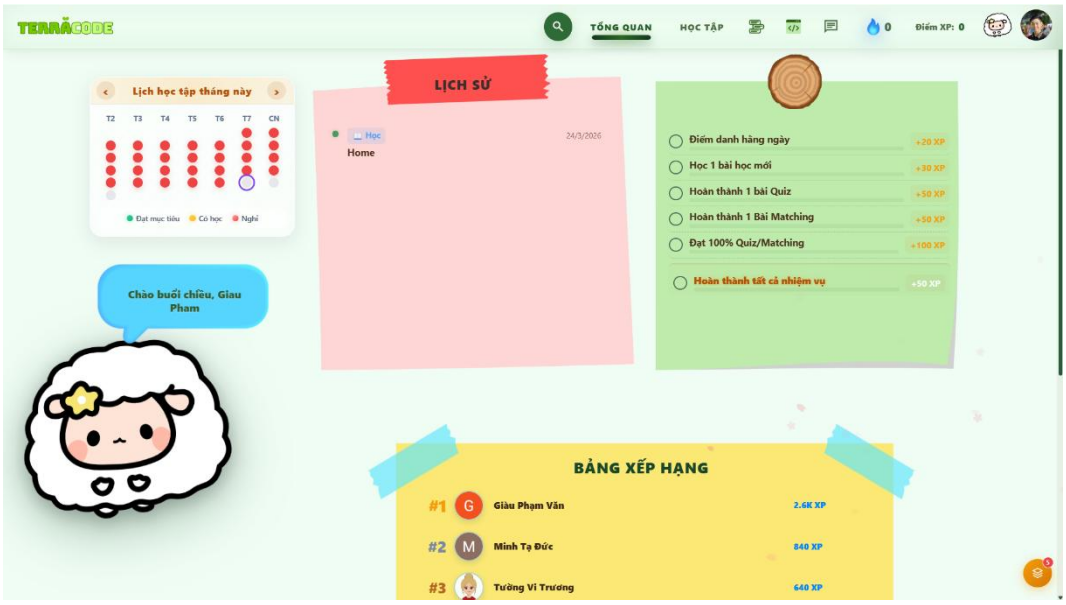

Figure 4.6 User Dashboard Interface

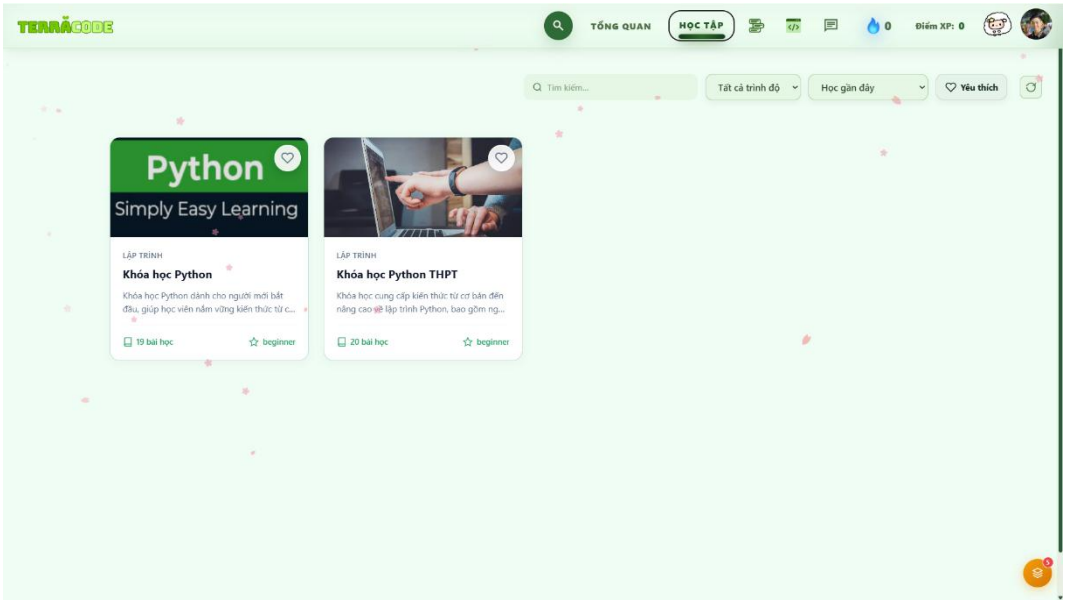

Figure 4.7 Course List Interface

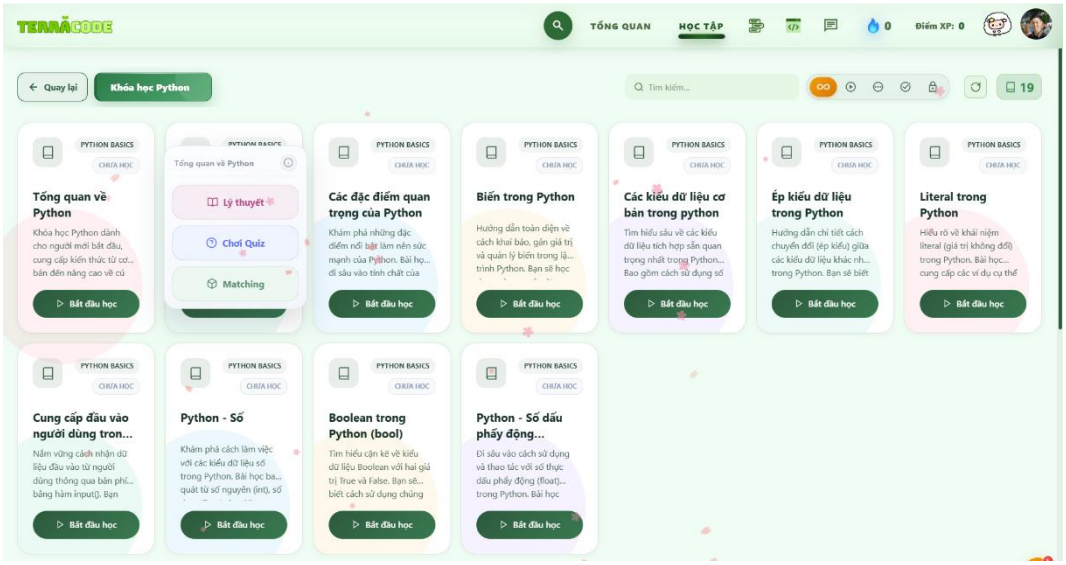

Figure 4.8 Course Details and Topic List Interface

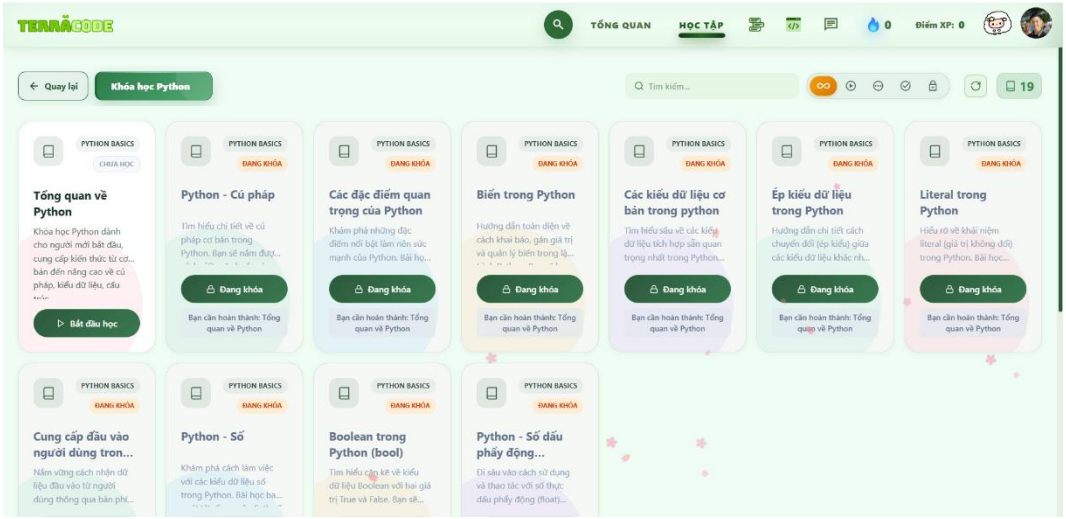

Figure 4.9 Topic Status When the Prerequisite Is Not Completed

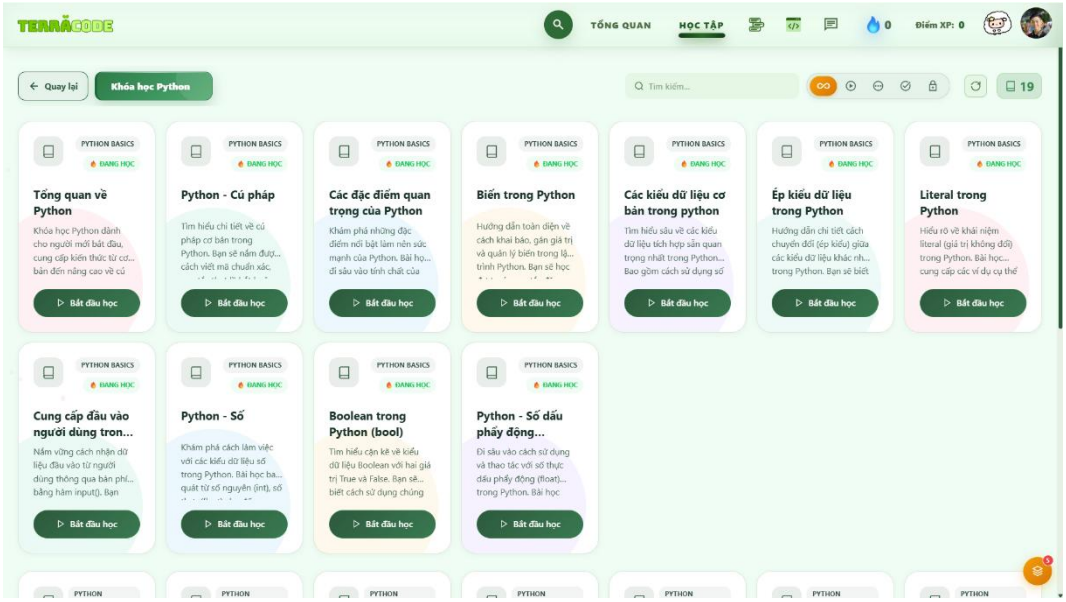

Figure 4.10 Interface for Continuing an Ongoing Course

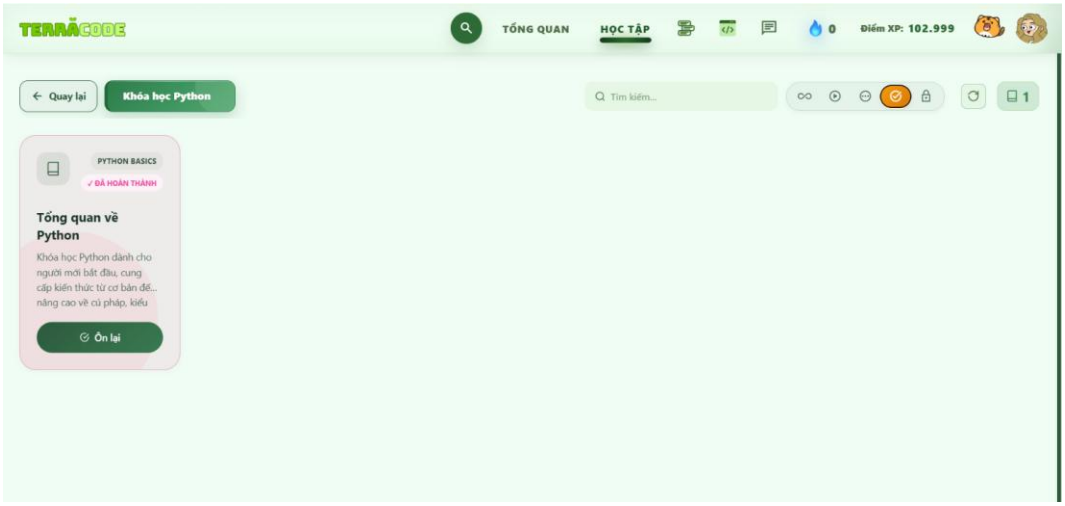

Figure 4.11 Completed Topic Status

4.3.3. Lesson and Learning Resource Interfaces

After selecting a topic, the User accesses the Lesson page. Learning resources, including Theory, Mindmap, and Flashcard, are placed in the same area so that the User can switch between them during the learning process.

The Theory section presents the main lesson content and provides navigation to other learning resources. The Mindmap presents knowledge in a branching diagram and helps the User understand the relationships between different parts of the lesson. Flashcard provides knowledge cards that allow the User to flip cards, move between cards, and mark content as learned or not learned. The learning status of these resources is recorded to support topic progress tracking.

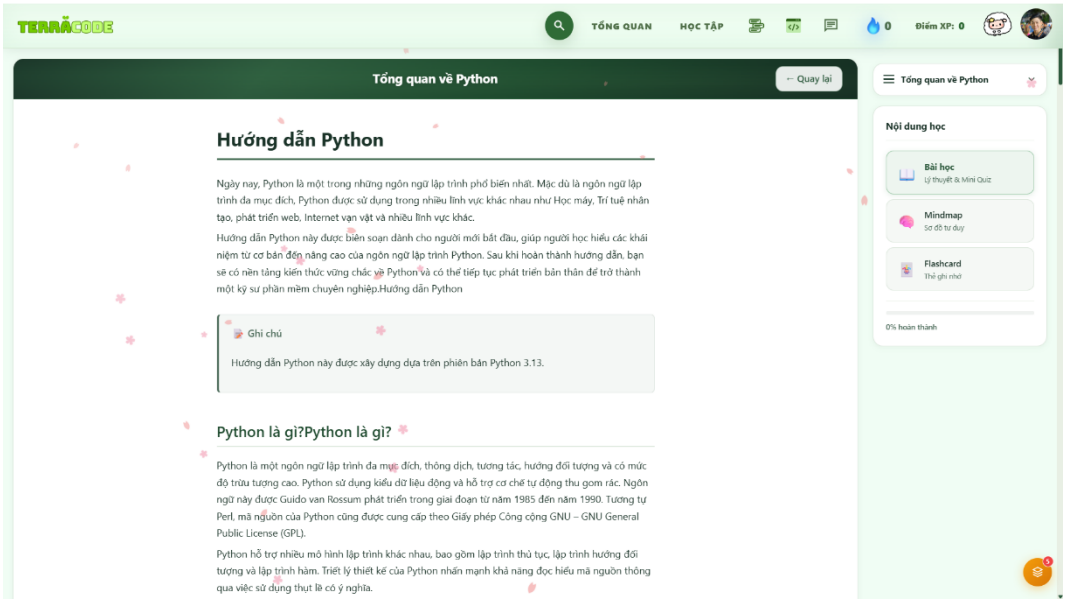

Figure 4.12 Lesson Content Interface

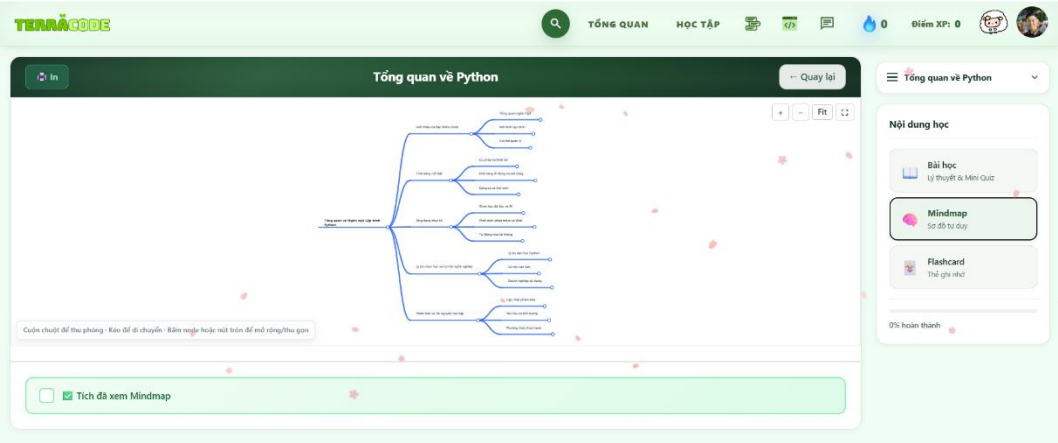

Figure 4.13 Lesson Mindmap Interface

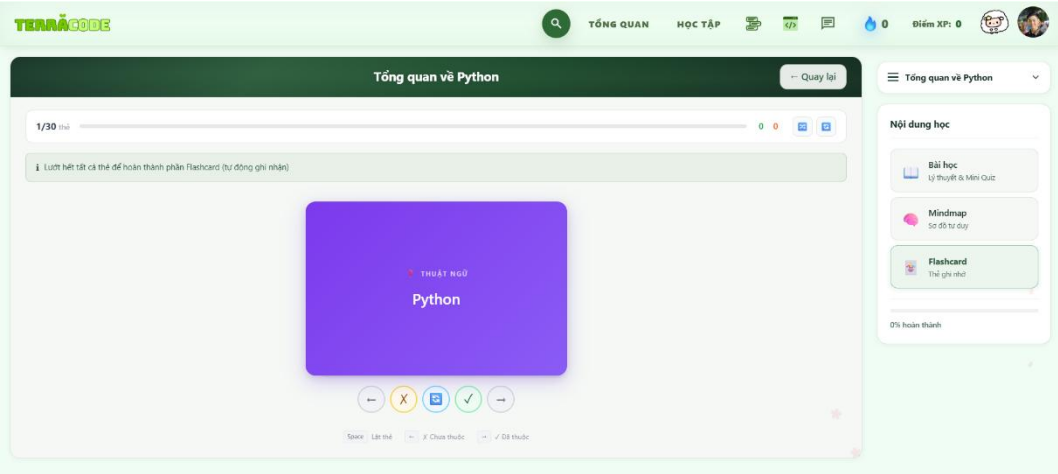

Figure 4.14 Flashcard Interface

4.3.4. Practice and Programming Interfaces

This group of interfaces includes Quiz, Matching Game, Code Arrangement, and Code Playground. These functions are used to check knowledge and practice programming skills.

The Quiz presents questions and answer options. It also shows the number of completed questions, score, and working time. After the User selects an answer, the system checks it and shows the result before moving to the next question.

The Matching Game requires the User to match terms with the correct definitions. The interface tracks the time, number of attempts, number of matched pairs, and current score.

Code Arrangement focuses on arranging code blocks in the correct order to create a complete program. After the User submits the result, the system checks the order and records the practice attempt.

Code Playground provides an environment for writing and running code directly in the web browser. The User enters code in the editor, performs actions such as Run, Stop, Save, or Reset, and views the result in Preview, Console, or Test Cases depending on the selected mode.

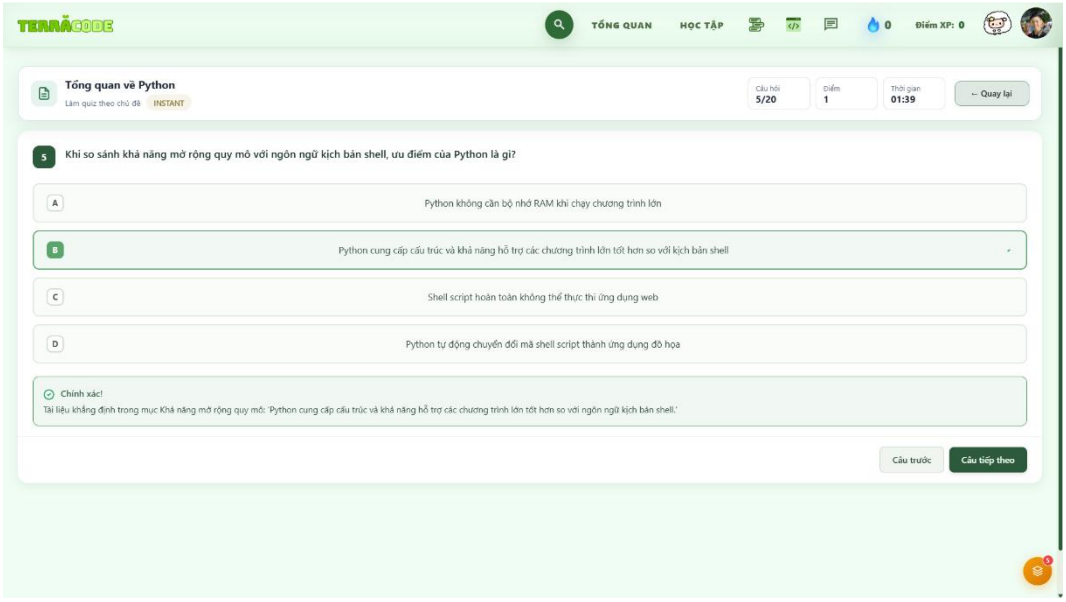

Figure 4.15 Quiz Interface

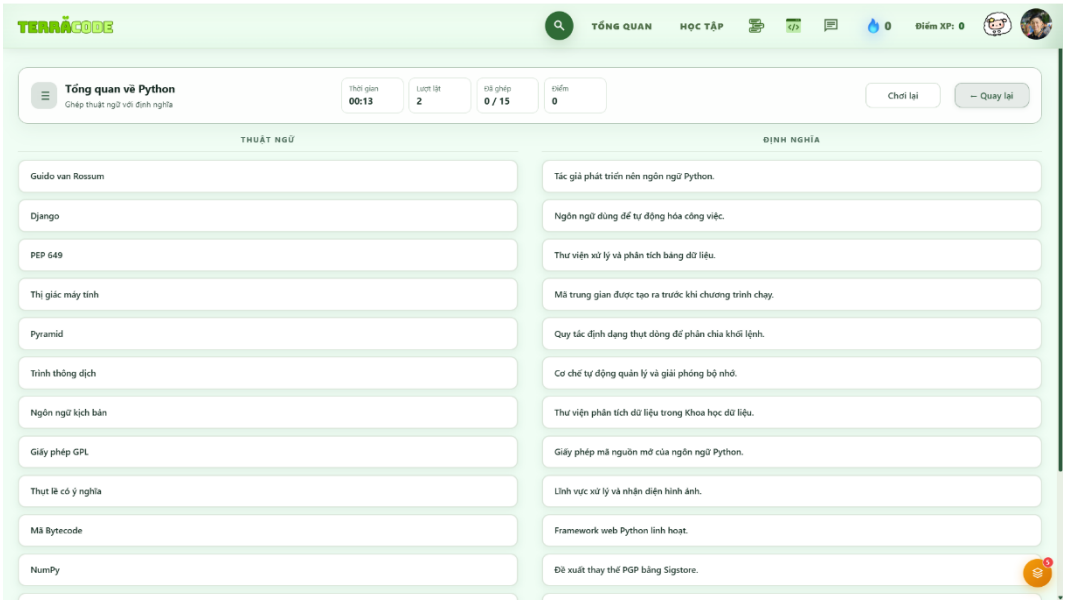

Figure 4.16 Matching Game Interface

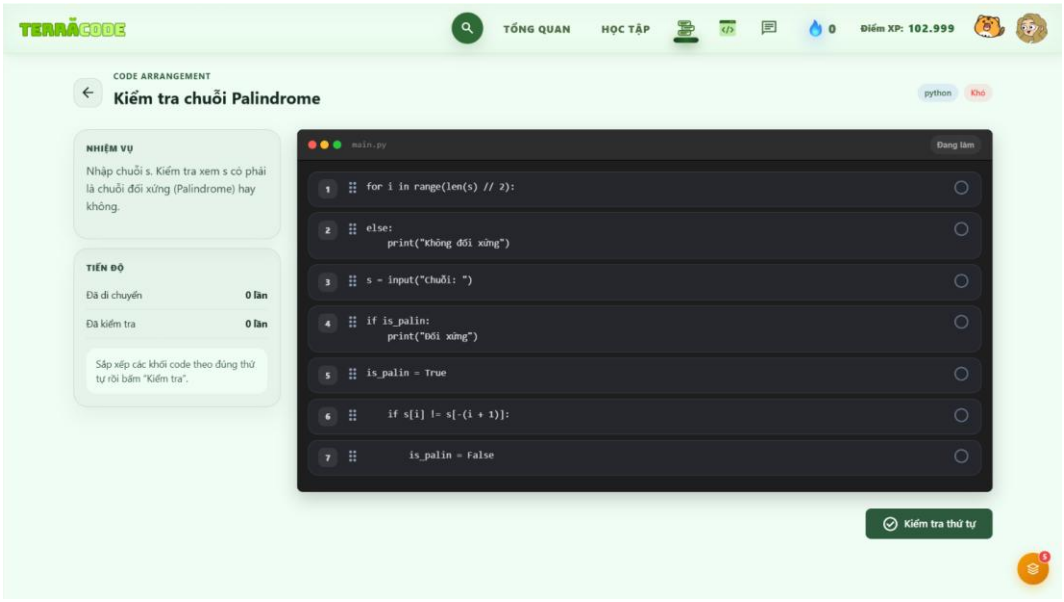

Figure 4.17 Code Arrangement Interface

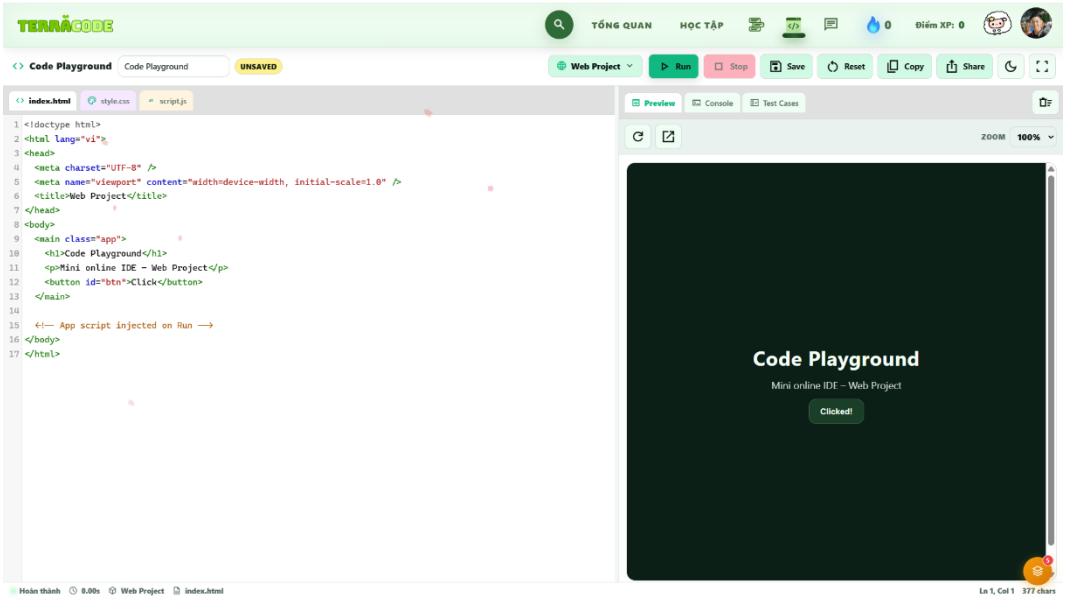

Figure 4.18 Code Playground Interface

4.3.5. Personal Settings Interfaces

The Profile page contains account and learning settings. The User can view and edit the display name and profile picture, change the password, and manage a personal Gemini API Key. The learning settings allow the User to set a daily goal and configure study reminders by email.

For study reminders, the User can select the reminder time, reminder days, the time before the study session, and other related options. The Reminder Log stores the history of reminder processing so that the User can check the reminders recorded by the system.

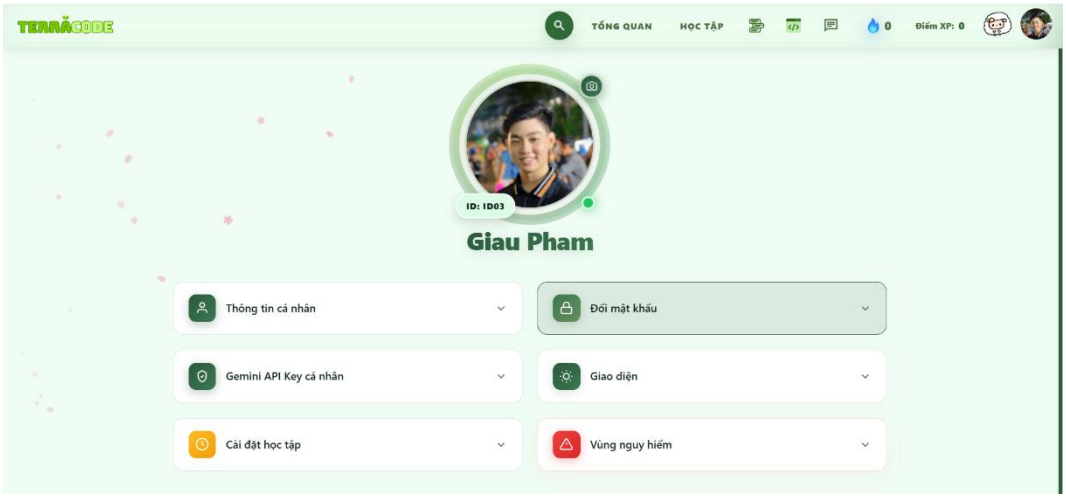

Figure 4.19 Profile and Learning Settings Interface

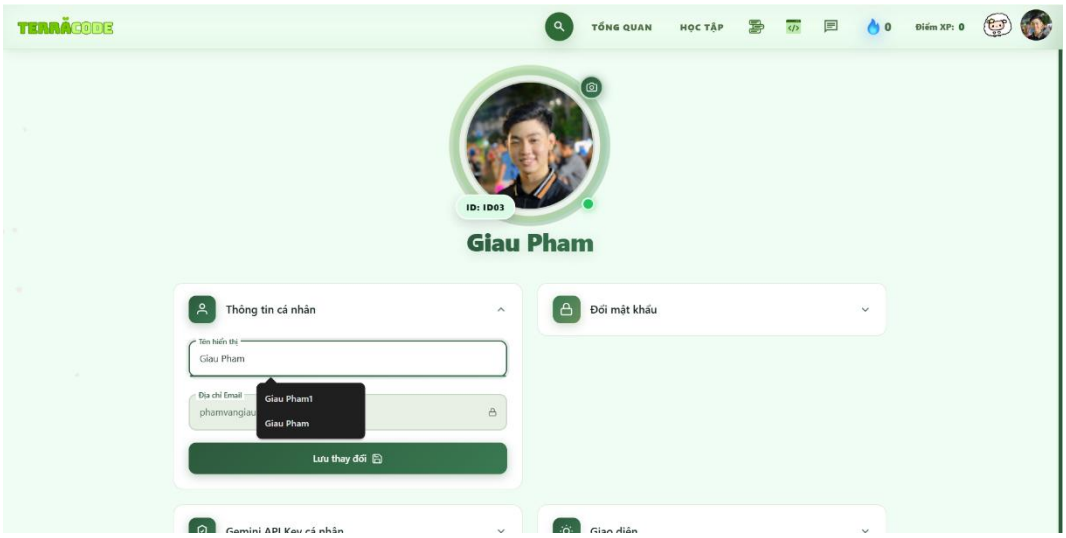

Figure 4.20 Edit Profile Interface

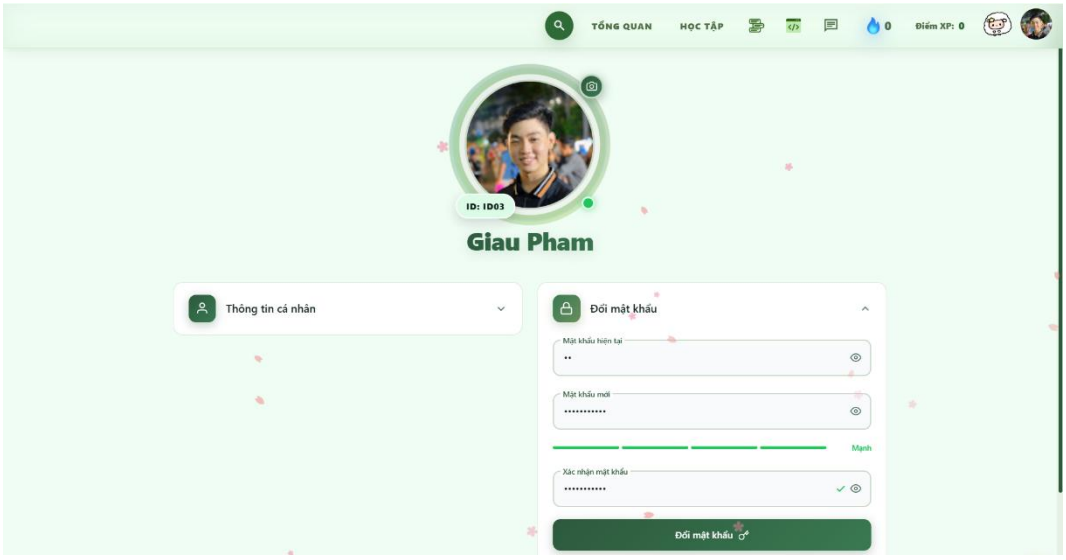

Figure 4.21 Change Password Interface

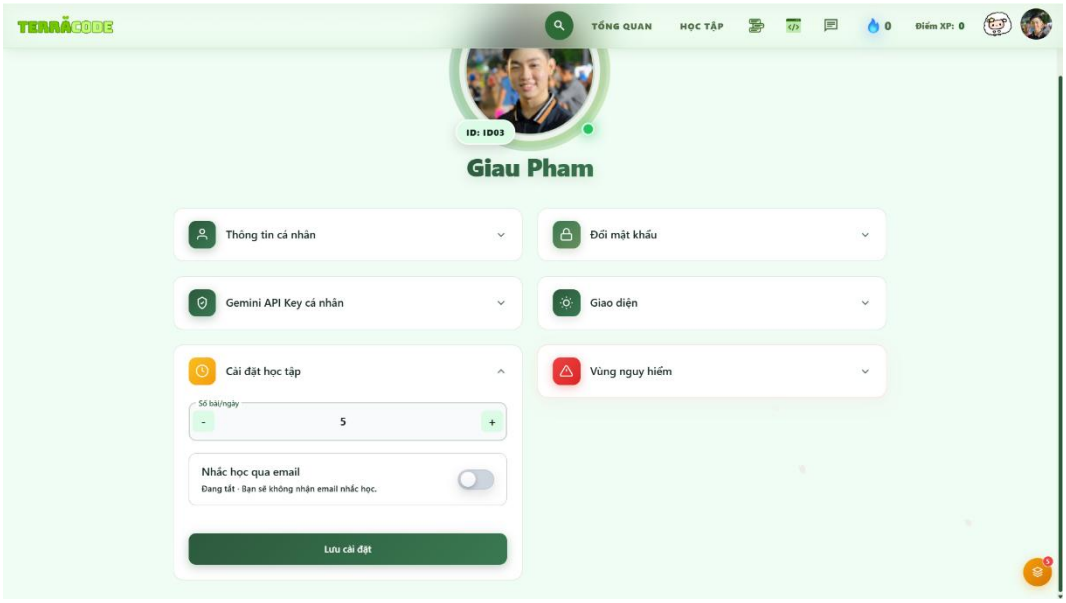

Figure 4.22 Daily Goal Settings Interface

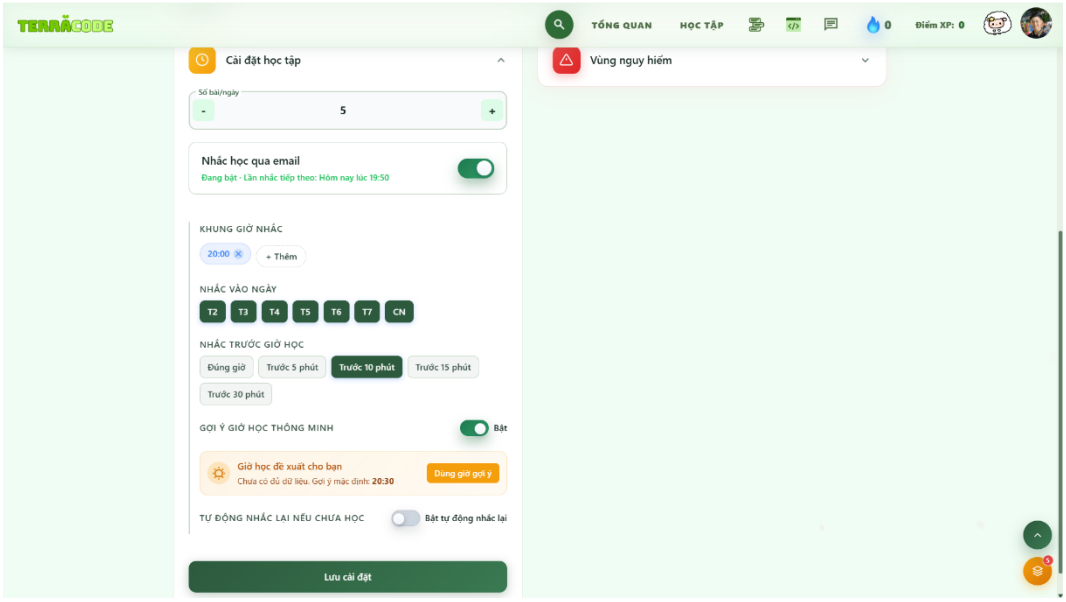

Figure 4.23 Study Reminder Settings Interface

4.4. Implementation of Interface and Functions for Admin

4.4.1. Sitemap for Admin

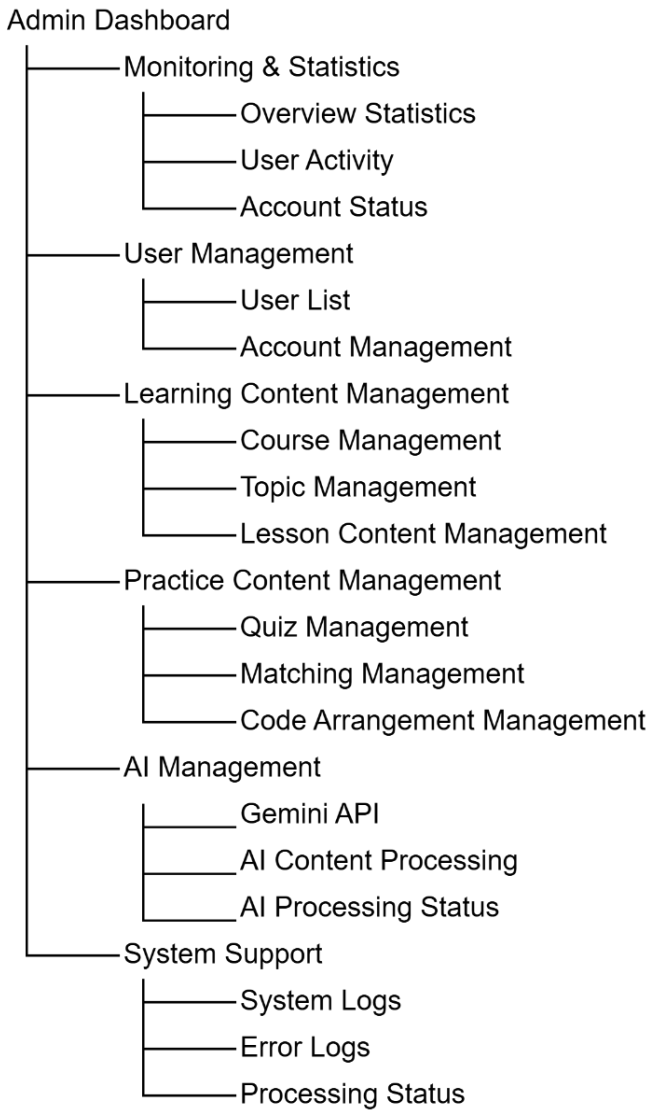

Figure 4.24 Sitemap for Admin

4.4.2. Monitoring and Statistics Interfaces

The Monitoring and Statistics area helps Admin monitor the general activities of the system through a group of management interfaces. The Dashboard shows important information such as the number of Users, active accounts, courses, learning activities, and recent activities.

From this area, Admin can also view and manage User account status. The account list provides basic User information and the current account status. Admin can check the information and take action when necessary.

The statistics section summarizes data about Users and their learning process. Charts and indicators show the number of new Users, active Users, completion rate, accuracy, and learning results by course or topic. This information helps Admin

understand how the system is being used and monitor the learning progress of Users.

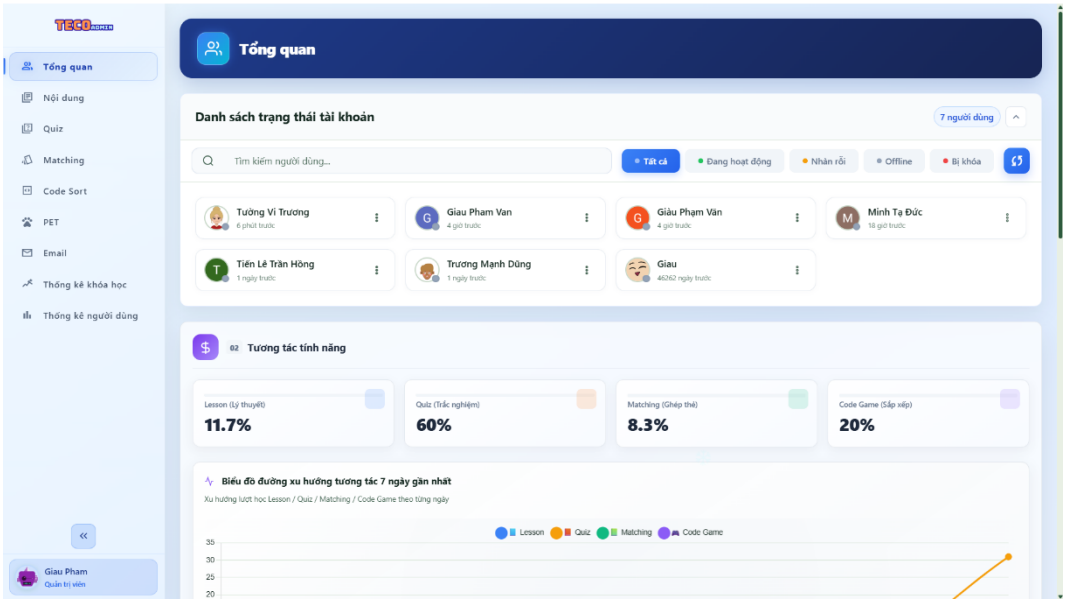

Figure 4.25 Admin Dashboard Interface

4.4.3. Course and Learning Content Management Interfaces

This area is used to manage the course structure and learning content. Admin can create, edit, delete, arrange, or change the display status of courses. Each course contains several topics, and these topics are managed under the related course.

The name, description, order, and other details of each topic can be changed by the administrator. When a topic requires the User to complete another topic first, this condition is defined as a prerequisite. Based on this configuration, topics can be locked or unlocked in the User interface.

Lesson content is managed for each topic. Admin can connect a Google Docs source, check the processed content, edit it when necessary, and preview it before publishing. Only approved content is provided to Users on the lesson page.

The content processing flow is organized as follows:

Connect Content Source → Process Content → Check Content → Preview → Approve → Publish

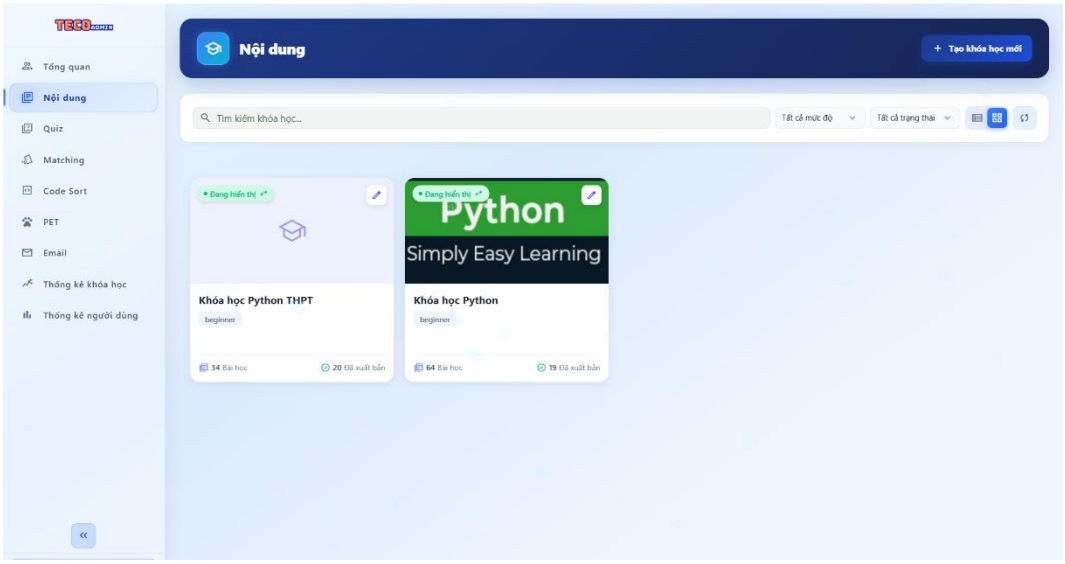

Figure 4.26 Course Management Interface

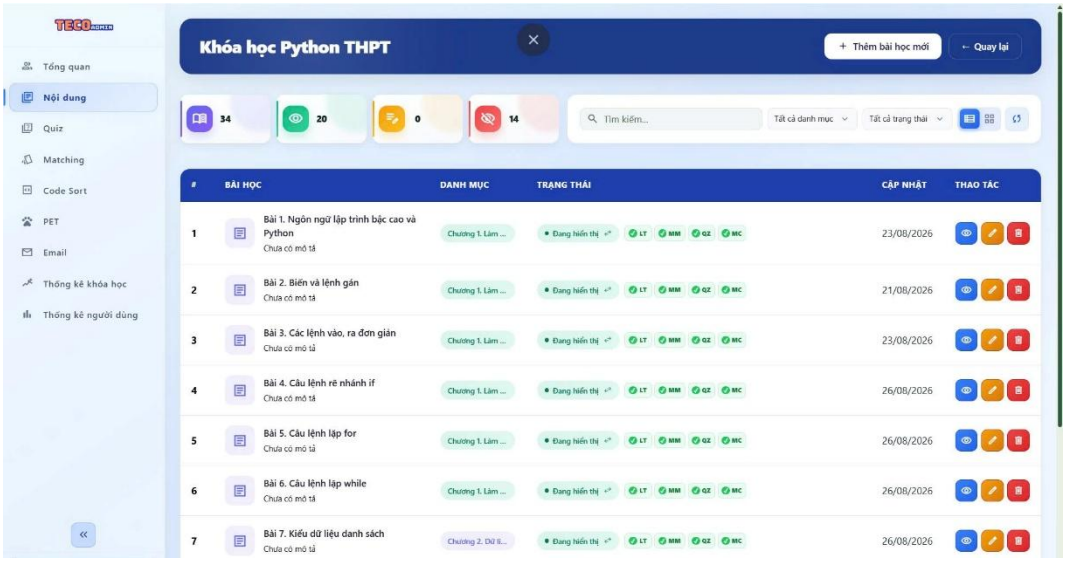

Figure 4.27 Topic Management Interface

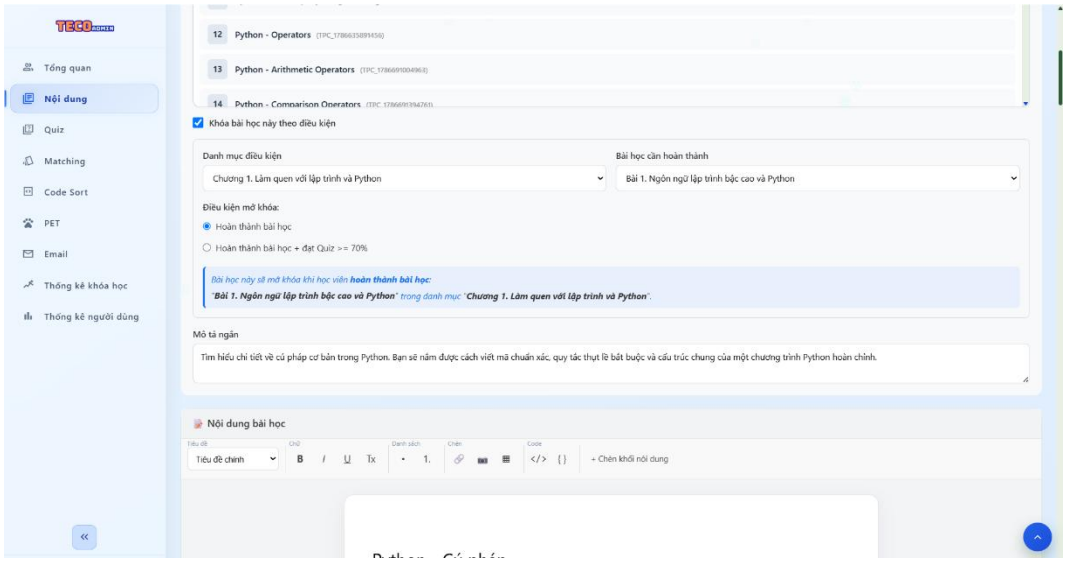

Figure 4.28 Topic Prerequisite Configuration Interface

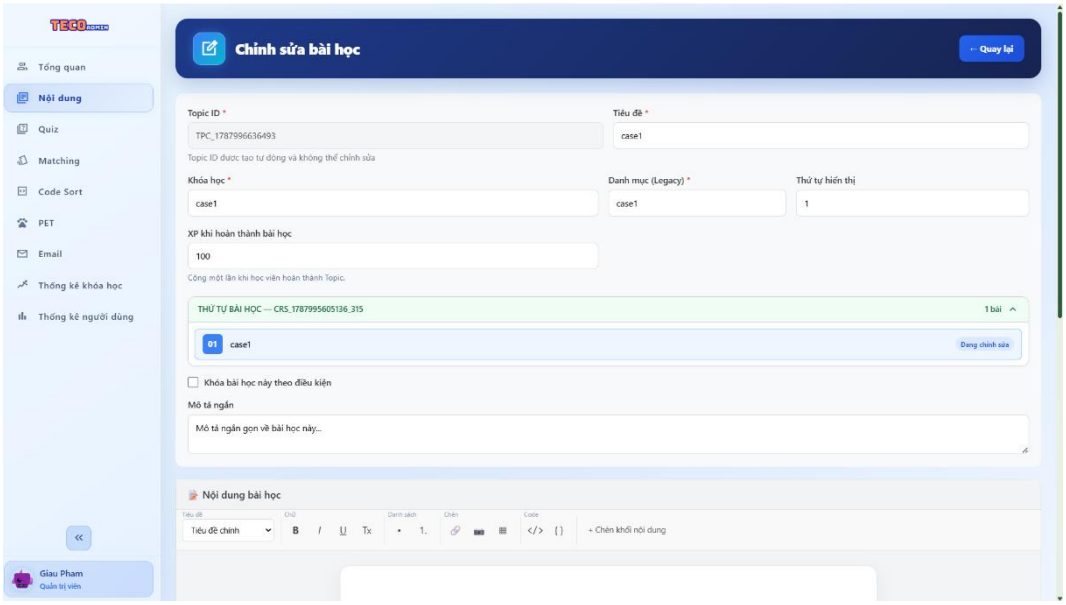

Figure 4.29 Edit Lesson Content Interface

Figure 4.30 Lesson Content Preview Interface Before Publishing

4.4.4. Practice Content Management Interfaces

Data for the Quiz, Matching Game, and Code Arrangement are managed under the Practice Content Management section. Although each practice type has a unique data structure, they are all related to the same course or topic.

Admin oversees a question bank for Quiz that contains the question's content, possible responses, right answers, and other pertinent data. Before questions are utilized in practice exercises, the administrator has the ability to create, modify, remove, import, export, and approve them.

Matching Game manages pairs of terms and definitions. Admin can create or edit matching content for each topic and check the data before it is provided to Users.

Code Arrangement stores exercises in which Users arrange code blocks in the correct order. Each exercise includes the task description, code blocks, and the correct order. Admin manages the content and status of each exercise before it is provided in the practice area.

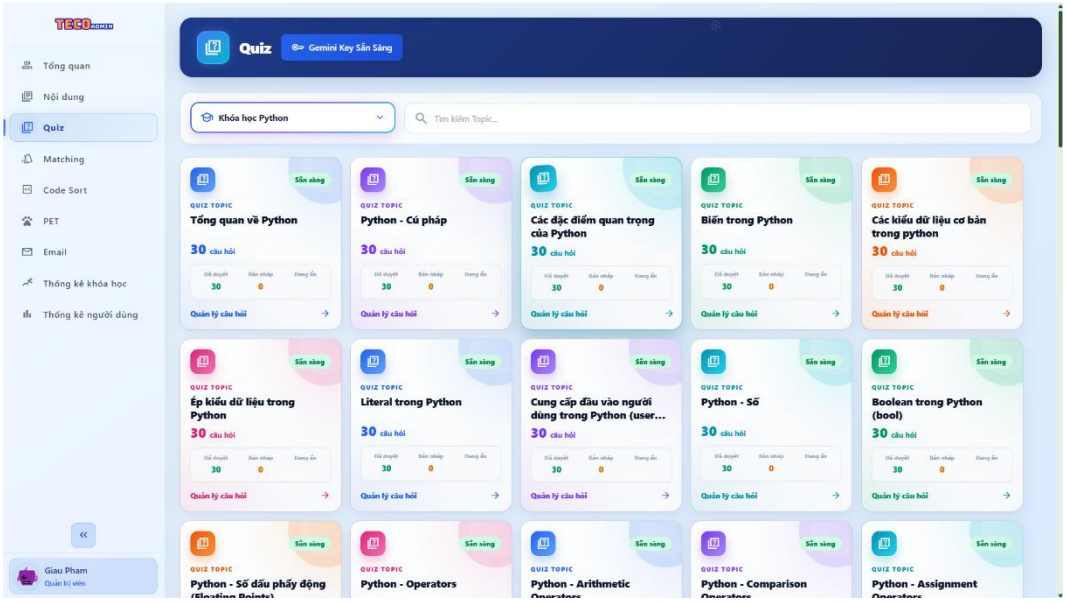

Figure 4.31 Quiz Question Bank Management Interface

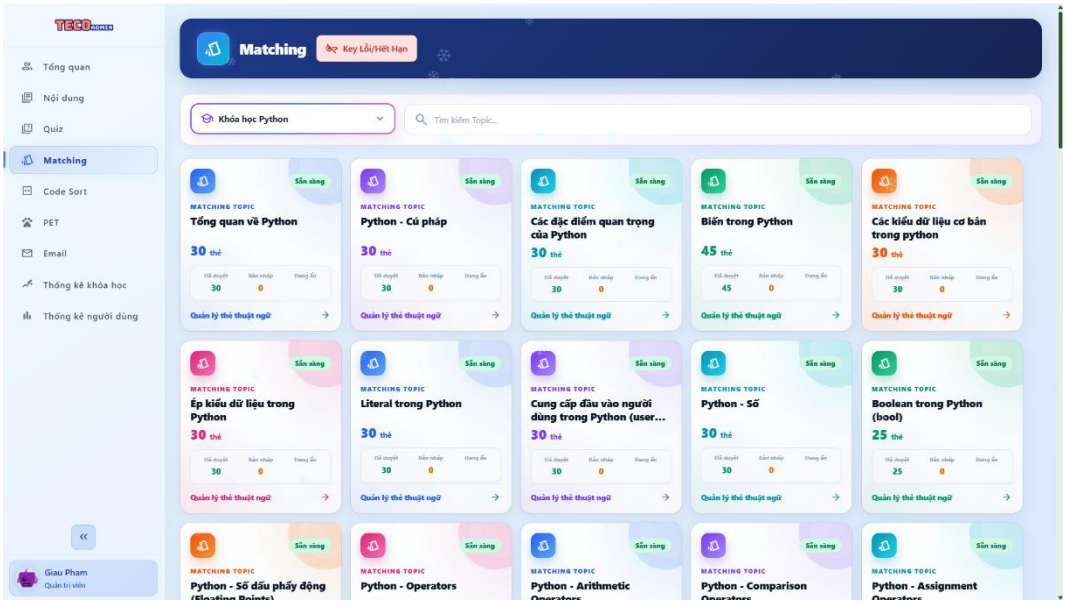

Figure 4.32 Matching Game Management Interface

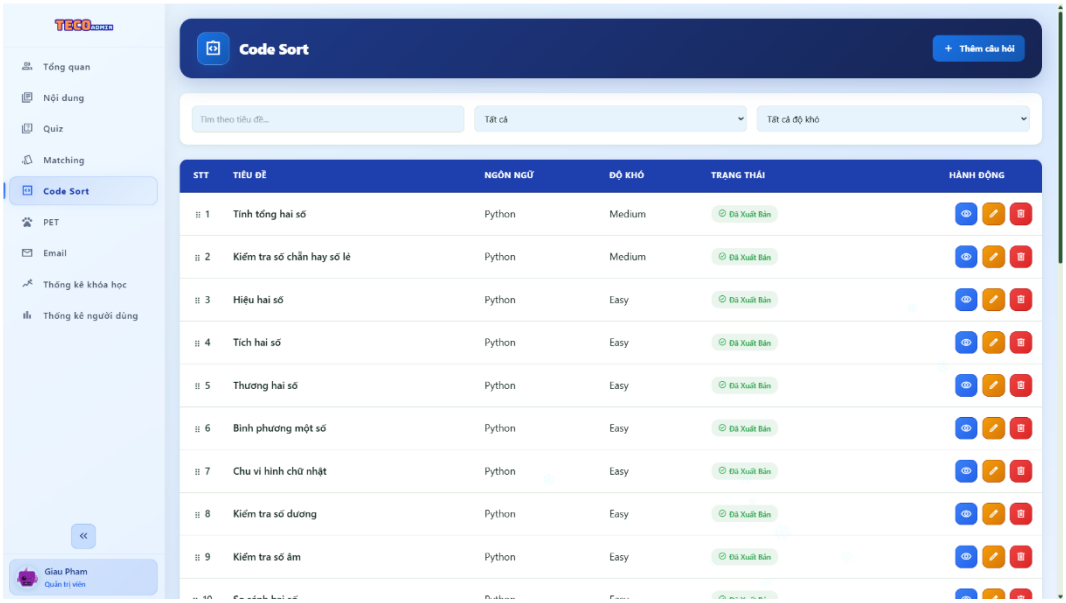

Figure 4.33 Code Arrangement Management Interface

4.5. Implementation of AI Functions

4.5.1. Gemini API Key Management

The Gemini API Key is used to connect TerraCode with the Gemini API when the system performs AI functions. The system provides an interface for users to enter, save, update, and test the API Key before sending content processing requests.

The system sends a test request to the Gemini API after verifying the value entered by the user. The test status is updated to indicate that the API Key is operational if the connection is legitimate. The system displays a relevant warning so the user can review the setup if the API Key is invalid, does not have access permission, or an issue occurs during the call.

Testing the API Key before using AI functions helps reduce errors caused by an invalid configuration. After the API Key is set successfully, it is used for the related AI content generation functions in the system.

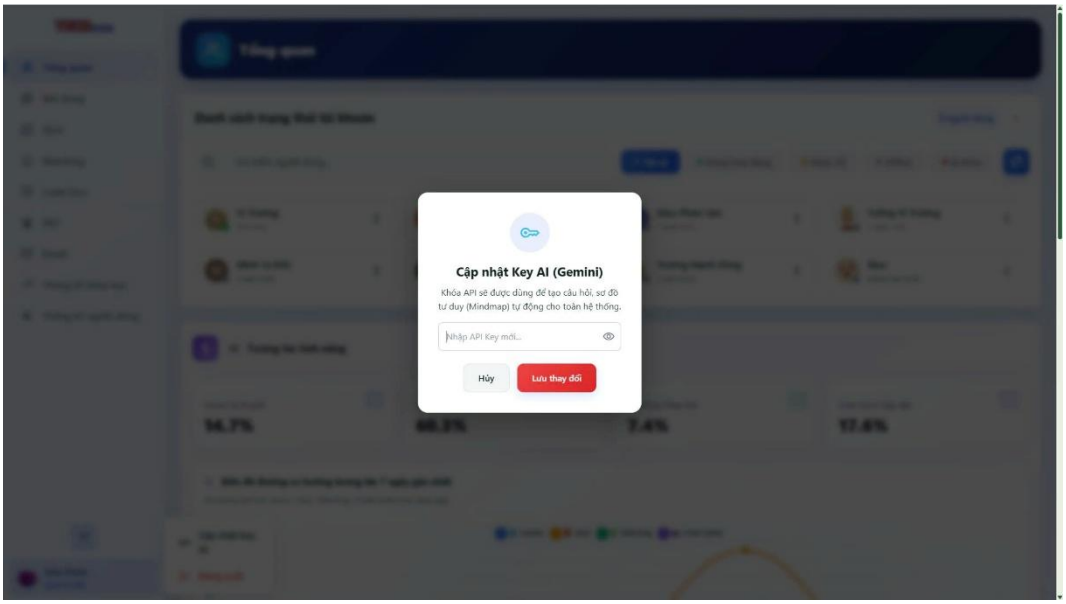

Figure 4.34 Gemini API Key Management Interface

4.5.2. AI-Based Lesson Content Processing

AI-Based Lesson Content Processing focuses on changing lesson content into different learning resources and practice data. The lesson content is sent to the Gemini API together with a request for each specific function. After Gemini returns the result, the system processes the data into the required structure before displaying or saving it.

For Mindmap, AI analyzes the lesson content and identifies the main ideas, supporting ideas, and the relationships between them. The result is changed into a branch structure and displayed as a Mindmap. Users can use this diagram to understand the knowledge structure of the lesson in a visual way.

For Quiz, AI creates questions based on the lesson content. The generated data includes the question, answer options, and the correct answer. The result is then used in the management interface or practice interface, depending on the system process.

For Matching Game, AI identifies important terms and their related information from the lesson. The result is organized into pairs of terms and definitions to create data for the matching activity. Each pair must have a clear relationship so that the system can check the User's answers during the practice activity.

AI is also used in the function for retrying incorrect Quiz questions. When a User answers a question incorrectly, the system uses the related lesson content to ask Gemini to create a new question. The new question checks the same knowledge area

but uses different wording or answer options. This method helps the User practice the content that they have not fully understood instead of repeating the same question.

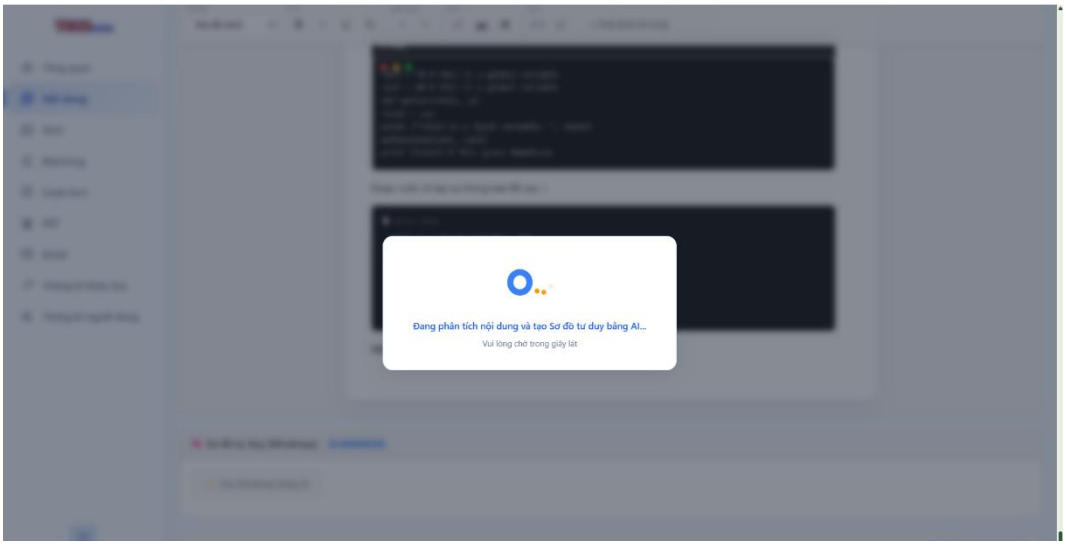

Figure 4.35 AI Mindmap Generation Interface

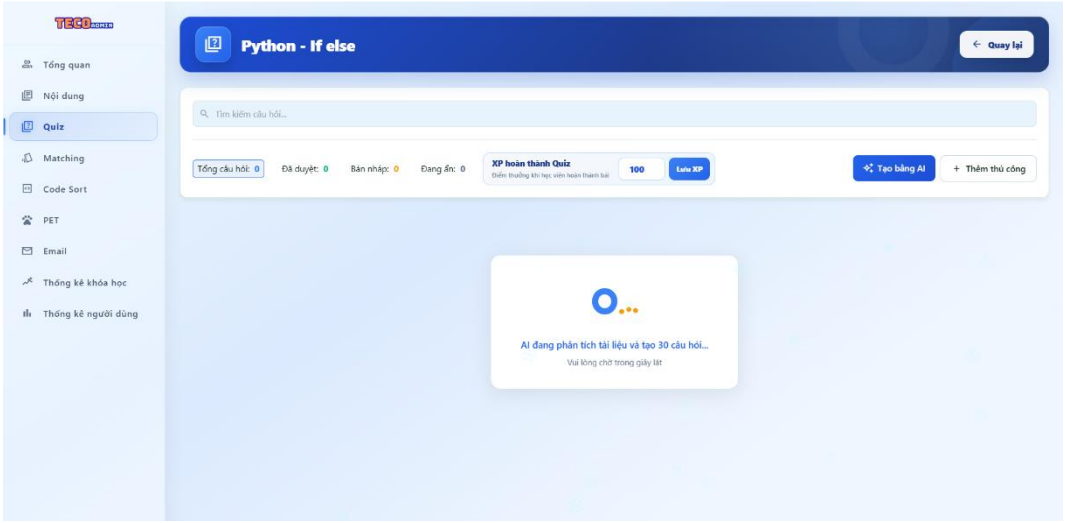

Figure 4.36 AI Quiz Question Generation Interface

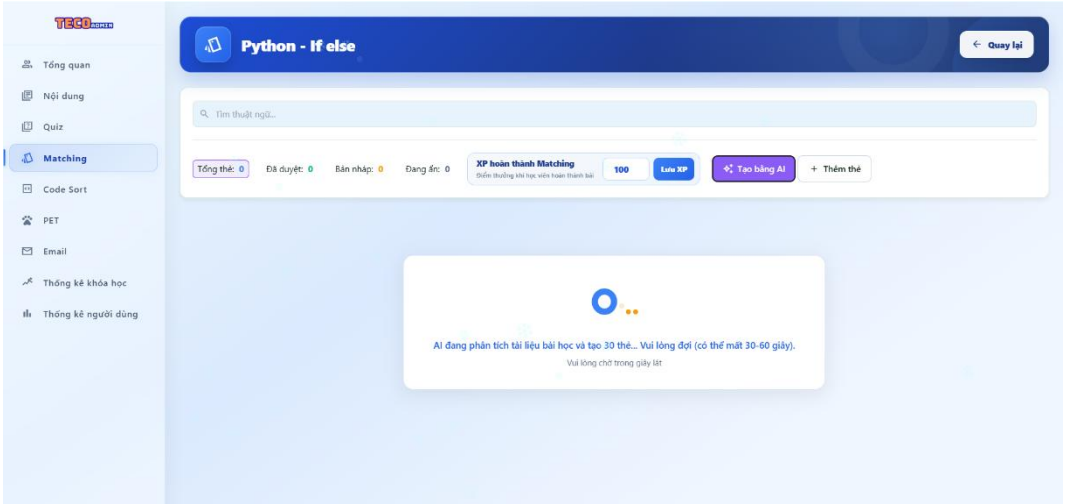

Figure 4.37 AI Matching Game Content Generation Interface

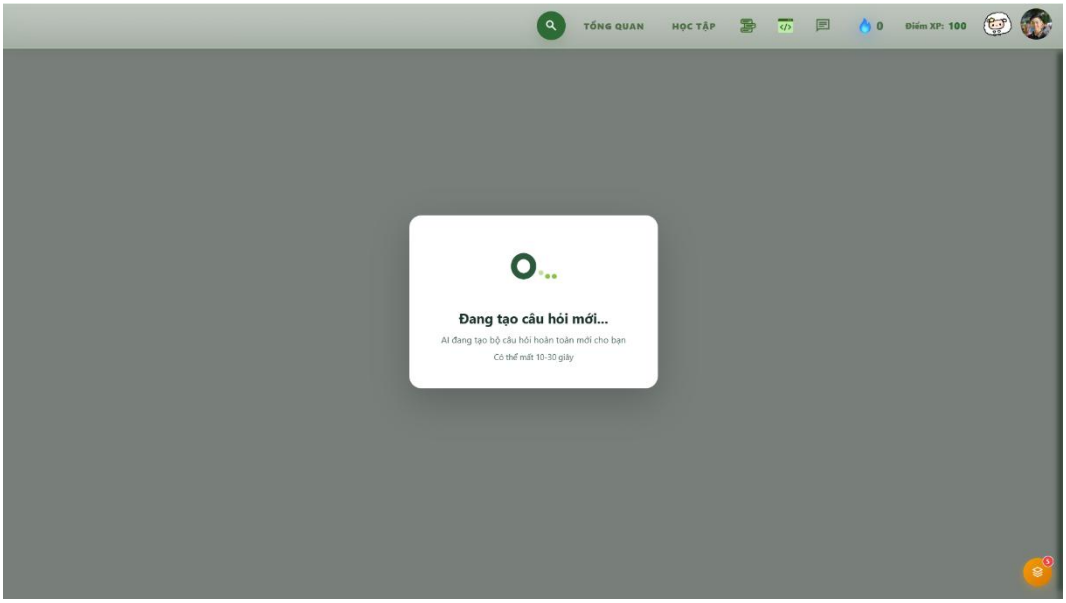

Figure 4.38 AI Quiz Generation Interface for Retrying Incorrect Answers

4.6. Implementation of Gamification, Pet, and Learning Progress

4.6.1. Visual Progress and Pet System

Pet is a gamification feature with its own function page. On the User side, the system shows the Pet, its current status, and information related to the interaction process. Users can open the Pet page from the navigation bar to view the Pet and perform the available activities.

The Pet status is stored for each User account. When an activity affects the Pet, the system updates the related data and shows the new status the next time the User opens the Pet page. Therefore, each User has their own Pet data instead of sharing the same Pet status with all learners.

On the Admin side, the system provides Pet management functions to control the data used for Users. Admin can manage the Pet list, display information, status, and settings related to the interaction system. This information is used as common Pet data and is then combined with the personal Pet data of each User.

The general process is organized as follows:

Admin configures Pet data

↓

System saves Pet data

↓

User opens the Pet page

↓

User plays or interacts with the Pet

↓

System updates the User's Pet status

↓

System displays the new status

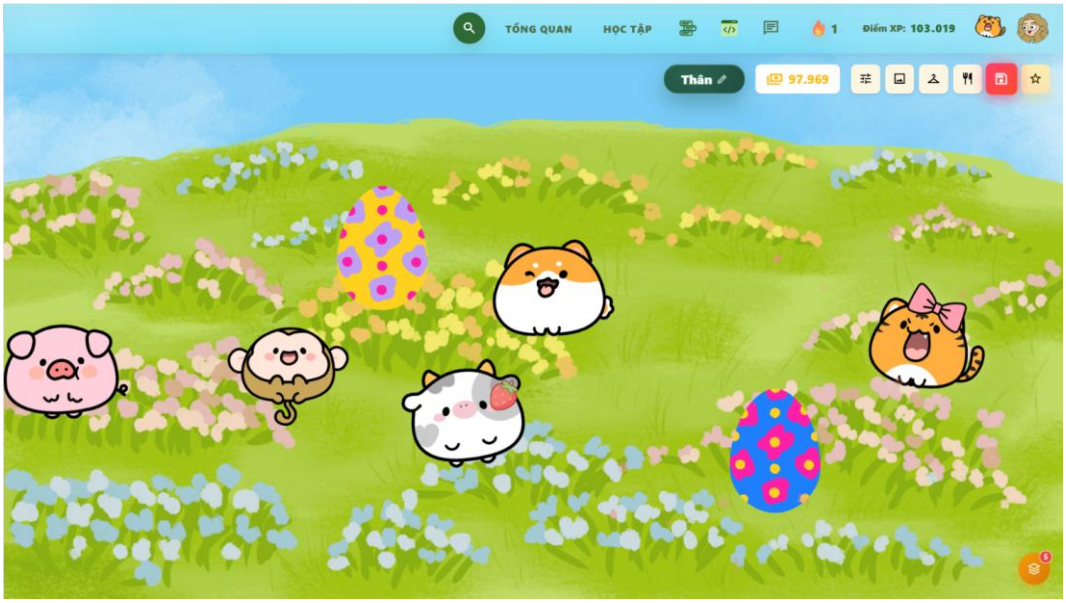

Figure 4.39 Pet Interface for User

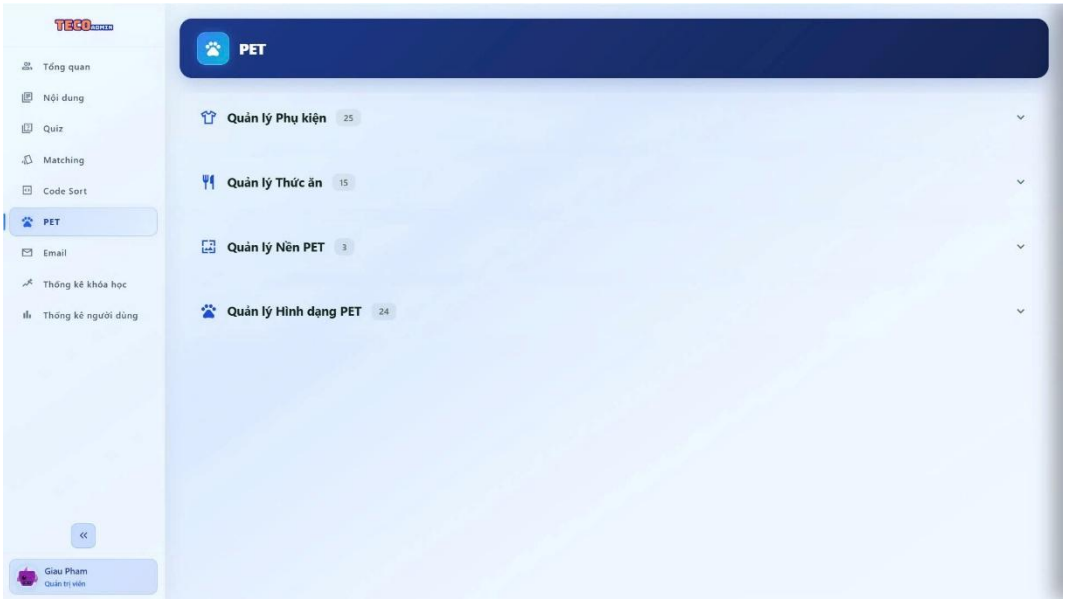

Figure 4.40 Pet Management Interface for Admin

4.6.2. XP, XQP, Daily Quest and Progress Tracking

XP, XQP, and Daily Quest are updated based on the actual activities of the User and are mainly displayed on the Dashboard. Each type of data has a different role in the gamification system.

XP is recorded when the User completes activities that have reward settings, such as learning a lesson, completing a Quiz, playing a Matching Game, or completing a Code Arrangement exercise. After the result is confirmed, the system calculates the related XP value and adds it to the User's data.

XQP is managed as a separate type of gamification point. The XQP value is updated based on the conditions configured in the system and is stored in the User's personal data.

Daily Quest includes daily learning tasks. Each task has its own completion condition.

When the User completes an activity, the system checks whether the activity is related to a Daily Quest. If the condition is met, the task status is updated. The Dashboard then shows the new progress of the User.

Learning progress is updated together with the gamification data. After the User completes part of the learning content or a practice activity, the system updates the related topic data. The topic status can then change between not started, in progress, and completed based on the configured conditions.

The general process is as follows:

User performs an activity

↓

System records the result

↓

System checks the completion conditions

↓

System updates learning progress

↓

System updates XP and XQP if needed

↓

System checks Daily Quest

↓

System saves the data

↓

System displays the new status on the Dashboard

4.7. System Testing

4.7.1. Testing Objectives

The purpose of system testing is to check whether the implemented functions work correctly according to the system requirements. It also checks whether the data is updated correctly after each action.

The testing process focuses on the following objectives:

- Check account registration, email verification, login, Google OAuth, and password recovery.
- Check the learning process through courses, topics, and lessons.
- Check Quiz, Matching Game, Code Arrangement, and Code Playground.
- Check personal profile, learning goals, and learning reminder settings.
- Check Admin functions for managing accounts, courses, topics, lesson content, and practice content.
- Check the Gemini API connection and AI data generation.
- Check XP, XQP, Daily Quest, learning progress, and Pet data.
- Check access permissions for Guest, User, and Admin.
- Check that each User can only access the personal data stored in their own USER_DB.
- Check system responses when the input data is incorrect, the requested data does not exist, or an external service returns an error.
- Check data consistency when one activity updates several types of data, such as Quiz results, learning progress, and XP.

A test case is considered Passed when the actual result matches the expected result and does not create incorrect data in related functions.

4.7.2. Testing Environment

TerraCode was tested in an environment similar to the actual deployment environment of the system.

❖ **Client Environment**

- A personal computer with an Internet connection.
- A web browser that supports JavaScript.
- The interface is developed with HTML5, CSS3, and JavaScript ES6+.

❖ **Server Environment**

- Google Apps Script Web App.
- Google Apps Script V8 Runtime.
- Server-side functions written in JavaScript.

❖ **Data Environment**

- MASTER_DB stores common system data.
- USER_DB stores the personal learning data of each User.
- Google Drive manages USER_DB files.

❖ **Related Services**

- Google Docs provides lesson content.
- Google OAuth 2.0 supports login with a Google account.
- Gmail API sends verification codes, password recovery codes, and learning reminder emails.
- Gemini API supports AI functions.
- Chart.js supports statistical charts.
- D3.js and Markmap support Mindmap display.

The testing data includes USER and ADMIN accounts, courses, topics, lesson content, Quiz questions, Matching data, Code Arrangement exercises, Pet data, and Gemini API Keys. Some incorrect data was also prepared to test the system's error handling.

4.7.3. Testing Methods

Testing was performed for individual functions and complete system flows. The tester performed actions on the interface, checked the returned results, and reviewed the stored data after each action.

The following testing methods were used:

- **Functional testing:** Test an individual function such as login, opening a lesson, doing a Quiz, or creating a course.
- **Flow testing:** Test several steps in order. For example, a User logs in, selects a course, opens a topic, studies a lesson, completes a Quiz, and checks the learning progress.
- **Valid data testing:** Enter correct data and check whether the system processes it successfully.
- **Invalid data testing:** Enter a wrong password, wrong verification code, empty required fields, or an ID that does not exist.
- **Integration testing:** Check the connection between the Client, Server, MASTER_DB, USER_DB, Google services, and Gemini.
- **Authorization testing:** Send requests as Guest, USER, and ADMIN to check the correct permissions.
- **Data testing:** Compare the information shown on the interface with the data stored after an action.
- **Regression testing:** Run related test cases again after fixing an error to make sure the problem is solved and existing functions still work correctly.

Each test case contains four main parts: Test Case ID, Test Case, Test Data or Action, and Expected Result. During actual testing, the status Passed or Failed can be added to the result table.

4.7.4. Guest Function Testing

Guest testing focuses on functions that are available before the user enters the learning area. The main flows include accessing the system, account registration, email verification, login, Google OAuth, and password recovery. TerraCode checks the account, password, account status, and email verification status before creating a login session.

Table 4-1 Guest Function Test Cases

Test Case ID	Test Case	Action	Expected Result

G01	Access the system without login	Open TerraCode without a session	The Landing Page and Guest functions are displayed.
G02	Register a valid account	Enter a new email, password, password confirmation, and display name	A new account is created and a verification code is sent to the registered email.
G03	Register with an existing email	Enter an email that already exists in the system	A new account is not created, and an appropriate message is displayed.
G04	Register with missing data	Leave a required field empty	The account is not created, and the system asks for the missing data.
G05	Password confirmation does not match	Enter two different passwords	The account is not created.
G06	Verify email successfully	Enter a correct verification code that is still valid	The email is verified, the account is activated, and USER_DB is created.
G07	Verify email with a wrong code	Enter an incorrect verification code	The account is not verified, and an error message is displayed.
G08	Verify email with an expired code	Enter an expired verification code	The account is not verified.
G09	Login successfully	Enter the correct email and password of a verified account	A session is created, and the User is redirected to the Dashboard.

G10	Login with a wrong password	Enter the correct email but an incorrect password	No session is created, and a login error message is displayed.
G11	Login with an unverified account	Use an account with an unverified email	The User is not redirected to the Dashboard and is asked to verify the email.
G12	Login with Google	Complete the Google OAuth 2.0 process	The system receives Google account information, creates or updates the account, and creates a session.
G13	Forgot password	Enter a registered email	A recovery code is created and sent to the registered email.
G14	Reset password successfully	Enter the correct recovery code and a new password	The password is updated, and the used recovery code becomes invalid.
G15	Reset password with a wrong code	Enter an incorrect recovery code	The password is not updated.
G16	Reset password with an expired code	Enter an expired recovery code	The password is not updated, and the User is asked to create a new recovery code.

4.7.5. User Function Testing

User testing covers the learning flow after login, from the Dashboard to courses, topics, lessons, practice activities, and personal settings. Each function is tested separately first. After that, related functions are tested together to make sure that data is updated correctly during the learning process.

Table 4-2 User Function Test Cases

Test Case ID	Test Case	Action	Expected Result

U01	Open Dashboard	Login with a USER account	The Dashboard displays the correct data of the current User.
U02	View course list	Open Course List	The available courses are displayed.
U03	Search for a course	Enter a course name or keyword	Matching courses are displayed.
U04	Filter courses	Select a filter condition	The course list is updated based on the selected condition.
U05	Add a course to favorites	Select the favorite function	The favorite status of the course is updated.
U06	View course details	Select a course	Course information and the topic list are displayed.
U07	Open a topic without meeting the prerequisite	Select a locked topic	The locked status is displayed, and the User cannot start the topic through the normal learning flow.
U08	Open an available topic	Select a topic with all required conditions completed	The correct topic content is opened.
U09	View Theory	Open a lesson	The correct lesson content of the selected topic is displayed.
U10	View Mindmap	Open the Mindmap section	The Mindmap related to the lesson content is displayed.

U11	Study with Flashcards	Flip cards, move between cards, and update learning status	The correct content is displayed, and the learning status is saved.
U12	Complete a Quiz	Answer the questions and finish the Quiz	The system checks the answers, calculates the result, and saves the Quiz attempt.
U13	Give a wrong Quiz answer	Select an incorrect answer	Feedback is displayed, and the wrong answer is saved for review.
U14	Complete a Matching Game	Match terms with their definitions	The system checks the pairs and updates the result.
U15	Complete Code Arrangement correctly	Arrange code blocks in the correct order	The correct result is recorded and the practice attempt is saved.
U16	Complete Code Arrangement incorrectly	Arrange code blocks in the wrong order	An incorrect result message is displayed and the attempt is recorded.
U17	Run valid code in Code Playground	Enter valid code and select Run	The program runs and the output is displayed.
U18	Run incorrect code	Enter incorrect code and select Run	Error information is displayed in the output area.
U19	Reset Code Playground	Enter code and select Reset	The working area returns to its designed initial state.
U20	Edit profile	Update the display name or avatar	The new information is saved and displayed correctly when the page is opened again.

U21	Change password successfully	Enter the correct current password and a new password	The password is updated.
U22	Change password with a wrong current password	Enter an incorrect current password	The password is not updated.
U23	Set a daily learning goal	Select and save a learning goal	The goal is saved in the User's personal data.
U24	Set a learning reminder	Select time, days, and reminder options	The settings are saved and displayed correctly.
U25	View Reminder Log	Open the reminder history	Only records of the current User are displayed.
U26	Restore data after logging in again	Complete an activity, log out, and log in again	Previous progress and results are loaded correctly.

4.7.6. Admin Function Testing

Admin testing focuses on managing common system data and monitoring system activities. Content updates are also checked on the User side to confirm that changes are displayed correctly after the Admin saves or publishes them.

Table 4-3 Admin Function Test Cases

Test Case ID	Test Case	Action	Expected Result
A01	Access Admin Dashboard	Login with an ADMIN account	The Admin area and overview data are displayed.
A02	View User statistics	Open the statistics section	Statistics and indicators are displayed based on stored data.

A03	Manage account status	Select a User and update the account status	The new status is saved and displayed correctly.
A04	Create a course	Enter valid course information	A new course is created.
A05	Edit a course	Change the name, description, or related information	The new information is saved.
A06	Change course status	Update the display status	The User interface shows the new status correctly.
A07	Reorder courses	Change the course order	The new order is saved.
A08	Delete a course	Select test data and delete it	The data is processed according to the course management function.
A09	Create a topic	Enter topic information and select the related course	The topic is created in the correct course.
A10	Edit a topic	Change topic information	The new data is saved.
A11	Set a prerequisite	Select a prerequisite topic	The condition is saved and applied correctly on the User side.
A12	Update lesson content	Link a Google Docs source	The correct content is read from the selected document.
A13	Preview content	Select Preview	The lesson content is displayed before publishing.

A14	Publish a lesson	Review and publish the content	Users receive the correct published content.
A15	Create a Quiz question	Enter a question, answer choices, and correct answer	The question is saved in the Quiz question bank.
A16	Edit a Quiz question	Change the question content or correct answer	The related data is updated.
A17	Delete a Quiz question	Delete a test question	The question is no longer provided according to its management status.
A18	Manage Matching content	Create or edit term-definition pairs	Matching data is saved under the correct topic.
A19	Manage Code Arrangement	Create an exercise and code block order	The exercise is saved with the correct structure.
A20	Manage Pet data	Update common Pet data	The new Pet data is available to Users.
A21	Check updates between Admin and User	Admin edits content and User reloads it	The User receives the correct newly published data.

4.7.7. AI Function Testing

AI testing focuses on the Gemini API connection and the data returned by each AI function. AI-generated content can be different between requests. Therefore, the testing criteria focus on the data structure, its relation to the lesson content, and whether the generated result can be used by the related system function.

The current AI functions include checking the Gemini API Key, generating Mindmaps, generating Quiz questions, generating Matching content, and creating new questions based on questions that the User answered incorrectly.

Table 4-4 AI Function Test Cases

Test Case ID	Test Case	Action	Expected Result

AI01	Save Gemini API Key	Enter a valid API Key	The Key information is saved according to the system mechanism.
AI02	Check a valid API Key	Send a Key checking request	A successful response is received, and the Key status is updated.
AI03	Check an invalid API Key	Enter an invalid Key	An error message is displayed, and content generation does not continue.
AI04	Generate a Mindmap	Send lesson content to Gemini	Data containing main ideas and relationships is returned for Mindmap display.
AI05	Generate a Quiz	Request Quiz questions from lesson content	Questions, answer choices, and correct answers are returned.
AI06	Generate Matching content	Send lesson content	Term-definition pairs are returned.
AI07	Generate a new question from a wrong answer	Select knowledge related to a question answered incorrectly by the User	A new question is created for the same knowledge area.
AI08	Gemini returns empty data	Receive a response without content	Empty data is not saved as valid learning content.
AI09	Gemini returns data with an incorrect structure	Receive a response with missing required fields	Incorrect data is not used in learning or management functions.

AI10	Gemini connection error	The API does not respond or returns an error	An error status is displayed, and the request processing stops.
AI11	Check generated data	Open the AI result in the related function	The data can be read correctly by Mindmap, Quiz, or Matching.

4.7.8. Gamification Testing

Gamification testing focuses on Pet, XP, XQP, Daily Quest, and learning progress. The values are checked before and after each activity to make sure that the system updates the correct User data.

Table 4-5 Gamification Test Cases

Test Case ID	Test Case	Action	Expected Result
GM01	Update XP after a lesson	Complete a lesson with an XP value	XP is added to the User data based on the configured value.
GM02	Update XP after a Quiz	Complete a Quiz	XP is recorded according to the activity conditions.
GM03	Update XP after Matching	Complete a Matching Game	The result is recorded, and XP is updated when the conditions are met.
GM04	Update XP after Code Arrangement	Complete a Code Arrangement exercise	The related XP is updated.
GM05	Update XQP	Complete an activity that meets the required conditions	The new XQP value is saved for the User.
GM06	Complete a Daily Quest	Complete an activity included in the daily task	The related quest is changed to completed status.

GM07	Daily Quest condition is not completed	Perform an activity that does not meet all conditions	The quest status is not updated incorrectly.
GM08	Start a topic	Open the topic and perform the first activity	The topic status changes to in progress when the conditions are met.
GM09	Complete a topic	Complete all required activities	The topic status changes to completed.
GM10	Check separate Pet data for each User	User A interacts with the Pet, then switch to User B	The two accounts keep separate Pet data.
GM11	Update Pet after interaction	User performs an activity with the Pet	The new Pet status is saved and displayed.
GM12	Admin updates Pet data	Admin changes common Pet data	The User receives the new common Pet data, while personal Pet status remains connected to the User account.
GM13	Check data after reloading	Update Gamification data and reload the Dashboard	XP, XQP, Daily Quest, and learning progress are displayed according to the saved data.

4.7.9. Security and Role Authorization Testing

Security and authorization testing focuses on authentication, passwords, sessions, USER and ADMIN permissions, API Keys, and access to personal data.

In TerraCode, Guest is not a role stored in the database. Accounts are stored with either the USER or ADMIN role. The Server checks the userId and role before performing Admin functions.

Table 4-6 Security and Authorization Test Cases

Test Case ID	Test Case	Action	Expected Result

S01	Check password storage	Register an account or change a password, then check the stored data	The original password is not stored directly.
S02	Check a correct verification code	Enter a valid verification code	The verification process is completed.
S03	Check a wrong verification code	Enter an incorrect code	The verification request is rejected.
S04	Check an expired code	Use an expired code	The request is rejected.
S05	Valid session	Send a request from a logged-in account	The correct User is identified, and the request is processed.
S06	Missing session	Call a function that requires login without a session	Access is rejected.
S07	Invalid session	Send an invalid session	Personal data is not returned.
S08	ADMIN accesses an Admin function	Login with an ADMIN account	The operation is allowed after the role is confirmed.
S09	Protect system configuration	Check the data returned from the Server to the Client	Server configuration data that is not required by the interface is not returned.
S10	Protect Gemini API Key	Check unrelated interface areas	The API Key is not displayed in unnecessary locations.

4.7.10. Error, Exception, and Data Synchronization Testing

This testing group checks situations where the system does not follow the successful flow. These cases include missing data, data that does not exist, Google Docs errors, Gmail errors, and Gemini API errors. It also checks data consistency when one activity changes several types of data.

Google Sheets may have data conflicts when several requests write data at nearly the same time. TerraCode uses LockService for related write operations. The system also stores System_Logs to support the checking of errors, actions, and sessions when a problem occurs.

Table 4-7 Error, Exception, and Data Synchronization Test Cases

Test Case ID	Test Case	Action	Expected Result
E01	Missing required data	Submit a form with a required field left empty	The system does not continue processing and displays an appropriate message.
E02	Incorrect data format	Enter data in an invalid format	Incorrect data is not stored in the system.
E03	Course ID does not exist	Send an invalid Course ID	Data from another course is not returned.
E04	Topic ID does not exist	Send an invalid Topic ID	The system does not process it as a valid topic.
E05	Question ID does not exist	Send an invalid Question ID	The result is not saved to another question.
E06	Google Docs cannot be accessed	Use an invalid document or a document without access permission	An error message for reading the content is displayed.
E07	Gmail fails to send an email	An error occurs while sending a code or learning reminder	The system does not report a successful send and records the error status.

E08	Gemini API error	The API returns an error	Content generation stops, and existing data is kept.
E09	Gemini returns invalid data	The response has an incorrect structure	The data is not saved as valid learning content.
E10	USER_DB cannot be read	An error occurs while loading User data	An error message is displayed, and data from another User is not used as a replacement.
E11	Complete a Quiz	Save the Quiz result and update XP	The Quiz result and XP represent the same attempt.
E12	Quiz contains wrong answers	Complete a Quiz with incorrect answers	The Quiz result and related wrong-answer data are saved.
E13	Activity related to Daily Quest	Complete a related activity	The activity result and Daily Quest are updated consistently.
E14	Update progress and XP	Complete a lesson	Learning progress and XP represent the same activity.
E15	Two write requests occur close together	Send two update actions one after another	Data is not lost or incorrectly overwritten.
E16	Reload after an update	Complete an activity and reload the page	The data shown on the interface matches the stored data.
E17	Log out and log in again	Complete an activity before logging out	Learning progress and results are restored correctly after login.

E18	Admin updates a course	Edit course data and let the User reload the page	The User receives the new data according to its published status.
E19	Admin updates a prerequisite	Change the condition for opening a topic	The new prerequisite is applied correctly on the User side.
E20	Record an error log	Perform an error situation that is tracked by the system	System_Logs stores information for checking the problem.

4.8. System Evaluation

4.8.1. Functional Evaluation

The system has implemented the main function groups for Guest, User, and Admin based on the system design.

For Guest, the system supports account registration, email verification, login with Email/Password, login with Google, and password recovery. After a successful login, the system creates a login session and moves the User to the learning area.

For User, the system provides learning functions based on courses, topics, and lessons. The User can study Theory, Mindmap, and Flashcard, then complete a Quiz, Matching Game, Code Arrangement, or practice in the Code Playground. Practice results are checked and saved for each account. Wrong Quiz answers are also saved for later practice.

The User can manage personal information, change the password, set a daily learning goal, set learning reminders, and view the reminder history. Learning data is loaded again after the User logs out and logs in again.

For Admin, the system supports the management of accounts, courses, topics, lesson content, Quiz, Matching Game, Code Arrangement, and Pet data. Admin can also view statistics and learning activities. When Admin updates and publishes content, the new data is displayed on the User side based on the configured status.

The main functions are connected in the same processing flow. For example, after the User completes a Quiz, the system saves the result, records wrong answers, and

updates related data. This allows learning, practice, and progress tracking functions to work with the same personal learning data.

4.8.2. User Interface and User Experience Evaluation

The system interface is divided based on each user group. Guest focuses on registration, login, and account recovery. User focuses on learning, practice, progress tracking, and personal settings. Admin has a separate area for managing data and learning content.

The User learning flow is organized in the following order: Dashboard → Courses → Topics → Lessons → Practice

After selecting a course, the User can view the topic list and the status of each topic. The system checks the prerequisite before allowing the User to open the next content. If the prerequisite has not been completed, the topic remains locked. When the User completes the required condition, the related topic is opened.

In each lesson, Theory, Mindmap, and Flashcard are provided as separate learning forms. Practice functions are also divided into Quiz, Matching Game, Code Arrangement, and Code Playground. This structure helps the User clearly understand which function is being used.

The Dashboard displays information such as XP, XQP, Daily Quest, and learning progress. Profile and personal settings are separated from the lesson area. This structure keeps the learning flow clear and avoids mixing personal settings with learning content.

For Admin, management functions are divided into User management, course management, topic management, lesson content management, practice content management, AI management, and system data management. Admin can work with each data group without moving to the User learning interface.

4.8.3. Self-Learning Support Evaluation

The system supports Python self-learning through the following continuous process: Select Content → Study Lesson → Practice → View Result → Save Progress → Continue Learning

Learning content is organized into courses, topics, and lessons. This structure helps the User study each part in order instead of working with separate learning documents.

Theory provides the main lesson content. Mindmap shows the relationship between the main ideas. Flashcard supports the review of terms and knowledge. After studying the lesson, the User can complete the related practice activities.

Quiz checks answers and records the result. When the User gives a wrong answer, the system saves the question in the wrong-answer data. Matching Game asks the User to match terms with their definitions. Code Arrangement asks the User to arrange code blocks in the correct order. Code Playground allows the User to enter code, run the program, and view the result.

The system stores learning progress separately for each account. When the User completes an activity, the related data is updated in USER_DB. After reloading the page or logging in again, the system displays the previously saved data.

The User can also set a daily learning goal and learning reminders. These functions help the learner organize self-learning based on their personal study time.

4.8.4. Gamification Evaluation

Gamification is directly connected to the User's learning activities. The main elements include XP, XQP, Daily Quest, learning progress, and Pet.

XP is updated when the User completes activities that have reward settings, such as lessons, Quiz, Matching Game, or Code Arrangement. After the system confirms the result, the XP value is saved in the User's data.

XQP is managed separately based on the configured conditions. Daily Quest includes daily learning tasks. When the User completes an activity, the system checks the conditions of the related task. If the conditions are completed, the Daily Quest status is updated.

Topic progress is updated based on learning activities. A topic can have three statuses: not started, in progress, and completed. After the User completes all required activities, the system changes the topic status to completed.

Pet data is stored separately for each User account. When the User interacts with the Pet, the system saves the new status in personal data. Different accounts keep separate

Pet data. Admin manages common Pet data, while each User still keeps their own personal Pet status.

This structure connects rewards and gamification status with real learning activities. The system does not update Daily Quest or learning progress when the required conditions are not completed.

4.8.5. AI Application Evaluation

The system integrates the Gemini API to process and generate learning content. The AI functions include Mindmap generation, Quiz question generation, Matching Game content generation, and new question generation based on questions that the User answered incorrectly.

For Mindmap, the system sends lesson content to Gemini. The result includes main ideas, related ideas, and the relationships between them. The system then converts the data into a suitable structure for Mindmap display.

For Quiz, Gemini returns questions, answer choices, and correct answers. For Matching Game, AI creates pairs of terms and definitions. When the User answers a Quiz question incorrectly, the system uses the related knowledge and asks Gemini to create a new question about the same knowledge area.

The system checks AI data before using it in related functions. If Gemini returns empty data or data without required fields, the data is not saved as valid learning content. If an API error occurs, the generation process stops and the system displays an error status.

The Gemini API Key is also checked before AI content generation. A valid API Key receives the correct status. An invalid API Key stops the content generation process. On the Admin side, AI-generated content goes through a checking process before publication. Admin controls the learning content before it is provided to the User.

4.8.6. Performance and Stability Evaluation

The system is deployed as a Web App on Google Apps Script V8 Runtime. MASTER_DB stores common system data. USER_DB stores personal learning data for each User. Google Drive manages USER_DB files. Google Docs provides lesson content. Gmail API is used to send emails. Gemini API is used for AI functions.

During testing, the system keeps related data consistent between functions. When the User completes a Quiz, the Quiz result and XP are recorded for the same attempt. When an activity is related to a Daily Quest, the activity result and task status are updated together. When the User completes a lesson, learning progress and XP are updated for the same activity.

The system also checks cases where two data-writing requests happen close together. LockService is applied to related write operations to control data conflicts in Google Sheets.

After the User completes an activity and reloads the page, the displayed data matches the saved data. When the User logs out and logs in again, previous learning progress and results are loaded again. When Admin updates a course or prerequisite, the new data is applied on the User side after the page is reloaded.

The system handles errors related to Google Docs, Gmail API, Gemini API, and USER_DB. When an error occurs, the related process stops, incorrect data is not saved to other functions, and the related information is stored in System_Logs for error checking.

Within the scope of this project, the system supports learning, administration, AI, and gamification processes on Google Apps Script. It also keeps data consistent between the interface, Server, and data storage sources.

CHAPTER 5

CONCLUSION AND FUTURE DEVELOPMENT

5.1. Conclusion

The topic “Design of a Gamified Python Self-Learning Support System on Google Apps Script” was carried out with the goal of building a Web system to support beginners in learning Python through an organized learning process. The system combines learning content, practice activities, progress tracking, Gamification, and AI in one environment. These were also the main goals defined at the beginning of the project.

Through the process of requirement analysis, design, development, testing, and evaluation, the system has completed the main groups of functions for Guest, User, and Admin. Guest can perform functions related to accounts and authentication. User can learn through courses, topics, and lessons, and then practice through different activities. Admin can manage users, learning content, practice data, Pet, AI, and statistical information.

The self-learning process is organized in a continuous process: Select Content → Learn Lesson → Practice → View Results → Save Progress → Continue Learning

The learning content is divided into Course, Topic, and Lesson. In each lesson, User can learn knowledge through Theory, Mindmap, and Flashcard. After that, User can practice through Quiz, Matching Game, Code Arrangement, or Code Playground. Results and progress are stored separately for each account, allowing learners to continue from their previous data when they return to the system.

Gamification is directly connected to learning activities instead of working as a separate activity. XP, XQP, Daily Quest, topic progress, and Pet are updated based on specific conditions. The system checks the required conditions before recording a quest or completing a topic. Pet data is also stored separately for each account.

The Gemini API is integrated to support the processing of learning content. The current AI functions include generating Mindmaps, Quiz questions, Matching Game content, and new questions based on the knowledge that User answered incorrectly. AI data is checked before it is used in the related functions. AI-generated content in the Admin area also needs to be checked by Admin before it is published.

From a technical perspective, the system operates using a Client-Server model on the Google Apps Script V8 Runtime. MASTER_DB stores common data, while USER_DB stores learning data for each User separately. The system connects with Google Drive, Google Docs, Gmail API, and Gemini API for the related functions. Testing results show that data related to learning results, XP, Daily Quest, and progress is updated based on the same learning activity. LockService is used for data-writing operations to reduce conflicts when working with Google Sheets.

The results of the project show that Google Apps Script is suitable for building a Python self-learning support system at the scale of this project. This platform makes it possible to connect Google services in one system without deploying a separate server. However, limitations in execution time, quotas, service calls, and data processing make the current system more suitable for a moderate number of users rather than a large number of users.

5.2. Achieved Results

After the development and testing process, the project has completed the main functional groups according to the proposed objectives. The system supports the full process from account registration, learning, practice, and progress tracking to content management, Gamification, and AI.

- For Guest, the system supports account registration, email verification, login with Email or Google, password recovery, and login session creation. These functions create the access process before the user enters the learning area.
- For User, learning content is organized into Course, Topic, and Lesson. User can learn through Theory, Mindmap, and Flashcard, and can also track the status of each topic. The system checks the prerequisite conditions before allowing User to access the next content.
- The practice functions include Quiz, Matching Game, Code Arrangement, and Code Playground. The results are stored separately for each account. For Quiz, the system records incorrect answers so that User can review them later.
- The system stores learning progress in USER_DB and restores the data when User reloads the page or logs in again. User can also update the profile, change the password, set learning goals, create reminders, and track personal results.

- The Gamification components include XP, XQP, Daily Quest, topic progress, and Pet. Reward points and quest status are updated based on learning activities. Pet data is stored separately for each User, while Admin manages the shared Pet data.
- The Admin area supports the management of accounts, courses, topics, lesson content, Quiz, Matching Game, Code Arrangement, Pet, AI, and statistical data. After the content is updated and published by Admin, it is displayed to User based on the configured status.
- The Gemini API is integrated to generate Mindmaps, Quiz questions, Matching Game content, and new questions based on the knowledge that User answered incorrectly. The system checks the API Key and the structure of AI data before saving or adding it to the learning content.
- The testing process included Guest, User, Admin, AI, Gamification, access control, login sessions, error handling, and data synchronization. LockService is applied to related data-writing operations.

5.3. Contributions of the Project

The project focuses on building a software product and organizing a Python self-learning support system on the Google Apps Script platform. The main contributions of the project are shown in the following areas:

- Building a connected learning model between lesson content, practice activities, and personal progress. Quiz results, incorrect answers, XP, Daily Quest, and learning status are stored for each User. This allows the learning process to be tracked continuously and provides a foundation for future personalized features.
- Integrating different learning and practice methods into the same Web App. Theory, Mindmap, and Flashcard support learning and reviewing knowledge. Quiz, Matching Game, Code Arrangement, and Code Playground support practice in different ways. This organization creates a continuous learning process from learning content to practice.
- Connecting Gamification directly to learning activities. XP, Daily Quest, Topic progress, and Pet are updated based on the actual conditions and results of User. Therefore, the game elements have a clear connection with the learning process instead of working as separate activities.

- Integrating Gemini AI into the process of creating and supporting learning content. AI is used to create Mindmaps, Quiz questions, Matching Game content, and new practice questions. AI data is checked before it is saved or published. This allows Admin to control the content before providing it to User.
- Building a separate data model for shared and personal data. MASTER_DB stores common system data, while USER_DB stores the progress, results, and learning information of each User. This organization makes data management clearer.
- Developing an educational Web App on Google Apps Script with connections to Google Sheets, Google Drive, Google Docs, Gmail API, Google OAuth 2.0, and Gemini API. The results of the project show that this model is suitable for a Python self-learning support system at the scale of a graduation project.

5.4. System Limitations

Although the main functions have been developed and tested, the system still has some limitations within the scope of the project:

- Google Apps Script has limits on execution time, quota, and the number of requests at the same time. Therefore, the current system is more suitable for a moderate number of users and has not been tested with many Users accessing the system at the same time.
- Google Sheets is convenient for deploying and managing data, but it is not suitable for systems with a large amount of data. When the number of records increases, searching, connecting, and updating data may reduce system performance.
- Some functions depend on Google Drive, Google Docs, Gmail API, Google OAuth 2.0, and Gemini API. When these services have errors, reach their quotas, or change their access permissions, the related functions may be interrupted.
- Content generated by Gemini still needs to be checked before it is used for learning. The current system mainly checks the data structure, while a deeper evaluation of accuracy, difficulty, and suitability for learners is still limited.
- The current Python content does not fully cover many learning levels and advanced topics. The number of courses, lessons, and practice activities is still suitable for the scale of a graduation project rather than a complete training platform.

- The current system evaluation mainly focuses on functions, data, access control, error handling, and stability. The project has not conducted a long-term study with a large number of learners to evaluate improvements in learning results and learning habits.
- The Gamification components, such as XP, XQP, Daily Quest, progress, and Pet, work according to the design. However, there is not enough real user data to evaluate the effect of each component on learning motivation and learning effectiveness.

5.5. Future Development

Based on the current results and limitations, the system can be further developed in the following directions:

- Expand the Python learning content and learning path for different levels, from basic knowledge to more advanced topics. Each Topic can include entry requirements, learning goals, difficulty levels, and suitable practice activities to create a clearer learning path.
- Develop a personalized learning system based on the saved data of each User. The system can suggest topics that need to be reviewed, identify knowledge areas that User often answers incorrectly, recommend suitable practice activities, and adjust the difficulty of questions based on learning results.
- Improve the AI functions by checking the generated content more carefully, classifying difficulty levels, detecting duplicate questions, saving content versions, and allowing Admin to edit the content before approval. AI can also be developed to support personalized learning instead of mainly creating learning content.
- Expand Gamification based on the actual learning progress of User. Daily Quest can be created flexibly for each User, combined with review tasks for weak knowledge areas and more interaction with Pet. Rewards should continue to be connected to learning activities instead of focusing too much on scores.
- Upgrade the data architecture when the number of users and the amount of data increase. Some frequently accessed data can be moved to a more suitable database management system, while Google Apps Script can continue to handle functions related to Google Workspace.

- Improve system performance by reducing the number of accesses to Google Sheets, Google Drive, Google Docs, and external APIs. CacheService, batch data processing, and reducing the reloading of unchanged data can help reduce response time.
- Expand testing with many Users working at the same time to evaluate response time, error rate, data-writing ability, and quota usage. These results can help identify the actual operating limits of the system.
- Conduct a longer evaluation with real Python learners. Data such as lesson completion rate, practice results, the number of reviews of incorrect answers, Daily Quest usage, and feedback on the interface, Gamification, Mindmap, or Flashcard can be used to evaluate the effectiveness of the system more clearly.

REFERENCES

[1] Chart.js, “Chart.js documentation,” 2025. https://www.chartjs.org/docs/latest/

[2] D3, “What is D3?,” n.d. https://d3js.org/what-is-d3 (accessed Aug. 31, 2026).

[3] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, “From game design elements to gamefulness: Defining ‘gamification’,” in *Proceedings of the 15th International Academic MindTrek Conference*, 2011, pp. 9–15, doi: 10.1145/2181037.2181040.

[4] DiceBear, “What is DiceBear?,” n.d. https://www.dicebear.com/start/what-is-dicebear/

[5] Gemini Team et al., “Gemini: A family of highly capable multimodal models,” *arXiv*, 2023, doi: 10.48550/arXiv.2312.11805.

[6] Git, “About Git,” n.d. https://git-scm.com/about

[7] Google, “clasp: Command line Apps Script projects,” GitHub, n.d. https://github.com/google/clasp

[8] Google, “Gmail API overview,” Google for Developers, 2026. https://developers.google.com/workspace/gmail/api/guides

[9] Google, “Google Apps Script overview,” Google for Developers, 2026. https://developers.google.com/apps-script/overview

[10] Google, “Google Docs API overview,” Google for Developers, 2026. https://developers.google.com/workspace/docs/api/how-tos/overview

[11] Google, “Google Drive API overview,” Google for Developers, 2026. https://developers.google.com/workspace/drive/api/guides/about-sdk

[12] Google, “Google Sheets API overview,” Google for Developers, 2026. https://developers.google.com/workspace/sheets/api/guides/concepts

[13] Google, “Quotas for Google services,” Google for Developers, 2026. https://developers.google.com/apps-script/guides/services/quotas

[14] Google, “V8 runtime overview,” Google for Developers, 2026. https://developers.google.com/apps-script/guides/v8-runtime

[15] Google, “Web apps,” Google for Developers, 2026. https://developers.google.com/apps-script/guides/web

[16] J. Hamari, J. Koivisto, and H. Sarsa, “Does gamification work? A literature review of empirical studies on gamification,” in *2014 47th Hawaii International Conference on System Sciences*, 2014, pp. 3025–3034, doi: 10.1109/HICSS.2014.377.

[17] html2canvas, “Documentation,” n.d. https://html2canvas.hertzen.com/documentation

[18] Markmap, “Markmap documentation,” n.d. https://markmap.js.org/

[19] MDN Web Docs, “CSS: Cascading Style Sheets,” Mozilla, 2026. https://developer.mozilla.org/en-US/docs/Web/CSS

[20] MDN Web Docs, “JavaScript technologies overview,” Mozilla, 2026. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/JavaScript_technologies_overview

[21] MDN Web Docs, “Working with the History API,” Mozilla, 2026. https://developer.mozilla.org/en-US/docs/Web/API/History_API/Working_with_the_History_API

[22] R. P. Medeiros, G. L. Ramalho, and T. P. Falcão, “A systematic literature review on teaching and learning introductory programming in higher education,” *IEEE Transactions on Education*, vol. 62, no. 2, pp. 77–90, 2019, doi: 10.1109/TE.2018.2864133.

[23] F. Miao and W. Holmes, *Guidance for Generative AI in Education and Research*. UNESCO, 2023. https://unesdoc.unesco.org/ark:/48223/pf0000386693

[24] Microsoft, “Get started with Visual Studio Code,” 2026. https://code.visualstudio.com/docs/getstarted/overview

[25] OECD, *Explanatory Memorandum on the Updated OECD Definition of an AI System*, OECD Artificial Intelligence Papers, no. 8. OECD Publishing, 2024, doi: 10.1787/623da898-en.

[26] Python Software Foundation, “General Python FAQ,” Python Documentation, n.d. https://docs.python.org/3/faq/general.html

[27] Python Software Foundation, “The Python tutorial,” Python Documentation, n.d. https://docs.python.org/3/tutorial/

[28] M. Sailer and L. Homner, “The gamification of learning: A meta-analysis,” *Educational Psychology Review*, vol. 32, no. 1, pp. 77–112, 2020, doi: 10.1007/s10648-019-09498-w.
[29] WHATWG, “HTML: The Living Standard,” 2026. https://html.spec.whatwg.org/dev/

APPENDICES

APPENDIX A. ADDITIONAL SYSTEM INTERFACES

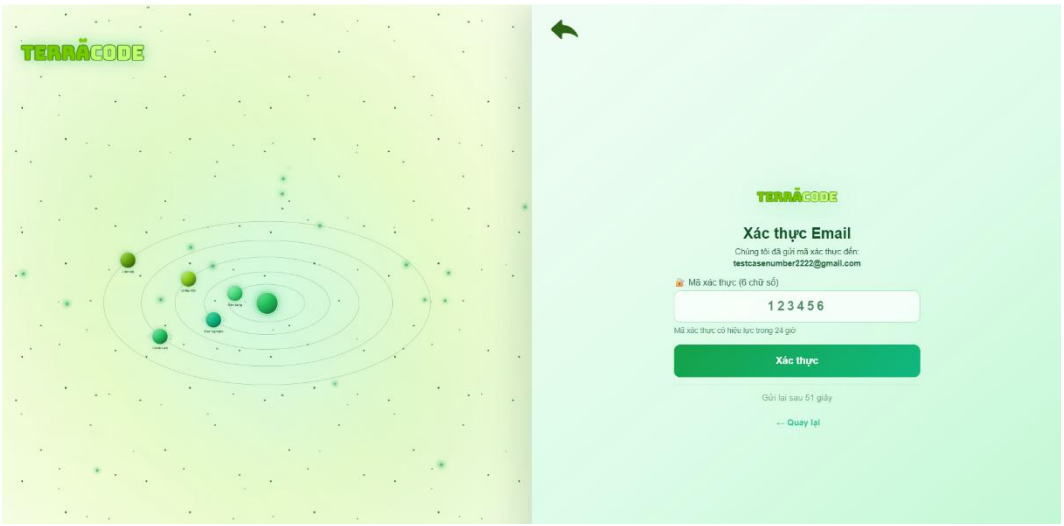

Figure A.1 Email Verification Interface

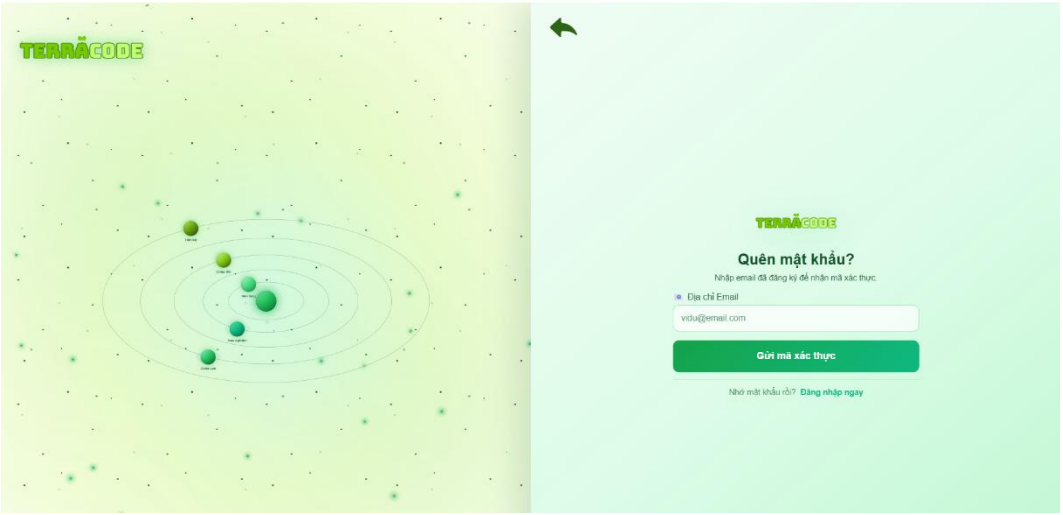

Figure A.2 Forgot Password Interface

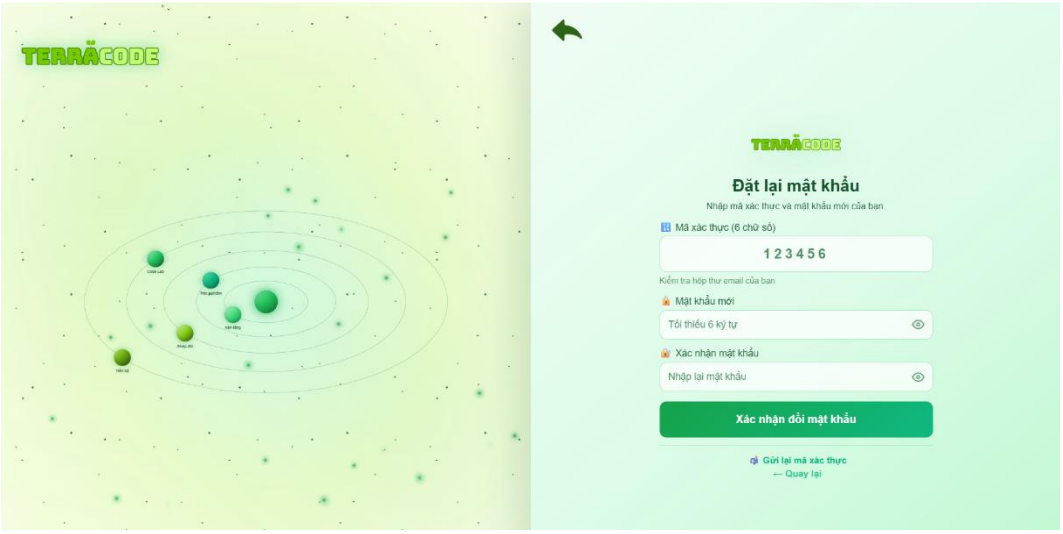

Figure A.3 Reset Password Interface

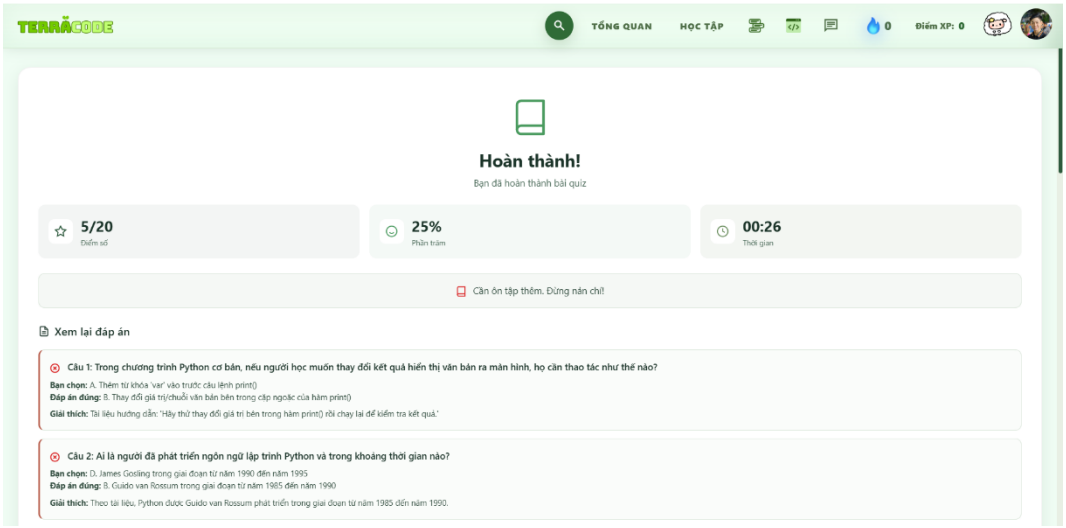

Figure A.4 Quiz Result Interface

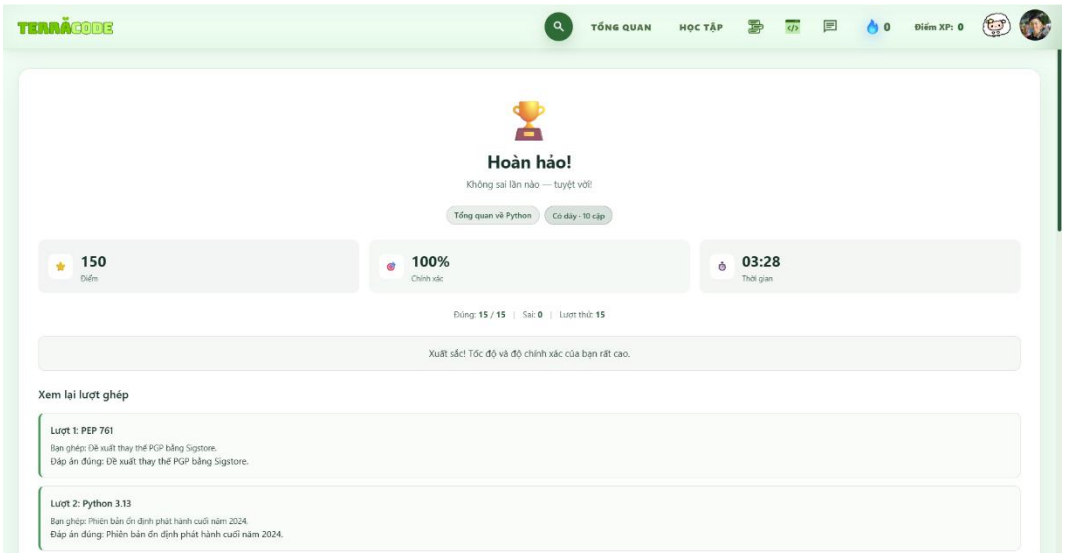

Figure A.5 Matching Game Result Interface

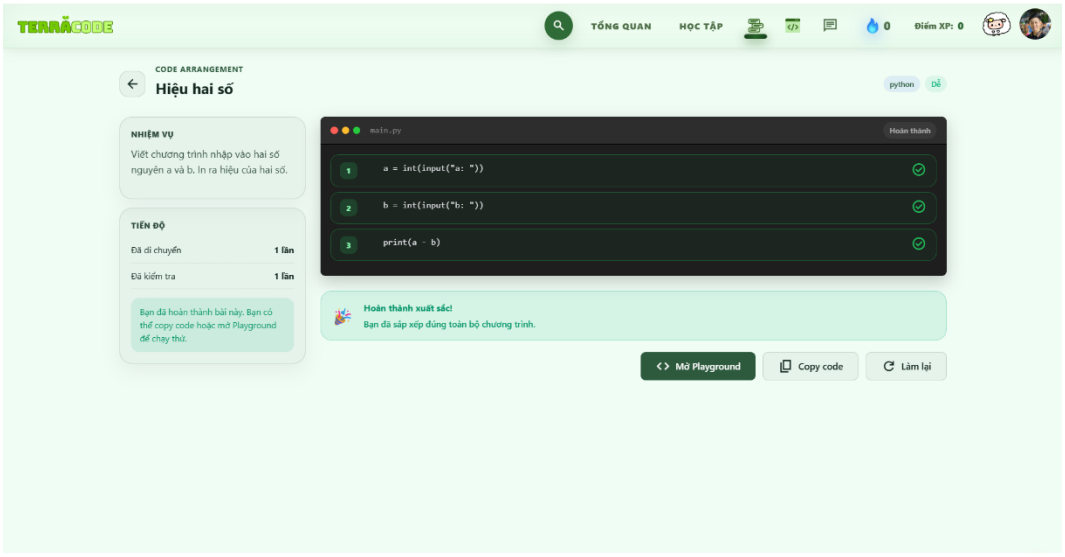

Figure A.6 Code Arrangement Results Interface

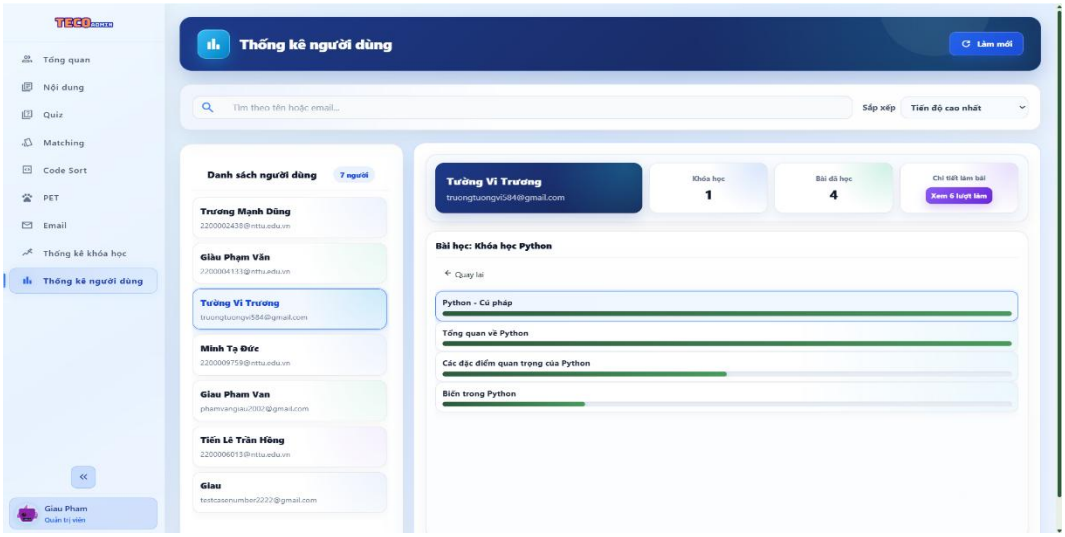

Figure A.7 User Statistics Chart

Figure A.8 Course Statistics Chart

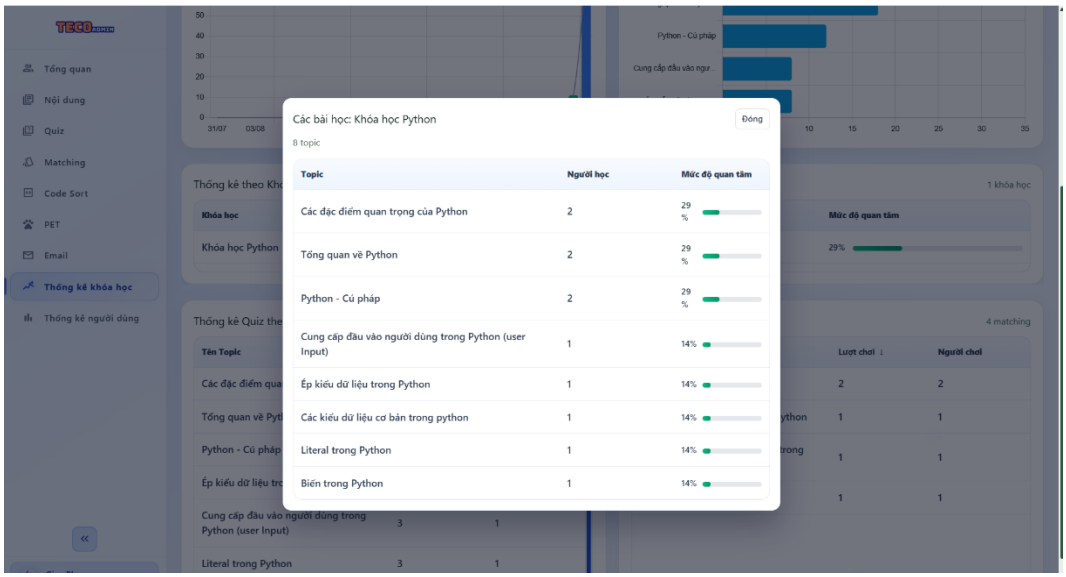

Figure A.9 Topic Performance Statistics

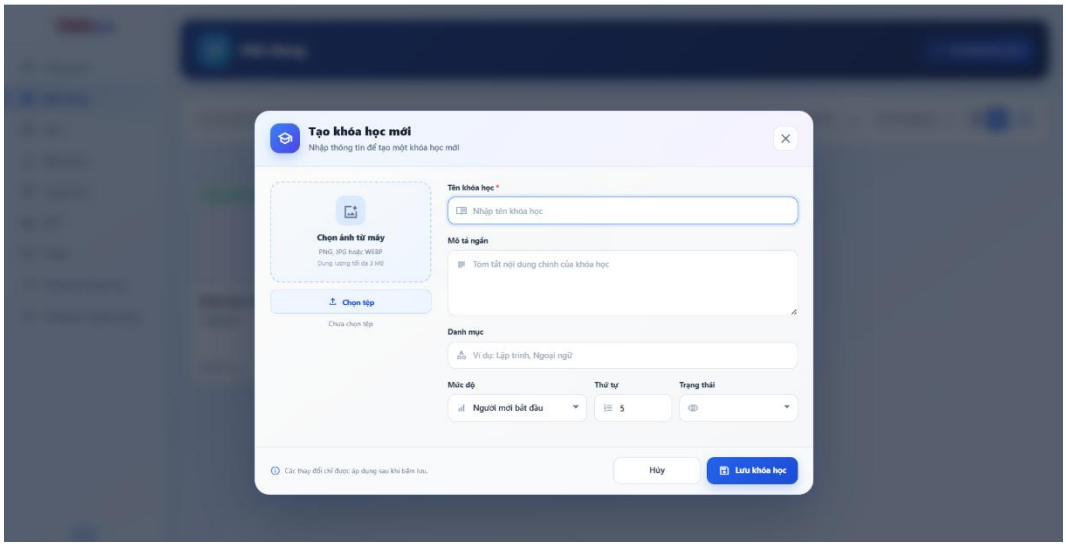

Figure A.10 Course Creation Interface

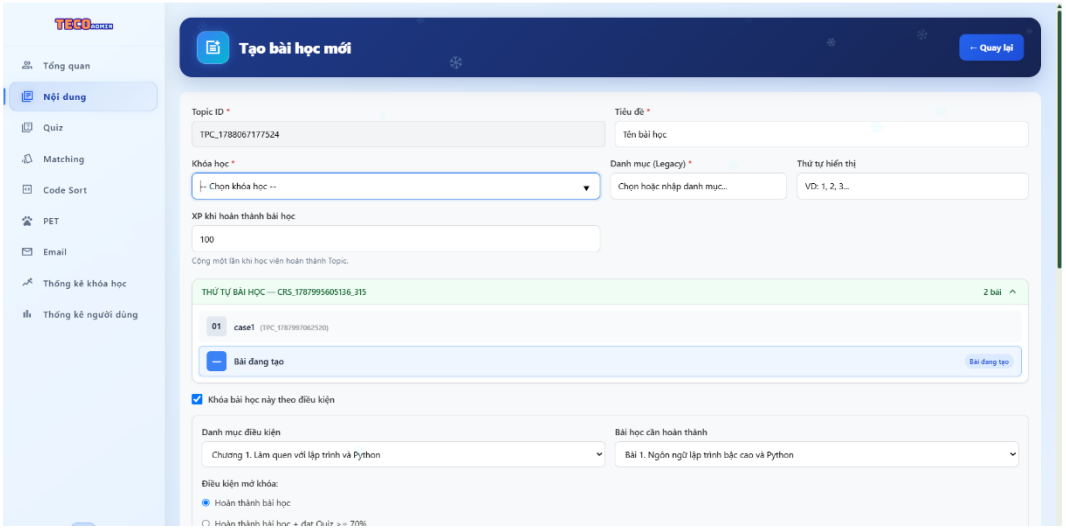

Figure A.11 Topic Creation Interface

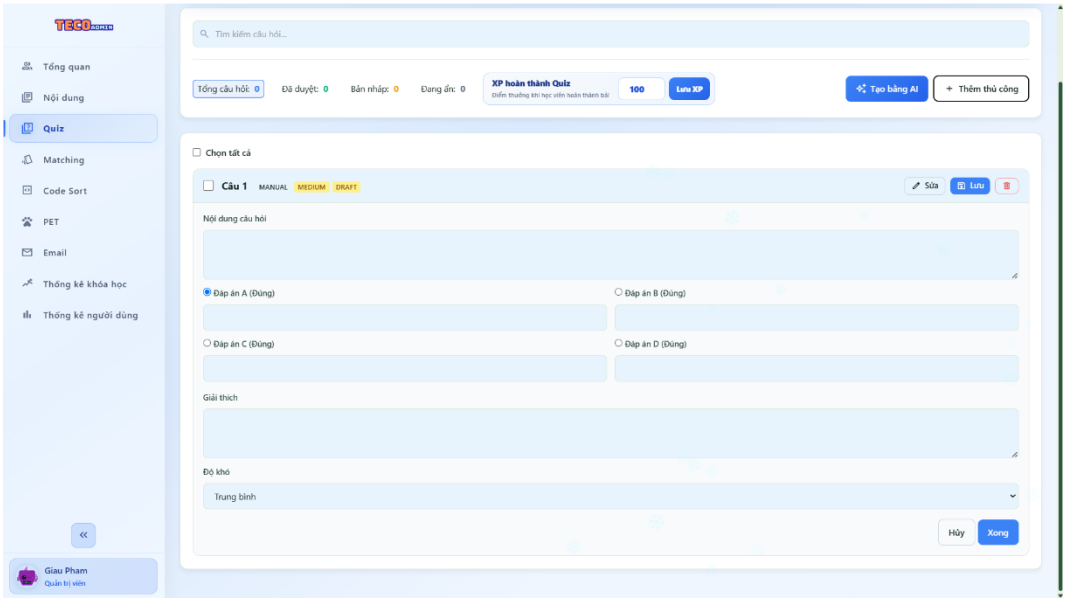

Figure A.12 Quiz Question Creation Interface

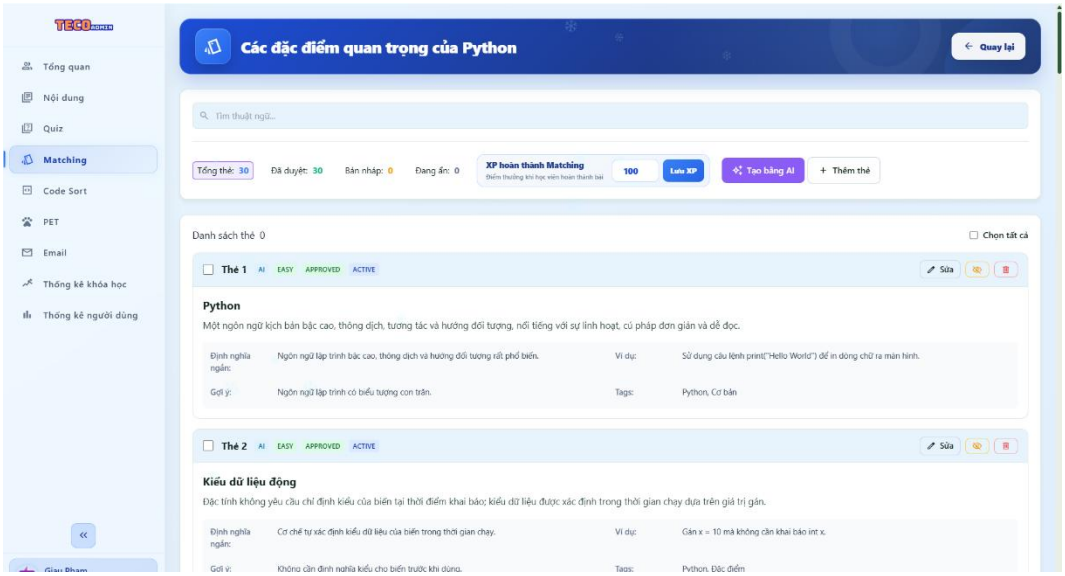

Figure A.13 Matching Game Content Creation Interface

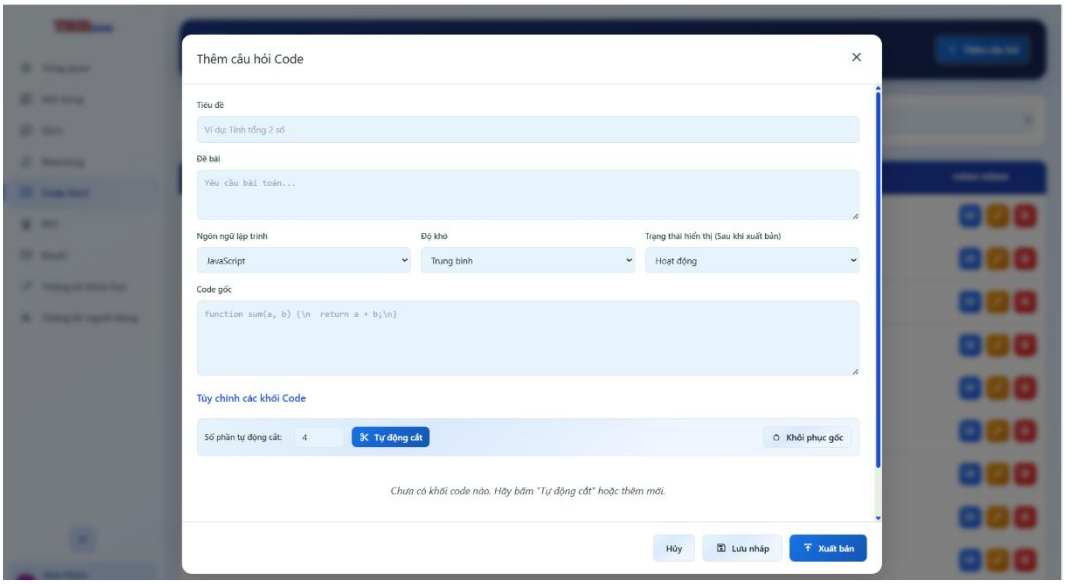

Figure A.14 Code Arrangement Exercise Creation Interface

Figure A.15 Admin Pet Accessory Management Interface

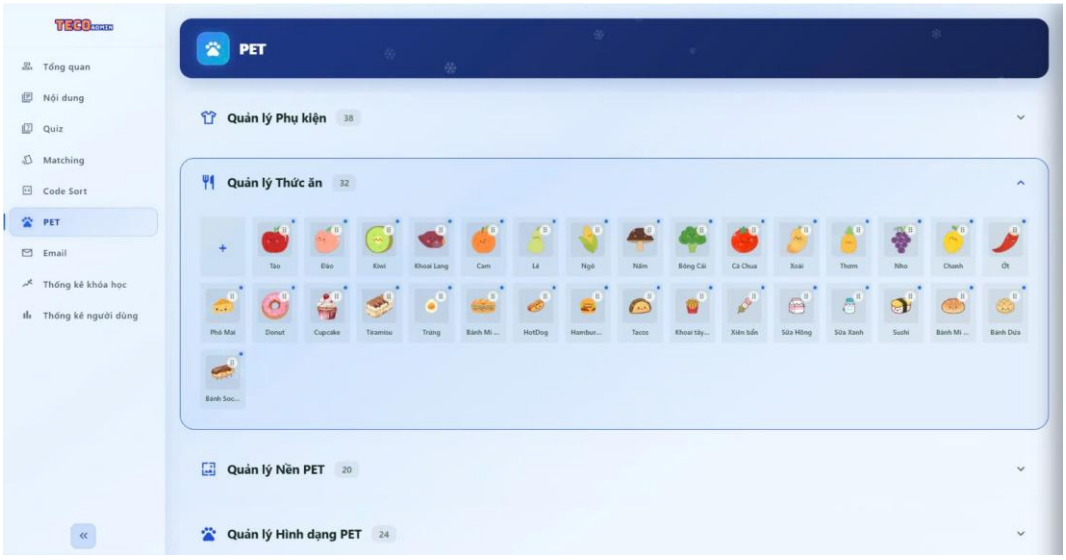

Figure A.16 Admin Pet Food Management Interface

Figure A.17 Admin Pet Background Management Interface

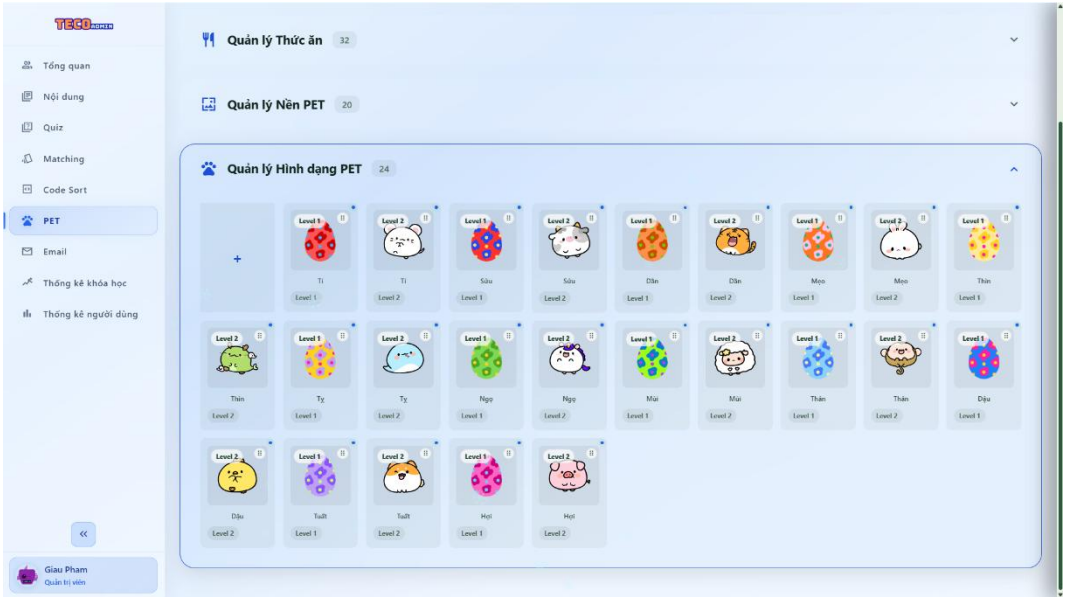

Figure A.18 Admin Pet Variant Management Interface

APPENDIX B. SYSTEM ACCESS AND DEMONSTRATION

B.1. System Access Information

The following accounts are used for system testing and function demonstration:

- **User 1**
  - Account: testcasenumber1111@gmail.com
  - Password: Testcase1111
- **User 2**
  - Account: testcasenumber2222@gmail.com
  - Password: testcasenumber2222
- **Admin**
  - Account: phamvangiau3834@gmail.com
  - Password: Giau@3834

B.2. TerraCode System Demo Link

https://script.google.com/macros/s/AKfycbyy_1IB7W9V85Dr6brcFFiyPi2tjCnJB2NjFTr6PAEqiUZPnc5vWXZZUMVk_OywIvRZXQ/exec

Note: Please sign in to the system using the same Google account currently used in your browser. If another account is signed in and an error occurs, open the link in an **Incognito window** and sign in again with the correct Google account.

B.3. TerraCode System Demonstration Video

- **Google Drive:**

https://drive.google.com/drive/folders/1-GrOPp1wMJEtnbyvxCjzZUaBYzaGw3xJ?usp=sharing

- **Youtube:**

https://youtu.be/rSpjeKNTixw?si=Bo63nwsZkeUjDjiX